



A priority-based congestion control algorithm for on-road traffic in smart cities

by

[REDACTED]

([REDACTED])

*A dissertation submitted in partial fulfilment of the requirements for the
degree*

MASTER OF COMPUTING: INFORMATION TECHNOLOGY

in the

Department of Information Technology

**FACULTY OF INFORMATION AND COMMUNICATION
TECHNOLOGY**

at

TSHWANE UNIVERSITY OF TECHNOLOGY

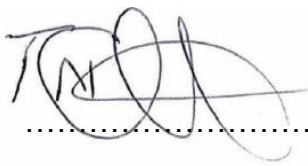
Supervisor: Prof. D.P. du Plessis

Co-Supervisors: [REDACTED]

January 2025

Declaration

I, Ndivhuho Tshilongamulenzhe, hereby confirm that this research, titled: A priority-based congestion control algorithm for smart cities, submitted for the degree of Master of Computing: Information Technology at Tshwane University of Technology, is my research and has not been submitted to any other institution of higher learning. Furthermore, all articles quoted are indicated and acknowledged by means of a comprehensive list of references.



.....

Signature

...31 Jan 2025.....

Date

Dedication

This research study is dedicated to my loving family; my mother, siblings, and my children, who guided, supported, and encouraged me to further my studies.

Acknowledgments

First and foremost, I would like to thank Almighty God, who gave me the power and guidance to finish this research study. I also wish to express my heartfelt gratitude for the kindness of my supervisors, Prof. D P du Plessis, Prof. T E Mathonsi, and Mr L Ndlovu, who contributed much to my completion of this research study. My supervisors gave me professional advice, such as focusing on the research topic; and provided me with good feedback on my research study. Lastly, I would like to thank Dr T M Tshilongamulenzhe, Livhuwani Ramanugu, Ntshuxeko Makondo, and my colleagues for their willingness to support and motivate me during the period of this research study.

Publications

Parts of this research study have been submitted and accepted for publication in the following journals and conference proceedings:

Conference proceedings:

1. Ndivhuho Tshilongamulenzhe, Topside E Mathonsi, Deon P Du Plessis, and Lungisani Ndlovu: "A Priority-based Congestion Control algorithm for on-road traffic in smart city." Presented at the 2024 International Conference on Computational Science and Computational Intelligence (CSCI'24), December 11-13, 2024, Luxor - an MGM property, Las Vegas, USA.

Journals:

1. Ndivhuho. Tshilongamulenzhe, Topside E. Mathonsi, Deon P. Du Plessis, and Lungisani Ndlovu, "Fusion-based Intelligent Congestion Control algorithm for on-road traffic in smart cities" **Still in the Process.**

Abstract

The concept of wireless sensor networks (WSNs) has generated significant attention on a global scale. According to the IEEE 802.11 standard, WSNs are wireless networks composed of sensor nodes (SNs) that interconnect with one another via wireless links. SNs can sense, process, and communicate wirelessly in a harsh environment. WSNs are primarily used for traffic communication in a wide range of environments such as smart cities, healthcare, homes, and military areas. The implementation of WSNs in various environments such as smart cities come with various challenges, such as traffic congestion. The traffic congestion that occurs on the roads within a smart city during peak hours is normally caused by numerous vehicles travelling at the same time. As a result, traffic jams, time delay, and accidents occur during traffic congestion under varying weather conditions, such as sunny days and rainy days. This research study proposed a new algorithm called the priority-based congestion control (PBCC) algorithm, which is designed by integrating the fusion-based intelligent traffic congestion control (fitcc) algorithm and traffic Congestion Detection (TCD) in order to reduce on-road traffic congestion within smart cities. This research study used Network Simulator 3 (NS-3) to evaluate the effectiveness of the proposed PBCC algorithm when compared with the FITCC and the TCD algorithms. During sunny days, the NS-3 simulation results demonstrated that the proposed PBCC algorithm has an average of a 0.83% false alarm rate (FAR), a 76.0% mean time to detection (MTTD), and an 84.3% detection rate (DR) improvement over the FITCC and the TCD. The NS-3 simulation results during rainy days demonstrated that the proposed PBCC algorithm has an average of 14.09% FAR, 57.03% MMTD, and 78.04% DR improvement over the FITCC and the TCD within smart cities.

Keywords: Traffic Congestion, WSNs, Network Simulator 3, Traffic Congestion Detection, Fusion-based Intelligent Traffic Congestion Control, Priority-based Congestion Control.

Table of Contents

Declaration	i
Dedication.....	ii
Acknowledgments	iii
Publications	iv
Abstract.....	v
Table of Contents	vii
List of Figures	ix
List of Tables.....	xi
List of Algorithms.....	xii
Glossary.....	xiii
Chapter 1: Introduction	1
1.1. Background to the Study	1
1.2. Motivation of the Study.....	3
1.3. Problem Statement.....	4
1.4. Research Questions	5
1.5. Research Aim and Objectives.....	5
1.6. Significance and Benefits of the Study	6
1.7. Research Methodology	6
1.8. Contributions of the Study	8
1.9. Ethical Considerations	8
1.10. Structure of this Dissertation	8
1.11. Chapter Summary.....	9
Chapter 2: Literature Review.....	10
2.1. Introduction	10
2.2. Overview of Wireless Sensor Networks	10
2.3. Design of WSNs in Smart Cities.....	12
2.4. WSNs Protocols for Smart City Applications	19
2.5. Overview of Traffic Congestion in WSNs	28
2.6. Techniques Used for Traffic Congestion in WSNs	30
2.7. Related Works	34
2.8. Chapter Summary.....	41
Chapter 3: System Architecture and Design.....	42
3.1. Introduction	42
3.2. System Architecture	43

3.3.	Fusion-based Intelligent Traffic Congestion Control (FITCC) and Traffic Congestion Detection (TCD) Algorithms	45
3.4.	Design of Priority-based Congestion Control (PBCC) Algorithm	50
3.5.	Chapter Summary.....	53
Chapter 4: System Implementation		54
4.1.	Introduction	54
4.2.	Overview of Wireless Network Simulators	54
4.3.	The Network Simulator 3 Environment.....	64
4.4.	Simulation of the Priority-based Congestion Control (PBCC) Algorithm.....	68
4.5.	Summary	75
Chapter 5: Results and Discussion		76
5.1	Introduction	76
5.2	Computer Experimental Evaluation	77
5.3	Findings and Discussion.....	77
5.3.1	Scenario 1: Performance evaluation for sunny days	79
5.3.2	Scenario 2: Performance evaluation for rainy days	82
5.4	Chapter Summary.....	85
Chapter 6: Conclusion and Future Work		87
6.1	Introduction	87
6.2	Aim and Objectives Achievement.....	87
6.3	Research Summary.....	90
6.4	Challenges Encountered	92
6.5	Future Work	92
6.6	Summary	93
References		94
APPENDIX A: TABLE OF NOTIONS.....		113
APPENDIX B: TCL SCRIPT		115
APPENDIX C: PYTHON CODE		118
APPENDIX D: R CODE.....		120

List of Figures

Figure 2. 1: Smart city wireless sensor network structure (Medeiros et al., 2022)	11
Figure 2. 2 :Tree topology structure (Jain et al., 2022)	15
Figure 2. 3 : Ring topology structure (Agrawal et al., 2022)	30
Figure 2. 4: Hybrid topology structure (De Beelde et al., 2021)	17
Figure 2. 5: Mesh topology structure (Amodu et al., 2021).....	18
Figure 2. 6: IEEE 802.15.4 protocol stack	204
Figure 2. 7: IEEE 802.16 WiMAX protocol.....	21
Figure 2.8: IEEE 802.11a protocol.....	22
Figure 2. 9: IEEE 802.11b protocol.....	24
Figure 2. 10: IEEE 802.11g protocol.....	25
Figure 2. 11 : Satellite communication Infrastrucuture	40
Figure 2. 12: Link-level congestion (Yakymchuk et al., 2022).....	29
Figure 2. 13: Node-level congestion (Zhang et al., 2022).....	30
Figure 3. 1: Proposed solution architecture	44
Figure 3. 2: PBCC algorithm flowchart	51
Figure 4. 1: NS-3 simulation interface	70
Figure 4. 2: NS-2 simulation interface	71
Figure 4. 3: QualNet simulation interface.....	72
Figure 4. 4: OMNeT++ simulation interface.....	73
Figure 4. 5: MATLAB simulator interface	74
Figure 4. 6: Basic NS-3 architecture.....	78
Figure 4. 7: NS-3 directory structure.....	79
Figure 4. 8: NS-3 C++ hierarchy	80

Figure 4. 9: Smart city NAM simulation scenario 1	81
Figure 4. 10: Smart city NAM simulation scenario 2.....	84
Figure 4. 11: Smart city NAM simulation scenario 3.....	85
Figure 5. 1: Average detection rate.....	93
Figure 5. 2: Average false alarm rate	94
Figure 5. 3: Average mean time to detection.....	95
Figure 5. 4: Average detection rate.....	96
Figure 5. 5: Average false alarm rate	97
Figure 5. 6: Average mean time to detection.....	99

List of tables

Table 1.1: Research Questions, Technical Objectives, and Research Methods	19
Table 2.1: Summary of WSNs Protocols for Smart City Applications	46
Table 4.1: Advantages and Disadvantages of Network Simulators.....	73
Table 4.2: Comparison of Network Simulators.....	75
Table 4.3: Simulation Parameters.....	84
Table 4.4: Dataset Structure Features.....	85

List of algorithms

Algorithm 3.1: PBCC algorithm **Error! Bookmark not defined.**

Glossary

AP(s)	Access Point(s)
ATLCS	Adaptive Traffic Light Control System
BSs	Base Stations
CARF	Congestion-Aware Routing using Fuzzy Rule-sets
CSMA/CA	Carrier Sense Multiple Access/Collision Avoidance
DSSS	Direct Sequence Spread Spectrum
ECFM	Fuzzy-based Congestion Mitigation
EE-FBCDCA	Energy Efficient Buzzy-based Congestion Detection and Control Algorithm
EVs	Emergency Vehicles
EVPS	Emergency Vehicle Priority System
FBACC	Fuzzy-based Adaptive Congestion Control
FDD	Frequency Division Duplex
FDMA	Frequency Division Multiple Access
FITCC	Fusion-based Intelligent Traffic Congestion Control
FITCCS	Fusion-based Intelligent Traffic Congestion Control System
FSMC	Fuzzy Sliding Mode Congestion Control
FHSS	Frequency Hopping Spread Spectrum
GPS	Global Positioning System

HCGM	Hop Count-based Geometric Method
HTCCFL	Hierarchical Tree-based Congestion Control using Fuzzy Logic
IoT	Internet of Things
IPv6	Internet Protocol version 6
IR	Infrared
ISO	International Organization for Standardization
ITMS	Intelligent Traffic Management Systems
ITU-T	International Telecommunications Union – Telecommunication
LCLD	Low-Cost Locations Discovery
LER	Large Error Rejection
LLC	Logical Link Control
ML	Machine Learning
MATALAB	Matrix Laboratory
MAC	Media Access Control
NS-2	Network Simulator-2
NS-3	Network Simulator-3
OBUs	On-Board Units
OptNet	Optimized Network Engineering Tools
OWENN	Optimized Weight Elman Neural Network
OFDM	Orthogonal Frequency-Division Multiplexing

PATSC	Priority-based Adaptive Traffic Signal Control
PBCC	Priority-based Congestion Control
RSUs	Road-Side Units
SNs	Sensor Nodes
TCP	Traffic Congestion Detection
TDD	Time Division Duplex
TDMA	Time Division Multiple Access
UAVs	Unmanned Aerial Vehicles
USPBR	UAV Same Position Broadcast Repeat
VNs	Vehicular Networks
WSNs	Wireless Sensor Networks
WLAN	Wireless Local Area Network
WEP	Wired Equivalent Privacy
Wi-Fi	Wireless Fidelity

Chapter 1: Introduction

1.1. Background to the Study

The utilization of Wireless Sensor Networks (WSNs) has become a revolutionary technique of data collection used every day worldwide. WSNs are normally deployed in smart cities, healthcare, home automation, industrial monitoring, and in the military to improve the network infrastructure (Kapil & Kumar, 2019). WSNs consists of sensor nodes (SNs) which are used for the collection of data from various environments and transmit it to the designated location such as a base station (BS) (Toor & Jain, 2019; Khalifeh et al., 2020; Nemer et al., 2021). However, SNs are primarily powered by batteries; and it is challenging to recharge them once they have been depleted. As a result, WSNs often face energy-consumption issues (Gupta, 2018; Sharma et al., 2019). SNs can sense, process, transmit data over a wireless channel in a complex environment and transmit the data using wireless channels from a source node to the destination node. This data is normally stored in the cloud, and the network users can access it using various devices (Arroyo et al., 2019; Jia, 2020).

In this research study, WSNs will be implemented in smart cities to measure the on-road traffic congestion caused by the high volume of vehicles travelling from one point to another. Suppose there is a high number of vehicles moving in a certain area of a smart city. In that case, the SNs will gather the data by detecting vehicle-traffic levels on the road, sending the data to the BS, and from the BS to the cloud so that the data can be analysed.

This research study outlines the traffic congestion that occurs in a smart city when there are many vehicles travelling from one point to another, with concomitantly increased numbers of traffic accidents. During peak hours there is more vehicle

movement which leads to time delays, noise pollution, and wasted fuel, increasing air pollution. Furthermore, not only does traffic congestion have a harmful effect on the ecology, cause air and noise pollution, and impact the quality of life, it also results in more travel time and fuel consumption. The time wasted in traffic impacts productivity and has negative consequences for the economy and society.

Saleem et al. (2022) stated that the latest traffic index has revealed that traffic congestion within the smart cities of South Africa has steadily increased over the years. This includes Johannesburg, Pretoria, Lanseria, Cape Town, and many others, making Pretoria city one of the most congested cities. In addition, there are limitations based on traffic congestion in smart cities which include parking space, traffic jams, illnesses due to the frustration and stress of the drivers, blocked lanes caused by accidents, and unexpected spikes in traffic (Hilmani et al., 2020; Ahmed & Fragonara, 2021; Asha et al., 2023). The above authors have demonstrated that traffic congestion has become one of the main challenges in smart cities.

Hence, in this research study, the proposed algorithm will set the priorities based on the collected data by SNs to avoid traffic congestion within a certain area in a smart city. A notification will then be sent to all drivers within the smart city radius, informing them of the traffic jam, and offering them a different route that is congestion-free. Vehicles can then avoid using areas suffering from traffic congestion. The input parameters that will be implemented in this research study include traffic volume, traffic density, occupancy, the traffic congestion index, and travel time, while monitoring and predicting traffic congestion using SNs in smart cities.

Thereafter, this research study will develop a priority-based congestion control (PBCC) algorithm by integrating the fusion-based intelligent traffic congestion control (FITCC) algorithm proposed by (Bharti et al., 2022) and the traffic congestion detection (TCD)

algorithm proposed by (Bawaneh & Simon, 2023) to reduce traffic congestion caused by the increased number of vehicles within smart cities. The FITCC algorithm which was proposed by (Bharti et al., 2022) demonstrated that their proposed algorithm managed to collect traffic data, thus rerouting the traffic to available routes. This alleviated traffic congestion which causes air and noise pollution in smart cities. Furthermore, the proposed algorithm makes use of machine learning (ML) techniques to allow drivers to view remotely the traffic flow and the volume of vehicles available on the road, thereby avoiding traffic jams. The TCD algorithm proposed by (Bawaneh & Simon, 2023) demonstrated that it can detect possible traffic congestion before it occurs based on the traffic features such as speed, occupancy, and flow. Their proposed TCD algorithm achieves this process because it first cleans and pre-processes the traffic data collected in the areas within the smart cities. Thereafter, it computes the absolute of the first derivative of each sample in the traffic feature to determine its anomaly score. Hence, the PBCC algorithm proposed by this research study will use a real-time process when collecting traffic data. The process will route the traffic on available routes as demonstrated by the FITCC algorithm, displaying the detection of possible traffic congestion as demonstrated by the TCD algorithm.

1.2. Motivation of the Study

The population of people is increasing, leading to more vehicles being purchased and more traffic congestion on the roads. Traffic congestion has a harmful effect on the environment, in adding to air and noise pollution. In addition to having a negative impact on the economy and society, traffic congestion results in more travel time and fuel consumption. Managing the traffic data on vehicles within a smart city is one of the key elements. This is because delays in updating traffic information on congestion on the roads within smart cities will result in more air and noise pollution. Hence, this research study proposes to develop an algorithm which will reduce traffic

congestion using WSNs in smart cities. There will thus be a smoother flow of vehicles. This research study improves the research skills and produces a solution that can be used in the telecommunication industry to ameliorate vehicle traffic flow and decrease congestion.

1.3. Problem Statement

In recent years, the implementation of WSNs within smart cities has grown rapidly. These networks perform various processes, including collecting data for monitoring purposes. Over the past years, the use of vehicles has climbed dramatically worldwide; this is due to urbanization, increased population, and economic development (Wang & Zhou, 2022, Tanveer et al., 2020). Smart cities repeatedly experience traffic congestion whenever there is a large volume of vehicles, and this affects the daily life routine of road users, increasing the congestion rate, accident levels, and time delay (Saleem et al., 2022; Jayakumar et al., 2021).

Furthermore, robot malfunction within smart cities increases congestion rate and time delay, leading to air and noise pollution, traffic density, and travel time (Baniata & Hong, 2017). This is because when traffic is congested, vehicles are forced to idle or travel at slower speeds. This can lead to higher emissions of pollutants such as carbon monoxide, nitrogen oxides, and particulate matter. This research study therefore aims to improve the transportation system implemented within smart cities to avoid traffic congestion caused by the high volume of vehicular travel. The proposed algorithm in this research study will set the priorities based on the collected data to avoid traffic congestion. The road users will then be notified and rerouted to the lane which is congestion free within the smart city.

1.4. Research Questions

The primary goal of this research study is to provide an answer to the following question: How can one develop an algorithm that will reduce traffic congestion caused by the high volume of vehicles within a smart city?

The sub-questions are as follows:

1. What are the challenges and limitations of the current algorithms in WSNs used for traffic congestion within a smart city?
2. How can the proposed PBCC algorithm be designed for WSNs to reduce traffic congestion within a smart city?
3. How will the proposed PBCC algorithm be implemented in WSNs to reduce and control traffic congestion within a smart city?
4. What is the effectiveness of the PBCC algorithm compared with the current algorithms?

1.5. Research Aim and Objectives

The aim of this research study is to develop an algorithm that will reduce and control the traffic congestion caused by a high volume of vehicles within smart cities.

The research sub-objectives are as follows:

1. To identify and analyse the current algorithms used in WSNs for traffic congestion within a smart city.
2. To design a PBCC algorithm in WSNs that reduces traffic congestion within a smart city.
3. To implement the PBCC algorithm in WSNs that reduces and controls traffic congestion within a smart city, thereby improving traffic flow in the city.
4. To evaluate the effectiveness of the PBCC algorithm compared with the current algorithms.

1.6. Significance and Benefits of the Study

This research study will benefit people who normally use smart city roads when travelling from one point to another.

The two areas of benefit in this research study are:

1. To reduce congestion rate, accidents, and time delay: the designed PBCC algorithm for WSNs will reduce the traffic congestion in the smart city roads.
2. The study can be treated as a learning curve for people who are interested in algorithms used on WSNs within a smart city to reduce traffic congestion.

1.7. Research Methodology

The experiential methodology that was employed in this research study to achieve the overall goal is divided into the following methods:

1.7.1. Literature Review

A comprehensive literature study of related works was conducted on academic journals, conference proceedings, and internet sources to gain a better understanding of what other researchers have done.

1.7.2. Modelling

Modelling was used as a guideline in designing and analysing the strategy of the proposed algorithm. Besides that, a mathematical model was presented.

1.7.3. Implementation

After designing and developing the proposed algorithm using mathematical models, a code of the proposed algorithm was written; thus, the effectiveness of the proposed algorithm could be evaluated using a network simulator.

1.7.4. Simulation

A code of the proposed PBCC algorithm was evaluated using Network Simulator 3 (NS-3). An NS-3 simulation was used in this research study because it presents a widespread simulation environment for network applications, traffic models, protocols, and various network types. NS-3 is an open-source code and is freely obtainable on the internet. NS-3 is considered helpful in studying and examining the dynamic nature of WSNs, because it is a simple, event-driven simulation tool. Furthermore, NS-3 allows for add-ons, modification of protocols, and implementation of multiple interfaces on a mobile node. This research study presents the alignment of research questions, technical objectives, and research methodology using Table 1.1.

Table 1.1: Research Questions, Technical Objectives, and Research Methods

Research Questions	Technical Objectives	Research Methods
What are the challenges and limitations of the current algorithms used in WSNs for traffic congestion within a smart city?	To identify and analyse the current algorithms used for traffic congestion in WSNs within a smart city.	Literature Review
How can the proposed PBCC algorithm in WSNs be designed to reduce traffic congestion within a smart city?	To design a PBCC algorithm in a WSN that reduces traffic congestion within a smart city.	Modelling
How will the proposed PBCC algorithm in WSNs be implemented to reduce and	To implement the PBCC algorithm in a WSN to reduce and control traffic	Implementation

control traffic congestion within a smart city?	congestion within a smart city thereby improving traffic flow in the city.	
What is the effectiveness of the PBCC algorithm compared with the current algorithms?	To evaluate the effectiveness of the PBCC algorithm compared with the current algorithms.	Simulation

1.8. Contributions of the Study

The major contributions of this research study compared with other related algorithms are structured as follows:

1. Developing a PBCC algorithm using WSNs that will manage traffic congestion while improving vehicular movement in smart cities.
2. Designing a PBCC algorithm using WSNs that selects the most cost-effective path for vehicles in smart cities.
3. Presenting the proposed PBCC algorithm for WSNs that will give differing priorities to vehicular movement in smart cities.

1.9. Ethical Considerations

No ethical clearance was required for this research study. This is because the data used in the study did not involve either directly or indirectly any human or animal information; data comprised purely a computer algorithm and simulations.

1.10. Structure of this Dissertation

The remainder of this research study is structured as follows:

In Chapter 2, this research study provided a review of relevant literature, which includes: WSNs overview, and related work. The researchers in this study evaluated all the information obtained from various readings and showed how the information

applies to this study. The research compared the views of various authors that implemented WSNs in smart cities, discussing the findings.

1.11. Chapter Summary

In this chapter, the background study of WSNs, the motivation, and the mechanism that outlines the importance of implementing WSNs into smart city, were discussed. Furthermore, the challenges of traffic congestion based on a high volume of vehicles within a smart city were also demonstrated. The increase of vehicles within the smart city caused high traffic congestion, which resulted in delays, low packet delivery ratio, and throughput impairment. To support the proposed PBCC algorithm, a set of research questions has been defined, which was answered by implementing some technical objectives. The research study adopted the experimental methodology and used the following methods: literature review, modelling, implementation, and simulation, to address the identified research problem. A comprehensive literature review and related work are provided in the next chapter of this research study.

Chapter 2: Literature Review

2.1. Introduction

WSNs are large-scale networks made of self-configured entities comprising a set of sensors that normally permits nodes to connect and share information through wireless signals (Akhtar & Feng, 2022; Subramani et al., 2023). WSNs continue to expand, thus experience increasingly high volumes of various sensing devices. Devices compete for the available services and resources within different environments, as demonstrated in Section 1.1 of Chapter 1 (Singh et al., 2022; Basheer & Itani, 2023; Kapil & Kumar, 2019). Furthermore, the WSNs contain several sensors positioned together in order to provide better communication when transmitting sensed information or data from one point to another. This chapter presents a comprehensive literature review of WSNs.

The remainder of Chapter 2 is organized as follows: In Section 2.2, the overview of WSNs is presented. In Section 2.3, the design of WSNs in smart city is provided. The WSNs protocols for smart city applications are discussed in Section 2.4. In Section 2.5, the overview of traffic congestion in WSNs is presented. In Section 2.6, the techniques used for traffic congestion in WSNs are discussed. In Section 2.7, related work is presented. Lastly, In Section 2.8, the chapter summary is presented.

2.2. Overview of Wireless Sensor Networks

WSNs refer to wireless networks that consist of spatially dispersed and dedicated sensors that monitor and record the physical conditions of the environment, forwarding the collected data to a central location, as demonstrated in Figure 2.1 (Toloueiashtian et al., 2022; Vijayalakshmi et al., 2023). WSNs can measure environmental conditions such as traffic density, sound, noise pollution, humidity, temperature, among many

others, depending on the functionality of the sensors placed within a specific environment (Raghuwanshi & Chaudhary, 2022). Furthermore, these sensors can transfer data from the SNs which are nodes that collect data from the smart city roads. The nodes are interconnected with the road-side units (RSU) transceiver, which is mounted on the road, or the pedestrian passageway and the On-Board Unit (OBU), installed in a vehicle for traffic data recording. Thereafter, the data is transferred to the central devices. The central devices are the nodes that receive the data from the SNs such as the sink node and base station (BS).

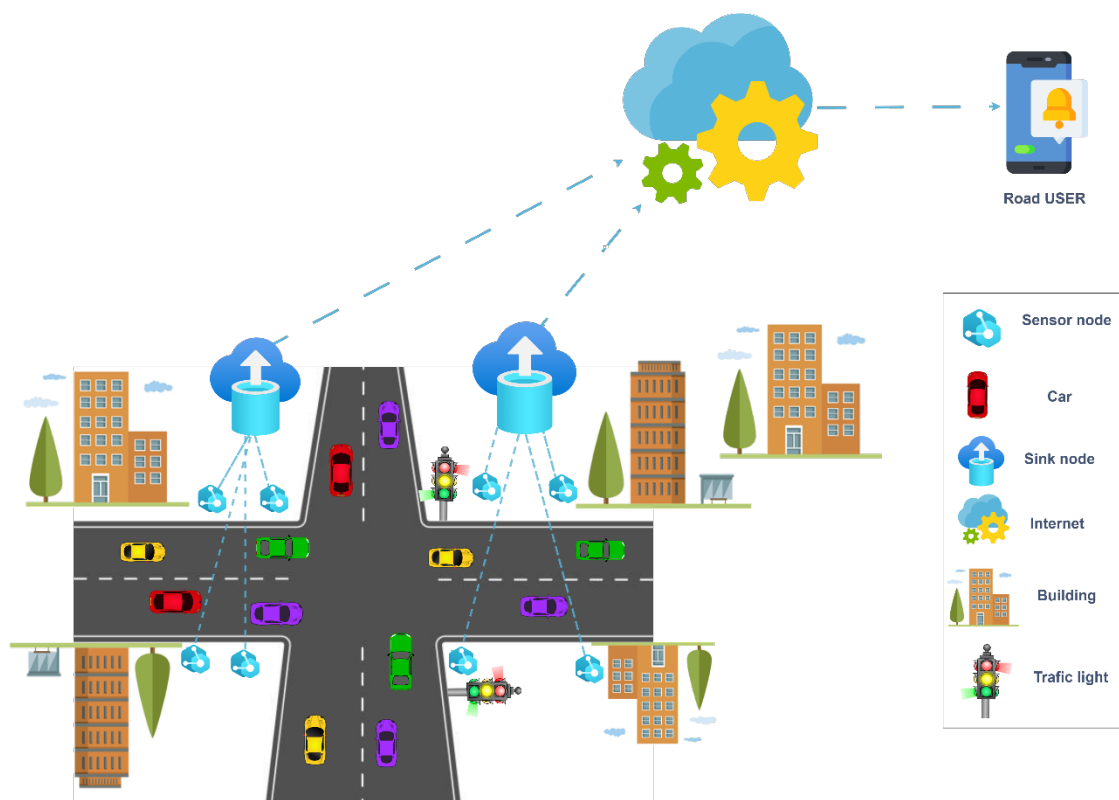


Figure 2. 1: Smart city wireless sensor network structure (Medeiros et al., 2022)

Figure 2.1 shows that SNs are placed near the smart city roads where there is movement of vehicles, allowing the sensors to collect data (Medeiros et al., 2022; Banerjee et al., 2022). These SNs placed near the smart city roads have the capability

to sense, collect, and transfer data from one node to another. Thereafter, the sink node receives the data from the SNs and transferred to the internet. There it is stored on the management node which can be servers. Users will then be able to access the data via the internet. All SNs share a single communication channel (transmission channel) through multiple access protocols: for example, random access protocols (protocols used on a transmitting node, to transmit the data always at the full rate of the channel); and a media access control protocol (protocol used for coordinating the access from active nodes).

2.3. Design of WSNs in Smart Cities

There are various structures that design WSNs in smart cities, which include different types of topologies and communication channels (Khalifeh et al., 2021; Abdulwahid & Mishra, 2022). Furthermore, there are sets of nodes (wireless nodes) within the design of WSNs which are all equipped with various sensors such as RSU, OBU, sink node, source node, and a transmission channel (Sharma et al., 2021). Therefore, when the nodes are connected to transmit data, they face the challenge of traffic congestion whenever there is a high volume of vehicles within the smart city. This research study demonstrated the most fundamental factors that influence the design of WSNs implemented into smart cities.

2.3.1 Key fundamentals shaping WSNs design for smart city applications

There are several key fundamentals that exert a profound influence on the design of WSNs within the context of smart cities. These fundamental types are enumerated below (Yousif et al., 2021).

- i) Fault tolerance: a process that enables an operating system to respond to a failure in hardware or software (Kumari & Kaur, 2021). Fault tolerance further has the ability to continue operating despite failures or malfunctions,

retaining the sensor network operating without any interruption when some nodes in a network fail (Naim et al., 2022). The fault tolerance system should be capable of examining faults within the smart cities, in particular, software or hardware components, failures of power, or other varieties of unexpected adversities, still fulfilling its specification.

- ii) **Energy efficiency:** Smart city WSNs often operate on battery power and need to be extremely energy-efficient to ensure a long lifespan (Xu et al., 2019; Sharma et al., 2021; Agrawal et al., 2022). This requires careful consideration of sensor technology, communication protocols, and network topology to minimize energy consumption.
- iii) **Scalability:** This refers to the number of nodes that are enabled to be acceptable to any increase or decrease within the wireless network implemented in the smart cities, so that it can meet the environmental needs (Barthwal & Verma, 2021). Furthermore, to achieve better scalability in a wide range of WSNs, specifically within the smart cities, the clustering approach is recommended. In this approach, sensor nodes are assembled into clusters (Gupta et al., 2021; Barthwal & Verma, 2021; Sharma & Verma, 2022).
- iv) **Reliability:** Smart city WSNs must be highly reliable to ensure the smooth operation of critical infrastructure and services (Barthwal & Verma, 2021; Taami et al., 2023). This involves implementing fault-tolerant mechanisms, redundant nodes, and self-healing capabilities to ensure system resilience (Jing et al., 2023).
- v) **Security:** Smart city WSNs collect and transmit sensitive data, making security of paramount concern (Huang et al., 2022; Majid et al., 2022). Robust encryption algorithms, secure authentication protocols, and key

management mechanisms are essential to protect against unauthorized access and data breaches (Arieth et al., 2022).

- vi) Topology: This is a technique of defining the connections between nodes in order to reduce the interference between them. The selected topology is able to save energy and extend the network lifetime (Hajjee et al., 2021). Topology in WSNs further plays a vital role in minimizing various constraints, such as latency, computational resource crisis, and quality of communication (Jubair et al., 2021; Bekal et al., 2023). The arrangement of high numbers of nodes requires high care of topology maintenance in a smart city.
- vii) Environmental sustainability: Smart cities strive to be environmentally sustainable, requiring WSN designs that minimize their environmental impact (Niu et al., 2021; Yousif et al., 2021; Amoordon et al., 2022). This involves using environmentally friendly materials, optimizing energy consumption, and minimizing waste generation.
- viii) Traffic: This feature demonstrates the amount of data transferred from the sensor node to the central devices. Such can be either a sink node or a BS within WSNs in the smart cities (Moridi et al., 2020). The traffic for hierarchical methods in smart cities is higher than in other frameworks due to the fusion and integration of data. The higher integration of data transmission leads to less network traffic congestion (Rajan et al., 2023).

This research study will consider the above key fundamentals that shape the design of WSNs during the system implementation within the smart city. These key fundamentals enhance the functionality of the network infrastructure through different elements such as topology, traffic, scalability, and reliability which are implemented in the smart city environment.

2.3.2 WSNs topologies for smart city applications

WSN topologies play a vital role in networks, allowing the components or nodes to be arranged in a different form (Ghazal et al., 2021; Jamshed et al., 2022). Topologies describe both the physical layout of nodes and gateways, and the paths that data follows between them (Jing et al., 2023). There are various types of topologies that are implemented into a smart city. Some of the examples of topologies which are implemented are listed below:

- i) **Tree Topology:** This is a type of network topology in which nodes resemble the shape of a tree: each node is related to the others in a hierarchy (Mazumdar et al., 2021). In WSNs, tree topologies are commonly used to arrange data in the databases and workstations within the networks (Jain et al., 2022). Furthermore, the tree topology structure combines the features of both bus and star topologies (See Figure 2.2).

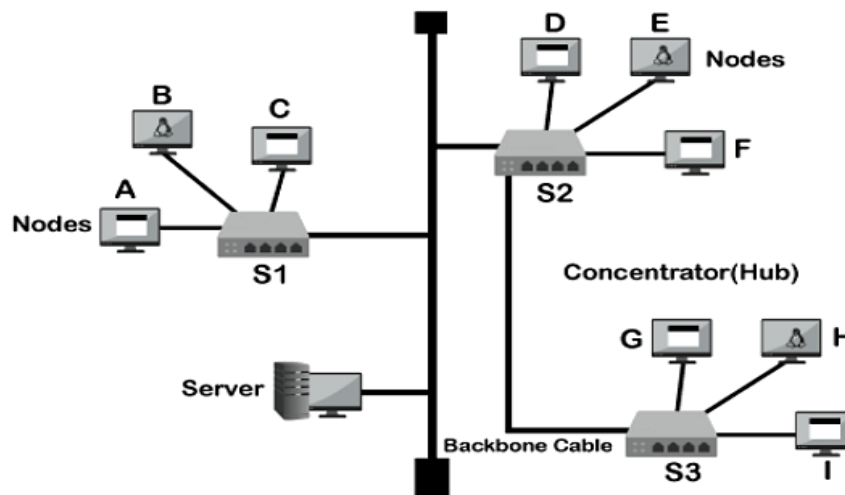


Figure 2. 2 :Tree topology structure (Jain et al., 2022)

As demonstrated in Figure 2.2, the data is transmitted through the transmission cable, which is in the form of a bus topology, to the whole network, as the information passes one-to-one between nodes (Li et al.,

2021). In a tree topology, there is one central node (trunk); each node is connected to the central node through a single path (Lin et al., 2020; Hasheminejad & Barati, 2021). The drawback of this topology is that it contains less reliable information than other topologies such as mesh or star, there being more potential points of failure. This research study does not consider using tree topology during the implementation.

- ii) Ring Topology: This is a network topology in which every node has a direct connection to exactly two nodes, creating a single continuous path for data to travel through each node (Xu et al., 2019; Agrawal et al., 2022). Furthermore, this topology is normally used in the telecommunication industry for data-transmission purposes, in which it has been utilized in SONET (synchronous optical network) fibre networks (See Figure 2.3).

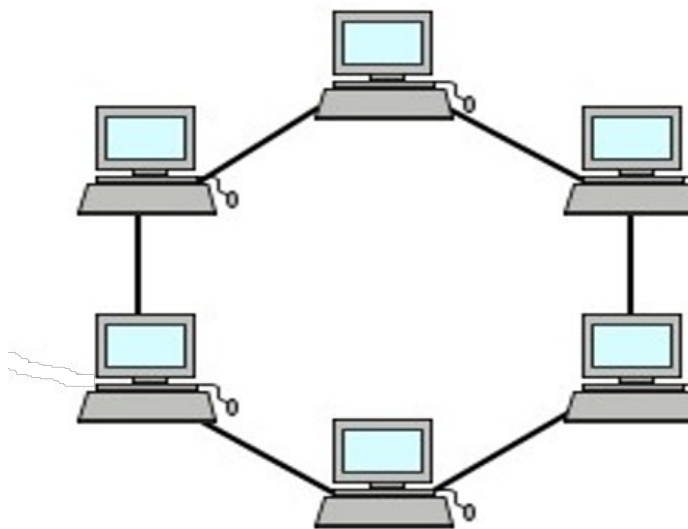


Figure 2. 3 : Ring topology structure (Agrawal et al., 2022)

The nodes in the ring topology are configured with equal priorities to all nodes for data transmission (Yi et al., 2021). The ring topology was further demonstrated by (Taami et al., 2023) as one of the topologies that provides data backhaul services (the side of the network that communicates with the

global internet), connecting together the ISP facilities such as central offices or headend. The drawback of this topology is that if one node or link connecting two nodes shuts down, this affects the whole network. The performance of ring topology is slower than the mesh topology (Theeyancheri et al., 2023). This research study does not consider using ring topology during its implementation.

- iii) Hybrid Topology: This is a network topology in which two or more different topologies, such as bus, mesh, and ring topologies, are merged or combined to lay out a network (Wan et al., 2021; Khalil & Ghanim, 2023). This topology is typically used in various institutions, including universities, banks, automated industries, the financial sector, nonprofit organizations, and multinational corporations (De Beelde et al., 2021) (See Figure 2.3).

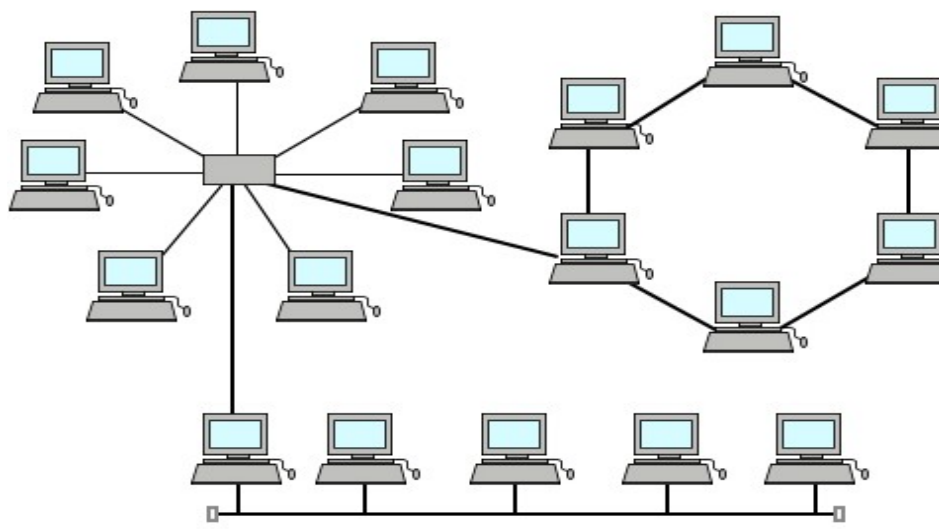


Figure 2. 4: Hybrid topology structure (De Beelde et al., 2021)

The usage of hybrid topology in a network depends on the deployment of nodes and the requirements needed, such as any performance-based or location-based of the desired network (Huang et al., 2022; Majid et al., 2022). The drawback of this topology is that it is very difficult to configure compared with the ring and tree topologies. The length of the segment is

limited; and the limit of the segment depends on the type of cabling used. This research study does not consider using hybrid topology during its implementation.

- iv) Mesh Topology: This is the topology that gives growth to network connectivity and the load-balancing traffic in various network environments (Kumar et al., 2021). This topology is characterized by the interconnectedness of nodes and redundancy, in which each node within the network is linked to the other (Alimi et al., 2021; Amodu et al., 2021) (See Figure 2.4.)

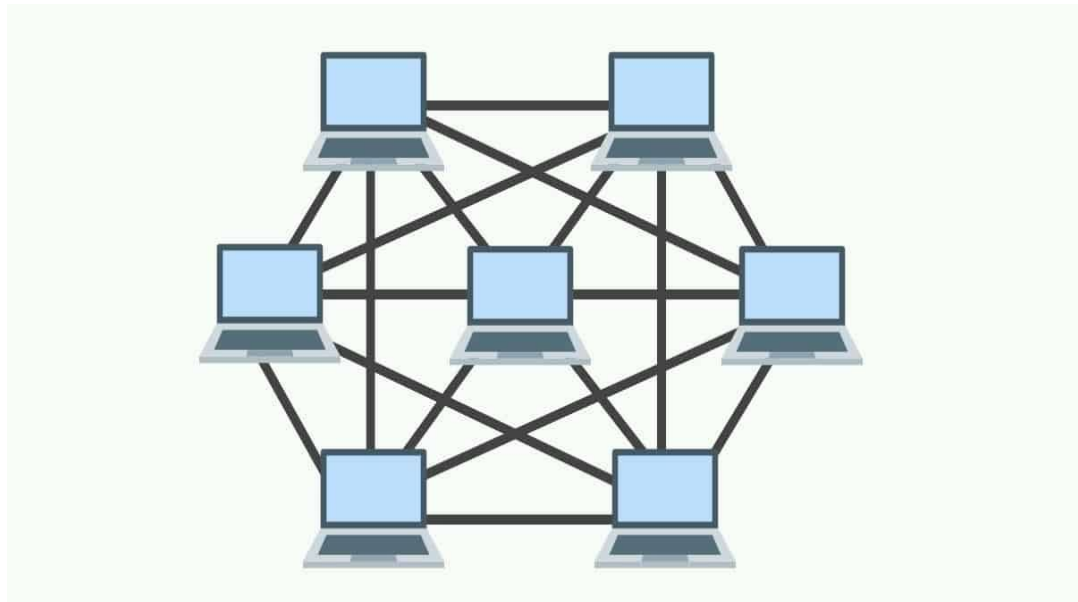


Figure 2. 5: Mesh topology structure (Amodu et al., 2021)

The mesh topology is known as multi-hop communication, which supports the forwarding of packets from one node to a node out of its range, per an intermediate node (Singal et al., 2021; Mugerwa et al., 2023). Furthermore, this topology increases the resilience and stability of the network in case of any node or link failure within the network. The advantage of this topology is that it can manage high levels of traffic data within large environments

such as smart cities, military set-ups, etc. (Samuylov et al., 2022; Tian et al., 2023). Moreover, it again provides multiple paths to succeed in the destination and the redundancies. The drawback of this topology is that the power requirement of nodes is too high: all nodes must remain active all the time so that they can share the data load in the network. This research study considers using the mesh topology during its implementation on the smart city because of how this handles the data transmission throughout the entire network.

2.4. WSNs protocols for Smart city applications

WSNs are utilized in various applications of our daily lives. Due to their cost-effectiveness and rapid deployment, WSNs are used for securing smart cities and controlling the movement of vehicles (Khalifeh et al., 2021; Mohamed et al., 2020). WSNs achieve this by providing remote monitoring and a sensing ability within the environment (Khalifeh et al., 2021). There are various types of institutes of electrical and electronics engineers (IEEE) protocols based on WSNs implemented in smart cities. Some of these protocols are discussed below:

2.4.1 IEEE 802.15.4 Zigbee

The IEEE 802.15.4 Zigbee is the wireless network standard defined in the network layer (layer 3) of the open systems interconnection (OSI) model which supports the protocol that the network infrastructure required for WSNs applications (Vineela et al., 2023). This protocol normally considers the network and application layer (layer 7) for its operations (Kampourakis et al., 2023). The IEEE 802.15.4 Zigbee normally works with the IEEE 802.15.4, which operates in the physical layer and data link layer (Layer 1 and 2) of the OSI model, as demonstrated in Figure 2.5.

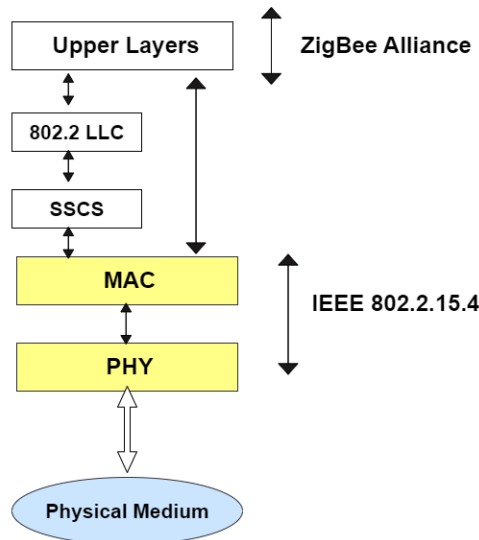


Figure 2. 6: IEEE 802.15.4 protocol stack

In IEEE 802.15.4 Zigbee, the packets in this network protocol are transmitted through different phases from the physical medium to the upper layer of the Zigbee alliance, as demonstrated by Figure 2.6. The communication based on this protocol normally occurs in a 5-Megahertz (MHz) channel which ranges from 2.405 to 2.480 Gigahertz (GHz) (Gómez-Tornero, 2022; Cena et al., 2023). The IEEE 802.15.4 Zigbee protocol supports both the hardware and software standard design for WSNs. This protocol has managed to show the high reliability, low power, low cost, low data rate, and scalability in WSNs (Arora et al., 2022). However, the limitation based on the IEEE 802.15.4 Zigbee protocol is that, should any problem occur in Zigbee-compliant applications, the replacement cost will be high (Midda & Koner, 2023).

2.4.2 IEEE 802.16 (WiMAX)

The IEEE 802.16 worldwide interoperability for microwave access (WiMAX) is broadband wireless access that supports both fixed and mobile internet access (Jayaraman et al., 2023). Furthermore, this protocol has a maximum speed of 75 Mbps under optimal conditions in the case of fixed WiMAX. Therefore, WiMAX provides the service throughout the coverage area to enable the continuous connectivity of wireless devices. The IEEE 802.16 WiMAX normally operates within layer 1 and at the MAC

which is on layer 2. The WiMAX therefore increases the network capacity, in particular, as demonstrated in Figure 2.7 (Ibrahim et al., 2022; Prabha et al., 2023b).

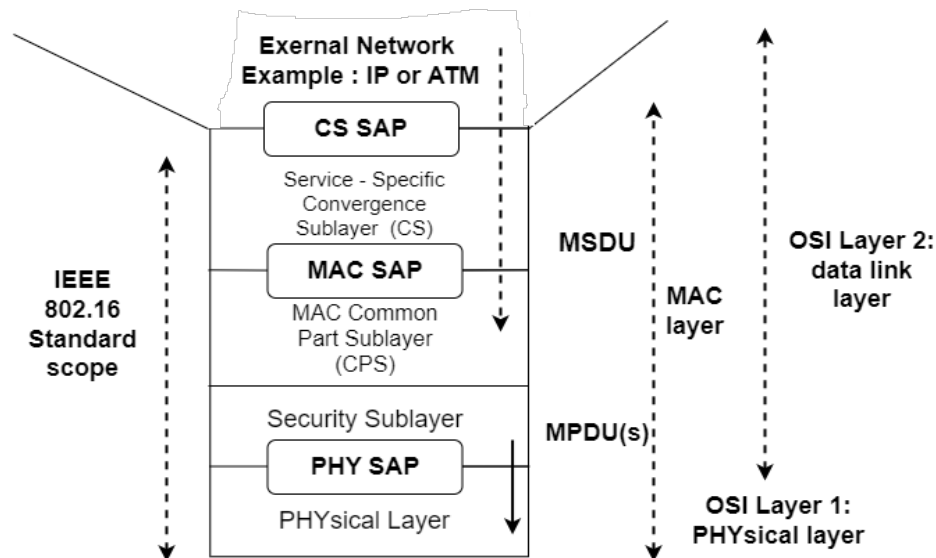


Figure 2. 7: IEEE 802.16 WiMAX protocol

The wireless communication range of IEEE 802.16 WiMAX is over 50-100 km, with a data rate of 0.4-1 Gbps transmission based on the IEEE standard protocol for local and metropolitan area network (MAN) (Debauche et al., 2021). Figure 2.7 demonstrates how IEEE 802.16 WiMAX operates in a network through the OSI layer model. In this protocol, the packets are transferred from Layer 1 to Layer 2 throughout the sublayers, such as the MAC common part sublayer (CPS) and services-specific convergences sublayer (CS) (Araújo et al., 2020). Furthermore, this protocol provides wireless data communication over long distances in a variety of ways, from point-to-point links to full mobile cellular-type access within a network. This protocol shows that it can serve hundreds of users within a single station in a network (Attrah et al., 2021; Prabha et al., 2023a). However, the drawback of the IEEE 802.16 WiMAX protocol is that when there are bad weather conditions such as rain, it normally interrupts the

signal of WiMAX, and further causes a loss of network connection (Abuhdima et al., 2022; Shaikh et al., 2022).

2.4.3 IEEE 802.11a

The IEEE 802.11a protocol is one of the wireless standards issued under IEEE 802.11 standards that govern the wireless networking transmission methods using radio frequency (Sidam et al., 2022). This protocol was one of the first wireless fidelity (Wi-Fi) standards to be launched in the year 1999, because it demonstrated the capabilities of transmitting the raw data (Chauhan et al., 2021; Mizuno et al., 2023). The IEEE 802.11a protocol was released with the hope of encountering less interference during the transmission of packets within the wireless network, most of the devices using 2.4 GHz. Furthermore, this protocol deploys a 5 GHz band in a network, and it supports a theoretical throughput of up to 54 Mbps.

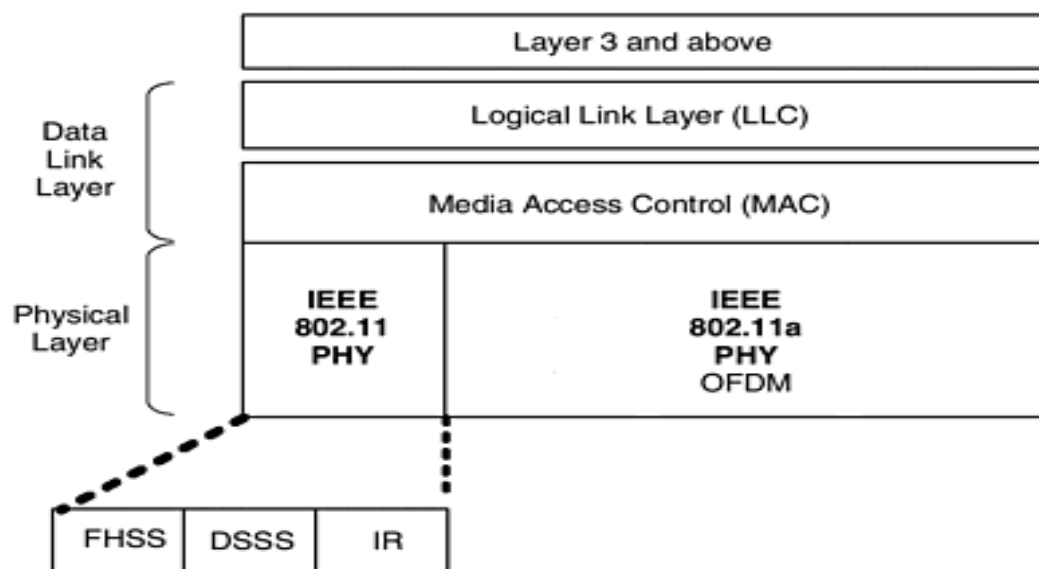


Figure 2.8: IEEE 802.11a protocol

As demonstrated in Figure 2.8, the IEEE 802.11a protocol uses a modulation technique known as orthogonal frequency-division multiplexing (OFDM) within layer 1 of the OSI layer model, while using the same layer 2 protocols (LLC and MAC) (Hu et al., 2021). This protocol was designed specifically to be used within the office settings.

Furthermore, the IEEE 802.11 applies the modulation techniques of frequency hopping spread spectrum (FHSS) which has a bandwidth of 2 Mbps maximum, and a direct sequence spread spectrum (DSSS). The DSSS transmits data using a wider frequency band, and infrared (IR) which uses ultrahigh-frequency infrared light (Ristić et al., 2022; Fatima et al., 2022). The IEEE 802.11a offers non-overlapping channels in which a large number of users can be accommodated without any performance degradation (Memon et al., 2021). However, the limitation of this protocol is that the distance covered is lower than with the IEEE 802.11b and it is more expensive (Coronado et al., 2021).

2.4.4 IEEE 802.11b

The IEEE 802.11b is designed to extend the wireless networking throughput up to 11 Mbps while using the same 2.4 GHz band (Parashar et al., 2022; Niu et al., 2021). This protocol is normally applied in a point-to-multipoint configuration, wherein an access point (AP) communicates through an omnidirectional antenna with mobile clients within the network range. The IEEE 802.11b uses the DSSS modulation method to reduce the signal interference within the network, as demonstrated in Figure 2.9 (Amoordon et al., 2022).

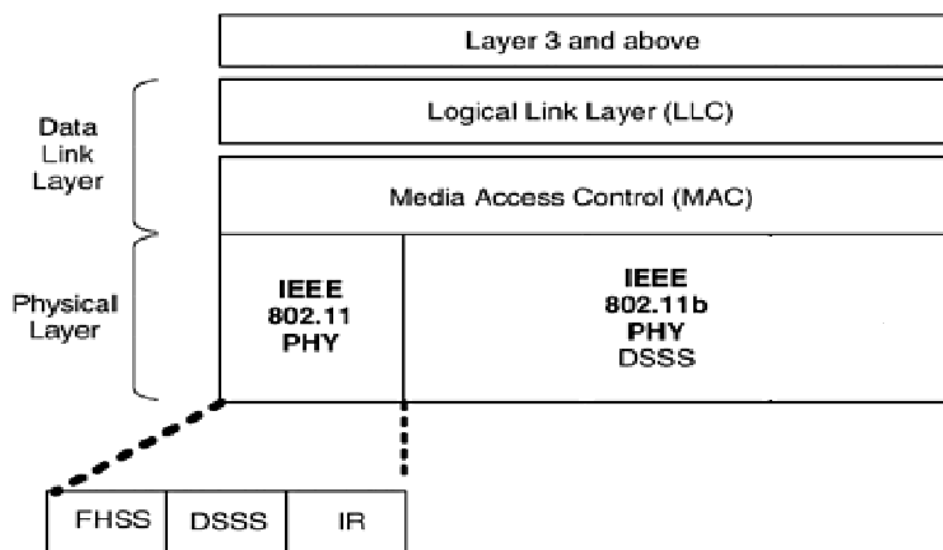


Figure 2. 9: IEEE 802.11b protocol

This protocol has managed to accommodate the weak signals by maintaining lower DSSS modes; the protocol has thus emerged as the signature within wireless local area network (WLAN) technology (Gong et al., 2023). Moreover, the IEEE 802.11b is applied to home-segment devices such as Wi-Fi, because the set-up is simple and the cost is lower. This protocol uses layer 2 protocols — the LLC and the MAC during packet transmission. The IEEE 802.11b protocol has managed to provide a mechanism for encrypting the traffic and authenticating nodes using the wired equivalent privacy (WEP) protocol within the network (Niu et al., 2021; Amoordon et al., 2022). However, the limitation of IEEE 802.11b is that it lacks interoperability with voice devices, and offers no quality of services (QoS) provisions for multimedia content (Bakiko et al., 2022; Albattah et al., 2022).

2.4.5 IEEE 802.11g

The IEEE 802.11g protocol is a higher-bandwidth successor which has provided the significant enhancement of the popular 802.11b protocol discussed in Section 2.4.4. The IEEE 802.11g protocol offers wireless transmission over a distance of 150 feet and up to the speed of 54Mbps, compared with the 11Mbps of the 802.11b protocol; and the IEEE 802.11g protocol operates in the range of 2.4GHz radio band (Svertoka et al., 2021; Mubashar et al., 2021). The APs implemented in this protocol are compatible with both 802.11b and 802.11g clients. Moreover, when the IEEE 802.11g protocol was designed, the developers managed to integrate the best qualities of both IEEE 802.11a and IEEE 802.11b to create the 802.11g protocol in the year 2003 (Val et al., 2021; Ying et al., 2022).

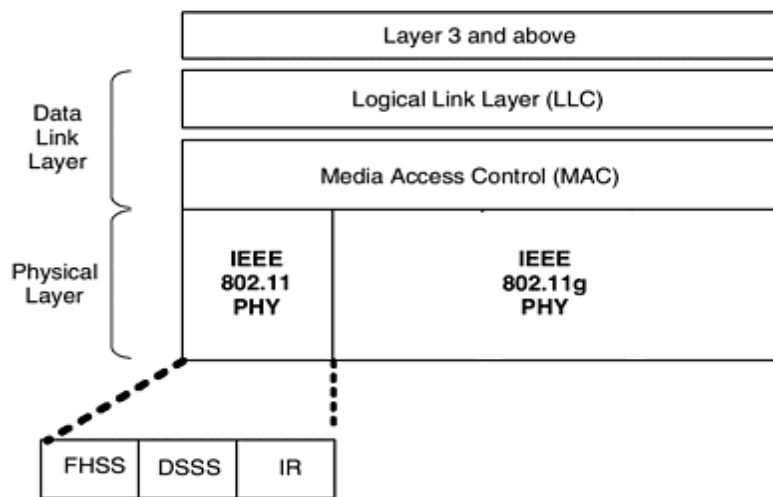


Figure 2. 10: IEEE 802.11g protocol

The IEEE 802.11g protocol has a very rapid operating speed; the signal range is good, and is not easily obstructed within the network. This protocol has a total number of 14 available network channels, of which some may be restricted, depending on the regulations of the particular country (Debauche et al., 2021; Dixit et al., 2022). However, the limitation of this protocol is that it is much more expensive than the 802.11b protocol (Cao et al., 2022; Elhilali et al., 2023). Furthermore, the appliances surrounding this protocol may interfere with the unregulated signal frequency, the operating frequency being the 2.4GHz radio band.

2.4.6 Satellite

Satellite communication is one of the protocols that govern the transmission of traffic data between two or more points in space (Ravishankar et al., 2021; Sun et al., 2022). This protocol is a set of extensions to the existing protocols that were developed to improve the performance of internet protocols within the network space environments. Satellite communication normally provides communication services in three different streams, such as telecommunication, data communication, and broadcasting (Popescu & Naaji, 2022; Fan et al., 2022; Dang et al., 2023).

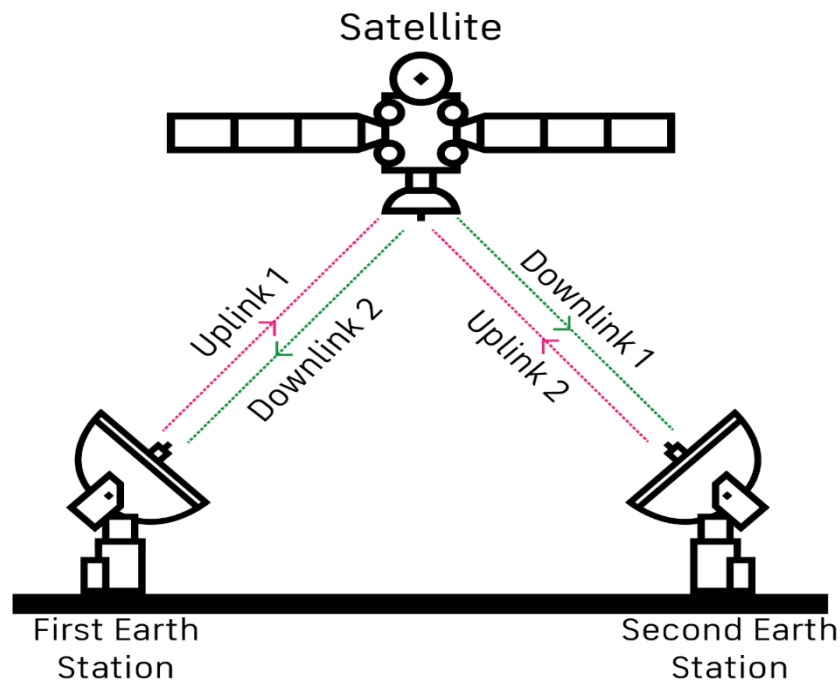


Figure 2. 11 : Satellite communication infrastructure

The satellite communication protocol uses multiple frequency bands to transmit data or information around the entire network environment. The most common frequency bands used in satellite communication are as follows: long-band (L-band), conventional-band (C-band), short-wavelength band (S-band), and Kurtz-above band (Ka-band) (Bhowmick et al., 2020; Tampuu et al., 2023). The L-band was designed to support the operating frequency range of 1–2 GHz in the radio spectrum with a wavelength range of 30–15 cm (Matolak et al., 2021). The C-band was designed for a portion of the electromagnetic spectrum designated for satellite transmissions with the frequency range of 4 to 8 GHz (Javid et al., 2021). The S-band was designed to support the microwave frequency range in the network, the range being from 2 to 4 GHz (Wu et al., 2022a). The Ka-band was designed to support some portion of the microwave frequencies ranging from 26.5 to 40 GHz within satellite communication (it offers wide bandwidth, high throughput, and high data rates) (Kähkönen et al., 2022). Furthermore, satellite communication is also now used to transport TCP/IP traffic between distant locations, and to offer internet access. Satellite communication further

provides a cost-efficient platform that allows users to connect remotely in different locations (Al-Hraishawi et al., 2022). However, the limitation of satellite communication is that interference may arise when there are bad weather conditions (Al Homssi et al., 2022; Wrona et al., 2023).

Table 2.1: Summary of WSNs Protocols for Smart City Applications

Protocol	Main Characteristics	Layer 1 Specs	Layer 2 Specs	Data Rate	Transmission Range	Smart City Application
IEEE 802.15.4 (Zigbee)	Energy saving, and a very short range	2.4 GHz band, DSSS modulation	Carrier sense multiple access/collision avoidance (CSMA/CA)	20 to 250 Kbps	10 to 20 m	Smart buildings and smart grid.
IEEE 802.16 (WiMax)	MAN	2 to 66 GHz band, OFDMA modulation	Time division duplex (TDD) and frequency division duplex (FDD)	2 to 75 Mbps	Up to 35 miles (56Km)	Smart buildings and smart grid.
IEEE 802.11a	Data networking and local area network (LAN)	5 GHz band, OFDM	CSMA/CA	6, 9, 12, 18, 24, 36, 48, 54 Mbps	120 m outdoors	All
IEEE 802.11b	Data networking and LAN	2.4 GHz Band, DSSS	CSMA/CA	1, 2, 5.5, 11 Mbps	140 m outdoors	All
IEEE 802.11g	Data networking and LAN	2.4 GHz Band, DSSS, OFDM	CSMA/CA	6, 9, 12, 18, 24, 36, 48, 54 Mbps	140 m outdoors	All
Satellite	Wide area network (WAN)	1.53 to 31 GHz	Frequency division multiple access (FDMA) and time division multiple access (TDMA)	10 Mbps (upload) and 1 Gbps (download)	Satellite can cover 100s of km to entire earth	UAVs, pipeline monitoring, and intelligent transportation

Table II shows the summary of some WSN protocols that are applicable for smart cities within the wireless network (Szott et al., 2022; El Houssaini et al., 2023; Al Homssi et al., 2022; Elhilali et al., 2023). This research study will make use of the satellite protocol alongside the IEEE 802.15.4 (Zigbee) to minimize the traffic congestion

within smart cities. This is because the satellite protocol supports the data range of 10 Mbps (upload) and 1 Gbps (download).

2.5. Overview of Traffic Congestion in WSNs

WSNs are prone to traffic congestion, in which unpredicted events such as packet and node congestion occur within the network (Sahar et al., 2021; Suma & Harsoor, 2022; Bagais et al., 2022; Wang et al., 2023). The traffic congestion primarily occurs near the sink node when packets are transmitted through a wireless link from SNs to the BSs or sink node (Saheb et al., 2022). Furthermore, traffic congestion also occurs on the link level during packet transmission within WSNs (Yadav et al., 2021; Saheb et al., 2022). Therefore, due to the resource constraints of WSNs, there are several deployed nodes that are event-driven in WSNs.

Hence, WSNs in some applications such as those event-driven, face significant challenges due to a huge amount of transmitted data from the source nodes to the destination nodes (Grover et al., 2022; Mohammad et al., 2023). This normally causes traffic congestion, which leads to the breakdown of a network. The huge amount of transmitted data decreases the network throughput and energy efficiency, increasing packet loss and delays, and reducing network lifetime (Arieth et al., 2022). There are several types of WSN traffic congestion based on how packets are lost, namely: link-level congestion and node-level congestion (buffer overflow).

- i) Link-level congestion occurs when multiple SNs transmit packets using the same transmission channel. The channel therefore becomes overloaded within the network (See Figure 2.12) (Yakymchuk et al., 2022). When packet congestion occurs, those transmitted packets may fail to reach the destination node because the packets overload the transmission channel (Ashraf et al., 2022; Khaniabadi et al., 2023; León et al., 2023). This type of

packet congestion increases the packet service time and decreases the link utilization within the network. Furthermore, this type of congestion increases traffic congestion because some of the collected packets using SNs from the smart city do not reach the destination nodes.

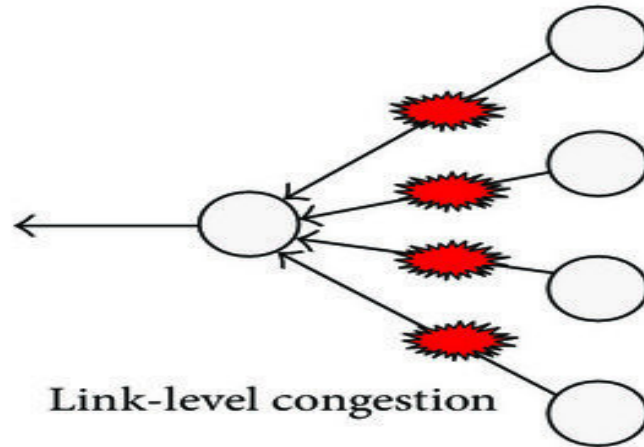


Figure 2. 12: Link-level congestion (Yakymchuk et al., 2022)

- ii) Node-level congestion occurs at the specific node which receives the transmitted packets from different SNs, and leads to buffer overflow (See Figure 2.13) (Zhang et al., 2022). Buffer overflow is normally caused by the mismatch of the packet departure rate and arrival rate during packet transmission; or when the packet arrival rate is higher than the packet service time (Wu et al., 2022b; Lin et al., 2023a; Gao et al., 2023). When this buffer overflow occurs, all the incoming packets are simply dropped, which leads to increased queuing delays and packet loss. In smart cities, when the buffer overflow occurs, it increases the traffic congestion, because some of the packets collected within the area are congested at the receiving node. This leads to both air and noise pollution. Thereafter, the packet loss must be retransmitted within the network, which decreases the network lifetime (Liu et al., 2022; Zhang et al., 2022).

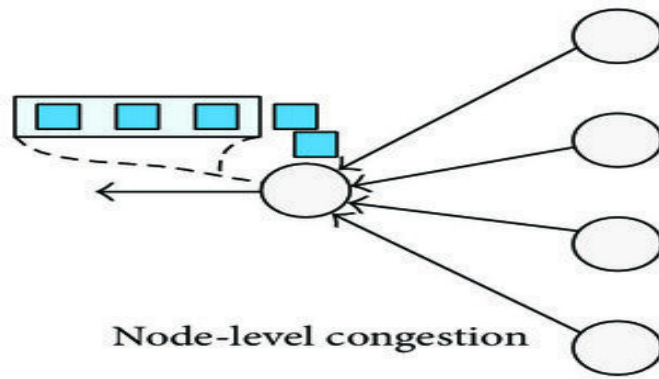


Figure 2. 13: Node-level congestion (Zhang et al., 2022).

In Figure 2.12 and Figure 2.13, this research study shows the two possible ways of causing packet congestion within WSNs. This research study will focus on node-level congestion. To minimize and manage the traffic congestion that occurs in WSNs within the smart city, an appropriate algorithm should be developed. Doing so, will allow the collected packets within the area to reach their destination nodes, which will lead to less traffic congestion within the smart city. Hence, the PBCC algorithm is developed to minimize the traffic congestion in WSNs within the smart city. Furthermore, load-balancing in this research study will be implemented to avoid any link-node congestion during the packet distribution, so that all packets can reach their intended destination nodes in time.

2.6. Techniques Used for Traffic Congestion in WSNs

There are various traffic congestion techniques used in WSNs which normally concentrate on assigning fair and efficient transmission data rates to each node within the network (Grover et al., 2022; Vanitha et al., 2022; Umar et al., 2023). These techniques adjust the traffic rate based on packet service time along with fair packet scheduling algorithms. The traffic-congestion techniques are divided into three parts: congestion notification, congestion detection, and congestion mitigation (Bawaneh & Simon, 2023).

i) **Congestion notification**

Congestion notification is a mechanism used for reporting the congestion state of the packets and node, up until the sink or gateway (Grover et al., 2022; León et al., 2023). This process of notification helps in monitoring, detecting, and signalling the presence of traffic congestion when it occurs within a network. Once congestion is recognized in a network, the congestion notification will employ the congestion-control technologies, namely: adjusting transmission rates, and rerouting traffic to regulate the flow of packets and alleviate congestion (Sunassee et al., 2021; Wang et al., 2022). There are some common techniques used for congestion notification in WSNs, namely (Alghazzawi et al., 2021; Navarro-Espinoza et al., 2022):

- **Queue-length monitoring:** This mechanism monitors the length of the traffic packets that queue at links or at the nodes, which can provide some insights into the congestion (Su et al., 2022). When the queue length of the traffic packets increases beyond a certain threshold, this shows that there is potential for packet congestion. The nodes will then notify other nodes about the congestion status within the network (Navarro-Espinoza et al., 2022).
- **Traffic monitoring:** Traffic monitoring observes the traffic patterns in the network (Ageyev et al., 2022; Wan et al., 2022). These traffic patterns help in identifying areas with high packet-transmission rates, which may lead to congestion. The congestion notification can then notify the possibilities of congestion based on the observed traffic conditions using the patterns (Yahyaoui et al., 2023).

- **Packet-loss monitoring:** Packet-loss monitoring refers to the monitoring of the packet-loss rate which can be the indirect indication of any possible congestion within a network (Sharan et al., 2022; Okada et al., 2023). In a network, should packet congestion occur, there will be a high increase in packet loss. Then the nodes will detect this increase and notify other nodes or the sink about the congestion (Ni et al., 2023).

ii) **Congestion detection**

Congestion detection is another mechanism used to identify and monitor congested packets within the network (Sunassee et al., 2021; Angurala et al., 2022). This mechanism was developed to maintain the overall network performance and reliability of packet transportation from the source node to the destination node. There are some common techniques used for congestion detection in WSNs, namely (Reddy et al., 2022):

- **Energy considerations:** The SNs in WSNs are often powered by batteries with limited energy (Nagaraju et al., 2022). Hence, congestion detection processes must consider the energy constraints of nodes, because this improves the reliability of the nodes used in WSNs. High-traffic packet transmission can lead to an increase of energy consumption due to frequent data transmissions and receptions (Shafi et al., 2022; Subramaniam et al., 2022).
- **Adaptive routing protocols:** The congestion detection aspect is also used in adaptive routing protocols within the WSNs (Rady et al., 2021; Gururaj et al., 2023). This process is implemented because these protocols allow congestion detection to dynamically adjust the routes that packets take through the entire network based on the

current conditions during packet transmission. Such helps the traffic packets to be rerouted away from the congested areas, which distributes the traffic more evenly (Doraisamy et al., 2022; Mishra et al., 2023).

- **Efficient congestion detection:** The efficient congestion detection aspect within the WSNs was developed to maintain the reliability, scalability, and longevity of the entire network, especially in applications where timely and accurate data is critical (Anitha et al., 2023). Hence, this aspect was developed because most of the mechanisms often focus on the development of a lightweight and energy-efficient congestion detection suitable for resource-constrained sensor nodes (Anitha et al., 2023; Heidari et al., 2023).

iii) **Congestion mitigation**

Congestion mitigation is another mechanism which involves the strategies and techniques used to alleviate the congestion problems in the network (Attar et al., 2022; Lakshna et al., 2023). Hence, this aspect is fundamental to ensuring the reliable and efficient operation of the network (Lakshna et al., 2023). There are some common congestion mitigation techniques used in WSNs, namely (Lin et al., 2023b):

- **Traffic prioritization:** The congestion mitigation normally prioritizes certain types of packets and the critical nodes within the WSNs (Grover et al., 2022; Trinh et al., 2022). This technique also assists in giving the packets priority to ensure that crucial information is transmitted with lower latency, even in congested situations.

- **Cross-Layer optimization:** Cross-layer optimization is also implemented in congestion mitigation (Bai et al., 2023). This technique is used when the information from multiple layers of the communication protocol stack is used to make decisions. For example, combining information from the physical layer, data link layer, and network layer can provide a more comprehensive view of the network state for congestion mitigation strategies (Bai et al., 2023; Wu et al., 2023).
- **Dynamic channel allocation:** Dynamic channel allocation is implemented when there are multiple communication channels available (Durga et al., 2022; Inga et al., 2023). Dynamic channel-allocation techniques can thus be employed to switch channels based on the congestion level. This technique helps in distributing the communication load across different channels, reducing the likelihood of congestion (Balador et al., 2022).

The traffic-congestion mechanisms discussed above demonstrated how traffic congestion is being minimized in WSNs. Therefore, this research study does consider these mechanisms during the development of the proposed PBCC algorithm. By augmenting the capabilities of the PBCC algorithm implementation, this research endeavours to achieve significant reductions in traffic congestion levels across the smart city infrastructure.

2.7. Related Works

The current related works that have been proposed, using various algorithms for smart cities to avoid traffic congestion, are discussed below. This research study will show the existing studies conducted by other researchers, before outlining the research gap.

However, these researchers focused on several different mechanisms for traffic congestion, as discussed below.

A new adaptive traffic light control system (ATLCS) algorithm was proposed by (Aleko & Djahel, 2020) to assist traffic-management authorities in efficiently reducing traffic congestion in smart cities. The authors outlined that, to reduce traffic congestion in smart cities, the proposed ATLCS algorithm should synchronize the number of traffic lights controlling consecutive junctions. Such can be achieved by creating a delay between the times at which each of them switches to green in a given direction, based on the load of vehicles in that area. The above authors further stated that the dynamic updated ATLCS algorithm is implemented based on the number of vehicles waiting at each junction. This will minimize the number of occurrences of the 'stop and go' phenomenon caused by traffic congestion within smart cities. The authors use the Network Simulator 2 (NS-2) simulation tool for the performance evaluation of their proposed ATLCS. The simulation results demonstrated that the average time taken by vehicles travelling in a synchronized direction has been significantly reduced by up to 39% over the non-synchronized fixed-time traffic light control systems. However, the proposed ATLCS algorithm does not consider the emergency vehicles travelling the other direction of the road. This research study will consider the use of the ATLCS algorithm during the development of the PBCC algorithm to reduce traffic congestion in a smart city.

A Congestion aware routing using fuzzy rule sets (CARF) was proposed by (Sangeetha et al., 2020) to handle the estimation of congestion levels and excess traffic conditions in smart cities. This was achieved by identifying the non-localized node paths, and adding them to the existing localized node paths. The authors further illustrated that using the fuzzy rule prediction assisted in selecting a more reliable and congestion-lightened path to the sink node. Sangeetha et al. (2020) demonstrated that

their proposed CARF integrated two methods, namely, multiple-path identification and congestion-mitigated routing. The NS-2.35 was used to simulate the proposed CARF. The simulation results demonstrated that the proposed CARF managed to reduce the congestion level by 15.3% compared with the hop count-based geometric method (HCGM) and the fuzzy-based congestion mitigation (ECFM) algorithm. However, the proposed CARF does not consider the traffic congestion index when estimating the congestion level in smart cities, which leads to traffic congestion. This research study will consider the CARF when developing the proposed PBCC algorithm.

A novel energy-efficient fuzzy-based congestion-detection and control algorithm (EE-FBCDCA) was proposed by (Ramasamy-Rajeswari et al., 2022) to control the traffic congestion caused by the high number of vehicles in smart cities. The authors illustrated that the proposed EE-FBCDCA consists of three phases, namely: a) Fuzzy inference system-based congestion detection, b) congestion notification and c) congestion control. Furthermore, once the proposed EE-FBCDCA detects the congestion in smart cities, all nodes will adjust their respective transmission rate in order to minimize the traffic congestion. Nodes will offer an alternative route to other vehicles so that they do not cause more congestion within the area. Ramasamy-Rajeswari et al. (2022) implemented and analysed their proposed EE-FBCDCA using a MATLAB simulator. Therefore, their simulation results demonstrated that the proposed EE-FBCDCA managed to enhance the network lifetime and reliability when compared with hierarchical-tree based congestion control using fuzzy logic (HTCCFL) and fuzzy-based adaptive congestion control (FBACC) within the smart cities. However, the proposed EE-FBCDCA does not consider the time delays of data, and the emergency vehicles when measuring the level of traffic congestion, thus leading to traffic congestion. This research study will consider the EE-FBCDCA when developing the proposed PBCC algorithm in order to minimize traffic congestion.

An intelligent traffic management systems (ITMS) was proposed by (Khanna et al., 2019) to predict the optimum routes based on vehicle categorization, accident occurrences, and levels of precipitation. The authors further demonstrated that the proposed ITMS makes use of the green corridor concept to allow the emergency services to travel without facing any kind of traffic congestion in smart cities. NS-2 was used to test the effectiveness of the proposed ITMS. The simulation results demonstrated that the proposed ITMS manages better to predict the optimum routers for emergency services within the smart cities compared with the fuzzy sliding mode congestion control algorithm (FSMC) and inter-vehicle communication. However, the proposed ITMS does not consider the traffic mobilization patterns when there is a high volume of vehicles travelling within the cities, which leads to traffic density and an increase of travelling time. This research study considers the ITMS during the development of the proposed PBCC algorithm to improve the transportation system within the smart cities.

A fusion-based intelligent traffic congestion control system (FITCCS) for vehicular networks (VNs) was proposed by (Saleem et al., 2022) to alleviate traffic congestion by collecting traffic data on available routes in smart cities. The authors further illustrated that their proposed model provides innovative services to the drivers that enable a view of traffic flow and the volume of vehicles available on the road remotely, intending to avoid traffic jams. Saleem et al. (2022) used the MATLAB simulation tool to simulate their FITCCS model. The simulation results showed that their model decreases the congestion and improves the traffic flow; also that the proposed model managed to provide the accuracy of traffic flow by 87%. Traffic congestion was decreased by 5% by comparison with other previous approaches. However, Saleem et al. (2022) proposed model does not consider the amount of data traffic that exceeds the real-time pollution management sensors' (RPMS) capacity (type of sensors used

to collect data in smart cities) on the roads. The RPMS used the MQTT (message queuing telemetry transport) protocol for setting up rules when collecting data from the smart cities. This still causes traffic congestion within the cities, due to the lower RPMS capacity implemented within the smart cities to collect data traffic.

The low-cost locations discovery (LCLD) scheme was proposed by (Teng et al., 2021) to reduce the localization error, the vehicle movement, and the cost of vehicle broadcasting, based on mobile vehicles and unmanned aerial vehicles (UAVs) in a smart city. The authors illustrated that they used the large error rejection (LER) algorithm and the UAV same position broadcast repeat (USPBR) algorithm to minimize the localization error and reduce the cost of broadcasting vehicles within the smart cities. The authors stated that they used the optimized network engineering (OpNet) as their simulation tool to test the effectiveness of the proposed LCLD scheme. Their simulation results demonstrated that the proposed LCLD scheme managed to reduce the localization error by 78.8%. Cost reductions were reduced by up to 16.5% compared with the adaptive UAV flight path planning (AUPPP) algorithm and USPBR algorithm. However, the proposed LCLD scheme does not consider the time delays of data and traffic congestion index distributed by sensors within the smart city. This remains a problem which causes traffic congestion within the smart city. This research study proposes the use of the PBCC algorithm to reduce the traffic congestion within a smart city. The proposed PBCC algorithm will make use of the LCLD algorithm to reduce the errors that may occur during the distribution of data within the smart city.

The optimal restricted driving zone (ORDZ) which uses the genetic algorithm was proposed by (Jan et al., 2020) for selecting suitable controlled traffic zones that can optimally control traffic congestion and air pollution. This amelioration will result in improved citizen satisfaction within the smart cities. The authors further demonstrated

that the proposed ORDZ uses an augmented genetic algorithm and determinant theory to randomly generate various foursquare zones within the smart cities. The fitness function in ORDZ considers a trade-off between traffic load and citizen satisfaction. The NS-2 simulation tool was used to simulate the proposed ORDZ. The simulation results demonstrated that the ORDZ managed to control traffic congestion by 30.6% compared with other methods. However, the proposed ORDZ does not consider the traffic patterns in which there are more vehicles within the smart cities, leading to traffic congestion. This research study will not consider ORDZ during PBCC algorithm development.

The optimized weight Elman neural network (OWENN) algorithm that makes use of efficient Internet of Things (IoT)-based traffic prediction was proposed by (Neelakandan et al., 2021) to monitor the traffic jams caused by the increase of vehicles and accidents occurring in smart cities worldwide. The authors stated that the proposed OWENN algorithm consists of five different phases, namely: IoT data collection, feature extraction, classification, optimized traffic IoT values, and traffic signal control system. The OWENN algorithm enhances the IoT values by using the controlled Intel 80,286a when processing the data within the smart cities. The NS-2 simulation tool was used to carry out the simulation results of the proposed OWENN algorithm. The simulation results shown by Neelakandan et al., 2021 demonstrated that the proposed OWENN algorithm managed to reduce the traffic jams in smart cities with 91.23% accuracy. However, the proposed OWENN algorithm does not consider the delay of data during the transmission within the network, which makes traffic congestion persist within the smart city. This research study will consider enabling the OWENN algorithm to minimize traffic congestion within the smart cities.

The priority-based adaptive traffic signal control (PATSC) system was proposed by (Mondal & Rehena, 2022) to manage the traffic congestion that occurs at the road

intersections. The authors stated that the proposed PATSC system acts according to the context of the traffic status within the intersections. Furthermore, the PATSC system makes use of a selection method based on the lane priority. The lane priority is calculated based on traffic parameters: vehicle density and waiting time. The MATLAB simulation tool was used to test the effectiveness of the proposed PATSC system. The simulation results demonstrated that the traffic flow has increased by 31.05% compared with the fixed-time traffic signal system in the traffic light. However, the proposed PATSC system does not consider the travel time and fuel consumption when sensors collect data within the road intersections. Such leads to traffic congestion within the smart city. This research study will consider the PATSC system during PBCC algorithm development in reducing the traffic congestion in smart cities.

The use of the emergency vehicle priority system (EVPS) was proposed by (Karmakar et al., 2020) to determine the priority level of emergency vehicles (EVs). The number of necessary signal interventions would be estimated, taking into account the effect of those interventions on the flow of traffic in the roadways next to the EVs' travel path, depending on the kind and severity of an incident. The authors further demonstrated that the proposed EVPS estimates the number of green signal interventions to attain the quickest incident response while concomitantly reducing the impact on other vehicles within the smart cities. The OpNet simulation tool was used to simulate the proposed EVPS. The simulation results demonstrated that the EVPS managed to produce an appropriate number of interventions that reduce emergency response time significantly. However, the proposed EVPS does not consider the traffic density when reducing the response time, which allows traffic congestion to persist within the smart city. This research study will not consider EVPS during the PBCC algorithm development.

2.8. Chapter Summary

In this chapter, this research study presented the overview, design, protocols, traffic congestion, and techniques of WSNs. In addition, the research study provided various related work based on traffic congestion algorithms, priority-based adaptive traffic controls, and the intelligent traffic management systems, according to their common methodology applied. Furthermore, this research study discussed several identified gaps in the existing algorithms and methods, reflected in Section 2.7. In the next chapter, this research study presents the design of the proposed PBCC algorithm.

Chapter 3: System Architecture and Design

3.1. Introduction

In this chapter, this research study explores the field of WSNs which is implemented within the smart city area, focusing on minimizing the on-road traffic congestion through the PBCC algorithm. This research study begins by providing a detailed exploration of the system architecture, highlighting the key components that form the basis of the proposed PBCC algorithm. Subsequently, this research study models the PBCC algorithm, which leverages machine learning (ML) techniques to optimize the PBCC algorithm within the on-road traffic congestion in the smart city. Furthermore, WSNs are observed as a revolutionary information-gathering method, which forms communication and information systems to significantly improve the consistency and efficiency of the system infrastructure (Tanveer et al., 2020; Trinh et al., 2022; Umar et al., 2023). With the recent technologies as discussed in Section 2.2 of Chapter 2, the implementation of a wireless network in a smart city advances the functionality of roads, which minimizes on-road traffic congestion. This research study also presents a comprehensive flow diagram of the system that illustrates the sequence of actions and decision-making processes. Finally, this research study concludes the chapter with a summary that gives a concise overview of the main insights and findings that arise from our exploration of the proposed PBCC algorithm for smart cities.

In this chapter, the research study presents the design of the proposed PBCC algorithm and related assumptions. Section 3.2 reflects the system architecture. In Section 3.3, this research study presents the design of the proposed PBCC algorithm. Finally, Section 3.4 provides a summary of the chapter.

3.2. System Architecture

The proposed architectural design represents the integrated WSNs infrastructure in smart cities. The system is designed to facilitate the seamless movement of road users by providing current updates on traffic congestion in the city. This connectivity is established by wireless technologies such as RSUs, OBUs, BSs, and the internet, forming a robust wireless network. In this network, vehicles are equipped with OBUs that allow them to act as dynamic nodes. This enables real-time communication between vehicles and network infrastructure, including RSUs and BSs. As a result, road users can receive an immediate update on traffic conditions, allowing them to make informed decisions to avoid congestion.

The internet, represented as a cloud, demonstrates the use of cloud computing services to process, store, and disseminate large amounts of data generated by WSN. This facilitates data management and analysis, and incorporates artificial intelligence (AI) to predict traffic patterns and suggest optimal routes. However, the operation of the global positioning system (GPS) is essential to the system's ability to provide traffic updates specific to the location. By ensuring that all nodes, infrastructure, vehicles, even traffic lights, are interconnected, the system supports GPS functions to update and inform in real time about the condition of the city's roads. The proposed system aims to improve traffic flow, reduce congestion, and make urban transport experiences more efficient.

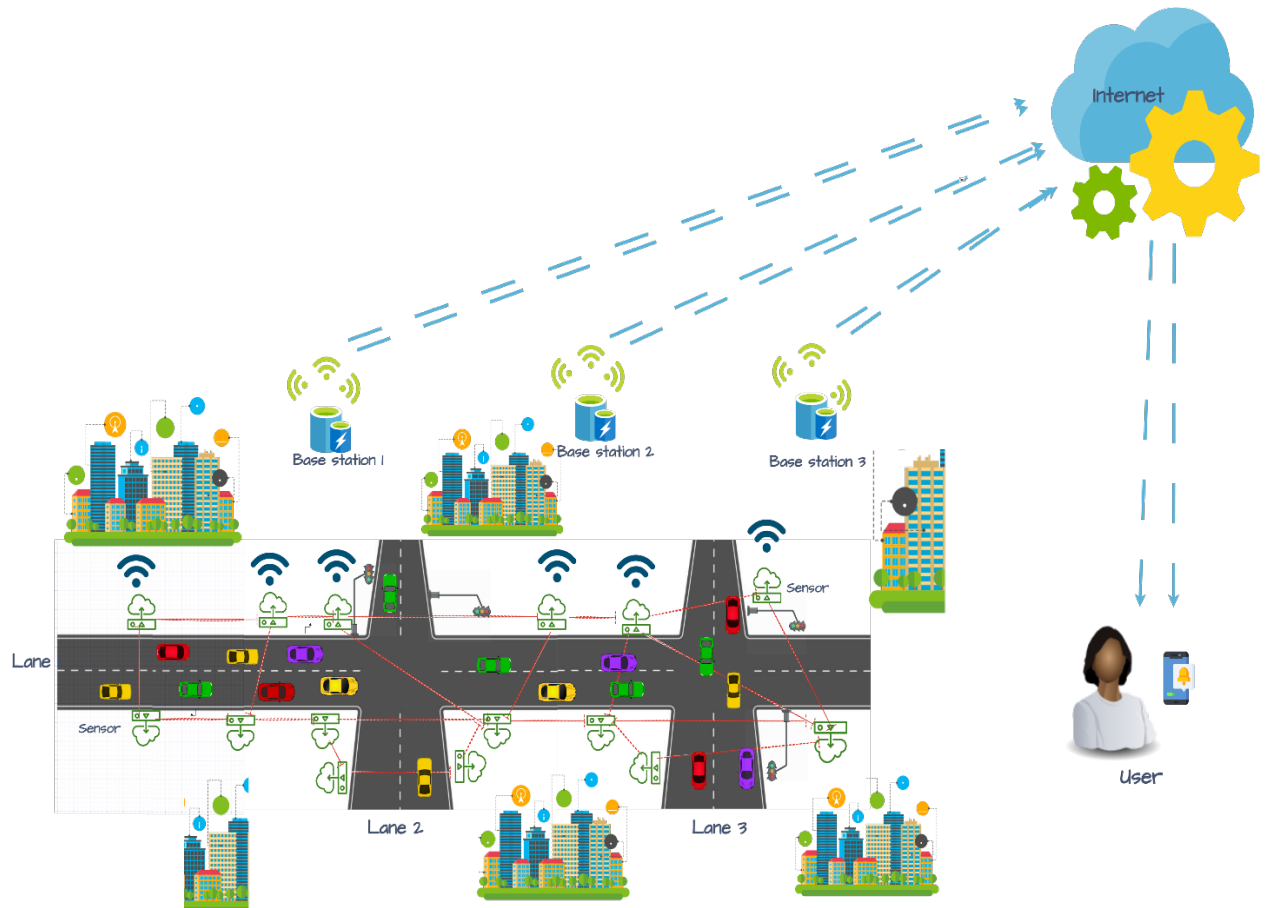


Figure 3. 14: Proposed network architecture

As shown in Figure 3.1., BSs are the central nodes that manage network communication. They serve as access points to the internet and are connected to various sensors and RSUs within their range. They collect, process, and relay information to and from other nodes. RSUs serve as local communication hubs. They are typically connected to BSs and communicate with passing vehicles via short-range wireless communication technologies such as dedicated short-range communications (DSRC) or cellular networks. OBUs are installed in vehicles and allow them to communicate with RSUs and other vehicles through vehicle-to-infrastructure (V2I) or vehicle-to-vehicle (V2V) communication. OBUs can also connect to passenger personal devices, allowing them to interact with the smart city network. Various sensors are deployed throughout the city to monitor conditions. These can include

traffic flow sensors, weather sensors, pollution sensors, and more. OBUs are wirelessly connected to the nearest RSU. The cloud acts as the backbone for data storage, processing, and analytics. Data is then being received from the RSUs and provides the computational power needed for complex tasks such as traffic prediction and system-wide optimization.

3.3. Fusion-based Intelligent Traffic Congestion Control (FITCC) and Traffic Congestion Detection (TCD) Algorithms

This research study integrates existing solutions, known as the fusion-based intelligent traffic congestion control (FITCC) and traffic congestion detection (TCD) algorithms when designing and implementing the proposed PBCC algorithm, to minimize the traffic congestion that occurs in the smart city.

3.3.1 Fusion-based intelligent traffic congestion control (FITCC) algorithm.

The proposed PBCC algorithm integrates the FITCC and TCD algorithms to overcome traffic congestion within smart cities, leading to traffic density, time delay, and air and noise pollution. The proposed PBCC algorithm uses the FITCC algorithm to estimate and classify the congestion on the road within the smart city. The FITCC algorithm makes use of ML techniques, namely: predictive analytics to predict future trends and events based on historical data. Predictions could, for example, relate to traffic congestion on roads, anomaly detection to identify abnormal patterns or events, and optimization algorithms. Accurate predictions could help to optimize resource allocation and decision-making processes when estimating and classifying the traffic congestion on roads from various lanes within the smart city. Furthermore, this technique implemented by the FITCC algorithm can analyse, identify, and collect the pattern based on the road traffic congestion. This process also allows the road user to be alerted to any traffic congestion that may occur on a certain lane (Lane 1, Lane 2,

or Lane 3) within the smart city. The FITCC algorithm provides innovative services to enable drivers to choose alternative routes with either no or less traffic congestion to avoid any delays caused by traffic jams and traffic light malfunctions. This research study applies the technique used by the FITCC algorithm to estimate possible on-road traffic congestion before occurring. Road users (drivers) may thus be made aware of possible delays on certain lanes within the smart city. Drivers may thereafter decide either to remain in the lane with traffic congestion or to avoid it. However, should the driver decide to remain in the lane with traffic congestion, this may result in traffic jams, time delay, air and noise pollution within the smart city area. The traffic congestion may be calculated using Equations (3.1) and (3.2).

$$T_p = \frac{1}{2} \sum_{k=1}^{v_i} -Pi \left(1 + \overline{P_e^J}\right) \quad (3.1)$$

where T_p represents the estimated total number of vehicles within the city, v_i represent the set of vehicles in motion within the city, and Pi represents the vehicles which are stationary. Meanwhile $\overline{P_e^J}$ represents the possibilities for traffic congestion within the city.

$$C_n = T_p + 1 \left(D_p * \left(\frac{1}{t_s - 1} \right) \right) + \left(1 + \frac{1}{t_s} \right) * 0.2 \quad (3.2)$$

where C_n reflects the classified number of vehicles detected in the incoming lanes within the city, and D_p represents the delay caused by the incoming vehicles, while t_s denotes the time per second taken by the incoming vehicles. Furthermore, the technique implemented to estimate the possible traffic congestion in the city before occurring can be defined by Equations (3.3) and (3.4).

$$ML_{ac} = Pi + 0.1 \left(\frac{C_n}{t_s - 1} \right) \quad (3.3)$$

$$ML_{ac} = \left(\frac{c_n}{t_s - 1} + Pi \right) * \frac{0.1}{t_s + 1} \quad (3.4)$$

where ML_{ac} represents the machine-learning technique that monitors the movement of vehicles within the city. Later, in Section 3.3.4, this study shows the integration of all the algorithms used to design the proposed PBCC algorithm.

3.3.2 Traffic congestion detection (TCD) algorithm

The proposed PBCC algorithm is designed and implemented to detect traffic congestion based on the features such as speed, occupancy, and traffic behaviour within the city. The TCD algorithm detects traffic congestion in real-time without any human intervention or prior knowledge. This algorithm uses three performance metrics to detect traffic congestion, namely: detection rate (DR), false alarm rate (FAR), and mean time to detection (MTTD). Furthermore, the TCD algorithm uses the average speed to calculate the speed of vehicles that are moving from one place to another within the smart city area.

When the TCD algorithm detects traffic congestion in the smart city, it redirects the road users to other lanes so that there is an increase in flexibility and stability of vehicles movement within the smart city. Therefore, this study implements the TCD algorithm to provide detection of traffic congestion. The TCD algorithm will redirect road users to alternative routes within the smart city, allowing for high flexibility and stability of vehicles movement within the smart city. The imbalance of vehicle movement within the smart city area causes an increase in time delay, congestion rate, accidents, and air and noise pollution. Therefore, the sequence of traffic congestion observation based on speed and occupancy is calculated using Equation (3.5).

$$X = (X_1, X_2, \dots, X_n) \quad (3.5)$$

where X represents the time series of the length based on the given traffic feature, n represents the observation sequence, and $(1, 2, \dots, n)$ represents the denoted time sequence when the observations based on the speed and occupancy were recorded by the sensors within the smart city. The average time of the vehicle's speed is calculated using Equation (3.6).

$$T_t = a + X(1 - a) * T_{t-1} \quad (3.6)$$

where T_t represents the average time calculated using the vehicle's speed within the smart city. To determine the absolute amount of the change in traffic congestion using vehicle speed or occupancy over time is calculated using Equation (3.7).

$$AA_c = \begin{cases} |T_{t-1} - \bar{T}_t| & t = [1, 2] \\ \left| \frac{1}{4} (T_{t+1} - T_{t-2}) \right| & t \in [3 \dots n - 2] \\ |T_{t-1} - T_t + 1| & t = [n - 1, n] \end{cases} \quad (3.7)$$

where AA_c represents the absolute amount of change in traffic behaviour in terms of increasing or decreasing within the smart city. When a significant change in traffic behaviour occurs (for example, when the traffic speed goes from low to high or vice versa), the AA_c value changes to indicate the potential for traffic congestion. The real-time congestion detection for short-term and long-term averages for AA_c is calculated using Equations (3.8), (3.9), and (3.10).

$$\tilde{\mu} = \sum_{i=0}^{\tilde{w}-1} \left(AA_c + \frac{t_{t-i}}{\tilde{w}} \right) \quad (3.8)$$

$$\mu = \sum_{i=0}^{w-1} \left(AA_c + \frac{t_{t-i}}{w} \right) \quad (3.9)$$

$$\sigma = \frac{0.2}{t_t} \left(\frac{\tilde{\mu} + \mu}{AA_c} \right) * 100 \quad (3.10)$$

where $\tilde{\mu}$ represents the short-term average of AA_c , and μ represents the long-term average of AA_c ; while the σ represents the standard deviation in real-time congestion detection. The $\tilde{w}$ and w represent the sliding-window protocol for short-term and long-term, respectively. The percentage of the detected traffic congestion based on the performance metrics is calculated using Equations (3.11) and (3.12)

$$DR = \frac{\text{No. of congestions detected}}{\text{No. of true congestions}} * 100 \quad (3.11)$$

where DR represents the detection rate percentage of the correct detected traffic congestions in the network.

$$FAR = \frac{\text{No. of false alarms signals}}{\text{No. of non - congestion instance}} * 100 \quad (3.12)$$

where FAR represents the false alarm rate percentage signal to the total number of non-incident instances. The mean time to detection (MTTD) is calculated using Equation (3.13).

$$MTTD = \frac{\sum_{i=1}^N (t_c - t_r)}{N} * 0.01 \quad (3.13)$$

where N represents the number of detected congestions, and t_c represents the alarm time when the congestion was detected; while t_r represents the time when the congestion appeared on the given road segment. The MTTD demonstrated the average of the congestion detection delay at the various times when the congestion was detected by the algorithm, and when it appeared.

3.4. Design of Priority-based Congestion Control (PBCC) Algorithm

As mentioned in Section 3.3, this research study integrates the FITCC and the TCD algorithms to design the proposed PBCC algorithm. In Section 3.4.1, this research study presents the flowchart of the proposed PBCC algorithm.

3.4.1 Priority-based congestion control (PBCC) algorithm flowchart

The proposed PBCC algorithm flowchart is demonstrated in Figure 3.2.

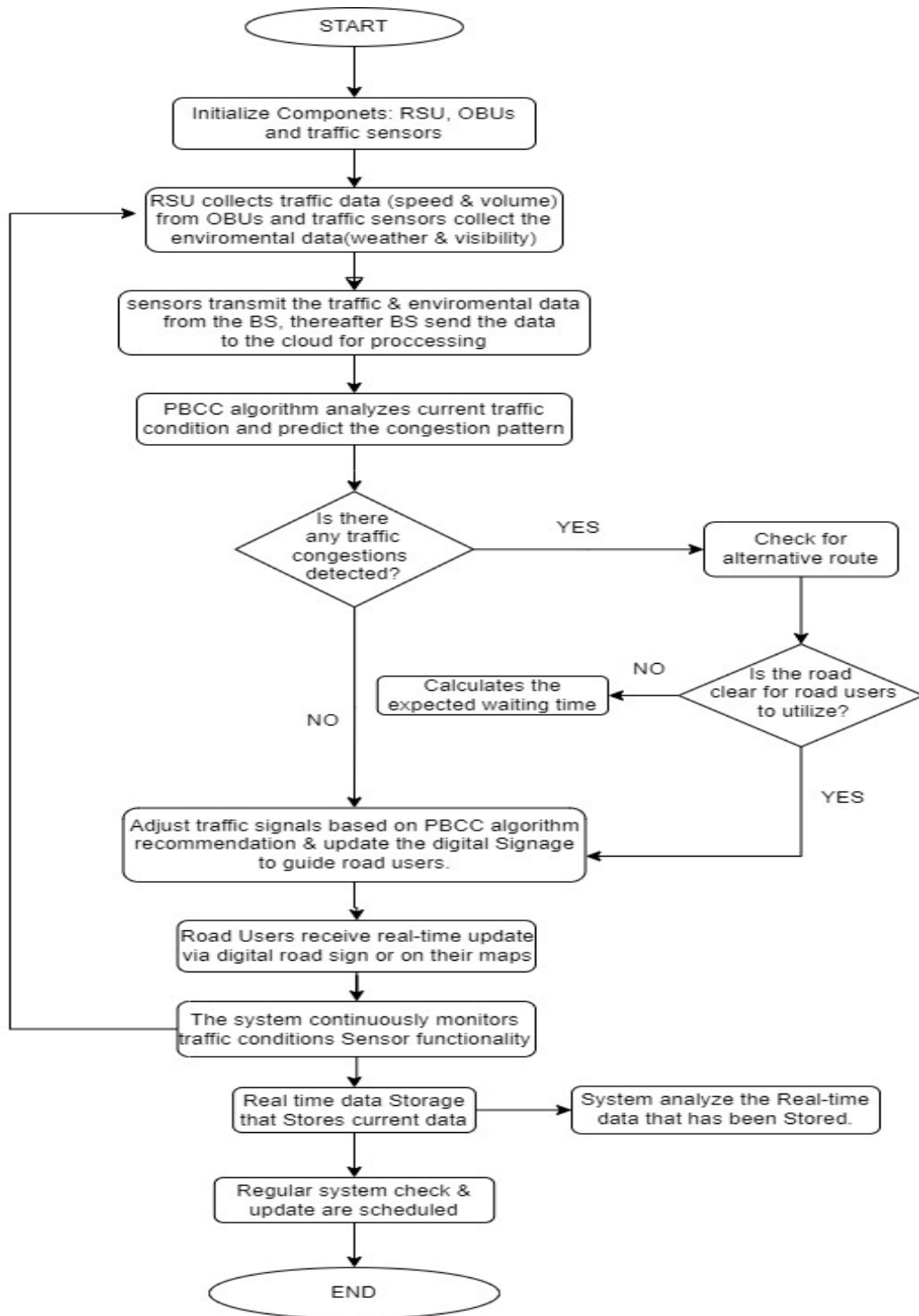


Figure 3. 15: PBCC algorithm flowchart

In this section, this research study outlined a flowchart of the proposed PBCC algorithm. Next, in Section 3.4.2, this research study presents the designed PBCC algorithm.

3.4.2 Proposed priority-based congestion control (PBCC) algorithm

The aim of the proposed PBCC algorithm is to minimize the congestion rate and time delay caused by traffic congestion. Furthermore, this research study aims to reduce air and noise pollution, traffic density, and travel time within the smart city, as given by Algorithm 3.1.

Algorithm 3.1: Priority-based congestion control (PBCC) algorithm

INPUT: Data packets from RSU, OBUs and Sensors

OUTPUT: Traffic Congestion Analysis and Management Actions:

1. Initialization:
2. **Do**
3. Define parameters for RSUs, OBUs, sensors, and constants used for traffic analysis
4. Establish encrypted transmission channels for secure data transfer
5. **WHILE** all nodes in the city lanes are initialized
6. Data-collection and routing process:
7. **DO WHILE** there are data to be collected and routed:
8. **IF** error in data transmission **Then**:
9. Apply the error correction mechanism and retry the transmission
10. **ELSE**
11. Compress data packets and transmit them using efficient batching
12. **LOOP**
13. Traffic congestion detection using FITCC:
14. **FOR EACH** vehicle in the smart city **DO**:
15. Estimate the total number of vehicles (T_p) using Equation (3.1)
16. Classify the number of vehicles detected in the incoming lanes (C_n) using Equation (3.2)
17. Apply machine-learning techniques for predictive analytics and anomaly detection using Equations (3.3) and (3.4) to monitor vehicle movement and to predict congestion
18. **LOOP UNTIL** traffic analysis is complete for all vehicles
19. Traffic Congestion Management using TCD:
20. **DO WHILE** managing detected traffic congestion
21. Utilize the sequence of traffic congestion observation based on speed and occupancy using Equation (3.5)
22. Calculate the average time of the vehicle's speed (T_t) using Equation (3.6)
23. Determine the absolute amount of change in traffic congestion using vehicle speed or occupancy over time with Equation (3.7)

24. Calculate the real-time congestion detection for short-term and long-term average for $\tilde{\mu}$ using Equations (3.8), (3.9), and (3.10)
25. **IF** real-time data indicates high congestion **THEN:**
26. Apply predictive analysis for pre-emptive traffic-flow adjustments
27. Execute real-time feedback actions based on traffic-signal adjustments and alternative routes
28. Calculate the percentage of the detected traffic congestion in the various lanes using Equations (3.11), (3.12), and (3.13).
29. **LOOP**
30. Continuous monitoring and evaluation:
31. **DO**
32. Monitor system performance against the established key performance indicator
33. Schedule regular system updates and maintenance for optimal operation
34. **WHILE** the system is operational
35. **END**

Algorithm 3.1: PBCC algorithm

3.5. Chapter Summary

This chapter presented the design of the proposed PBCC algorithm by integrating the FITCC and the TCD algorithms to minimize the congestion rate and time delay caused by traffic congestion; such leading to air and noise pollution, traffic density, and increased travel time within the smart city. When designing the proposed PBCC algorithm, this research study used the FITCC algorithm to estimate and classify traffic congestion within smart cities. Furthermore, this research study introduced the TCD algorithm into the proposed PBCC algorithm, to detect traffic congestion in real time without any human supervision or prior knowledge within the smart city. This means that the TCD algorithm provides innovative services to enable drivers to use alternative lanes due to traffic congestion caused by traffic light failures, car accidents, etc., within the smart city.

Chapter 4: System Implementation

4.1. Introduction

The implementation of the PBCC algorithm is presented in this Chapter. This research study further effectuated the PBCC algorithm per the satellite protocol alongside the IEEE 802.15.4 (Zigbee) model, using Network Simulator 3 (NS-3) version 35. NS-3 is a versatile network simulator that forms the basic simulations that run virtual machines on the Linux operating system. On the simulation platform, some of the parameters implemented during the simulation process are as follows: congestion level set to maximum or 100%, road length 300m, road speed limit 16.89 m/s–31.07 mph-5-0 km/h, traffic type with 10 - 512 packets size, and delay between the start of 2 vehicles is 2s. The simulation starts distributing packets at 0.5 seconds.

This chapter is structured as follows: In Section 4.2, an overview of wireless network simulators is furnished. Section 4.3 presents an overview of the NS-3 simulation tool. Section 4.4 gives details of the simulation of the PBCC algorithm. Finally, in Section 4.5, the summary of the Chapter is provided.

4.2. Overview of Wireless Network Simulators

Over the years, many researchers have used network simulators to evaluate protocols and applications under fluctuating network configurations. This is because setting up a real network is expensive as well as time-consuming. Network simulators allow researchers to implement and monitor the performance of the existing and newly designed models in a controlled manner. In this section, this research study provides an overview of the most-used network simulators, presenting a comparison of these network simulators. Researchers have applied varying network simulators to simulate communication within the WSNs. These network simulators are used to assess the performance of various methods or algorithms implemented in the study. This

research study chose to use NS-3 because it allows add-ons, modification of protocols, and implementation of multiple interfaces on wireless networks. In addition, this research study provides the key characteristics of some network simulators that have the capability of simulating traffic congestion within WSNs.

4.2.1. Network simulators

The research community chooses the network simulators because of their widespread usage. Most researchers use simulators to validate their proposed solutions, as demonstrated in Section 2.7 of Chapter 2. This research study only discusses open-source network simulators. This research study further discusses the reasons for NS-3 being preferred over other network simulators.

1. Network Simulator-3 (NS-3) is a discrete-event network simulator developed for the simulation of internet systems by the virtual internetwork testbed (VINT) project research group from the University of California. NS-3 is an upgrade of NS-2. NS-3 simulates network protocols with varying network architectures; and it can simulate both wired and wireless networks, as exemplified in Figure 4.1. NS-3 also provides support for the simulation of protocols such as NS-2, namely, the transmission control protocol (TCP), user datagram protocol (UDP), file transfer protocol (FTP), hypertext transfer protocol (HTTP), etc., over wired and wireless (local and satellite) networks (Munthali, 2024). In addition, NS-3 uses C++ only to implement the models and network topologies (Raghavi et al., 2024; Munthali, 2024). Furthermore, the authors demonstrated that NS-3 applies an object tool command language (OTCL/OTcl) to create a simulation interface. As a result, designing and implementing a network topology using NS-3 has become simpler, holding as it does some of the features and tools of the NS-2.

The benefits of the NS-3 include the network animator (NAM), whose interface consists of control features that allow users to play, forward, pause, and stop the simulation at any time (Fahmy, 2023; Raghavi et al., 2024). In addition, NS-3 allows for add-ons, modification of protocols, and implementation of multiple interfaces on a wireless network. Presently, NS-3 is becoming one of the most popularly used network simulators (Ramonet et al., 2024). The authors outlined that this is because there is an online support group for users to assist one another with their projects.

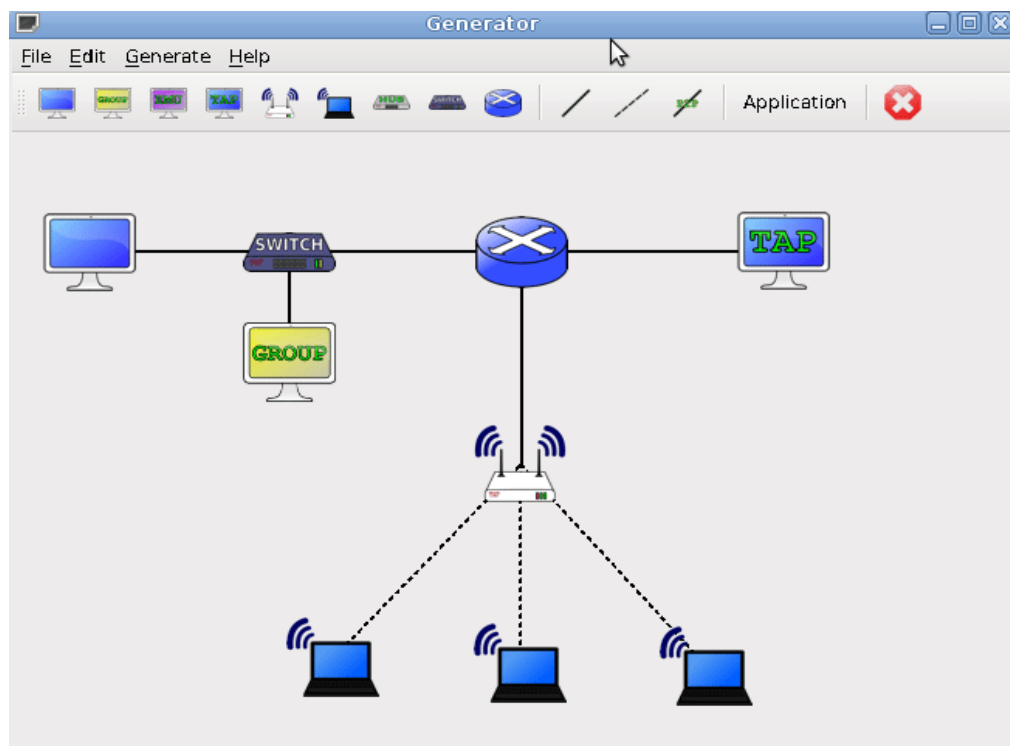


Figure 4.1: NS-3 simulation interface

2. Network Simulator-2 (NS-2) is also a discrete event network simulator developed for network simulation (Sharma et al., 2020). NS-2 also simulates network protocols with varying network architectures; and it can simulate both wired and wireless networks, as illustrated in Figure 4.2. NS-2 further provides support for the simulation protocols similar to NS-3, namely, TCP, FTP, HTTP, etc., over wired and wireless (local and satellite) networks (Sharma et al., 2020;

Suthar & Bhavsar, 2023). However, NS-2 currently struggles with scalability due to its older architecture; this may lead to performance issues, such as slower processing times or higher memory consumption. Siddiqui et al., (2022) outlined that NS-2 lacks comprehensive support for newer networking standards and protocols, such as modern 5G or Internet Protocol version 6 (IPv6) implementations. NS-2 makes use of C++ to carry out core communication functions (de Farias Medeiros et al., 2020).

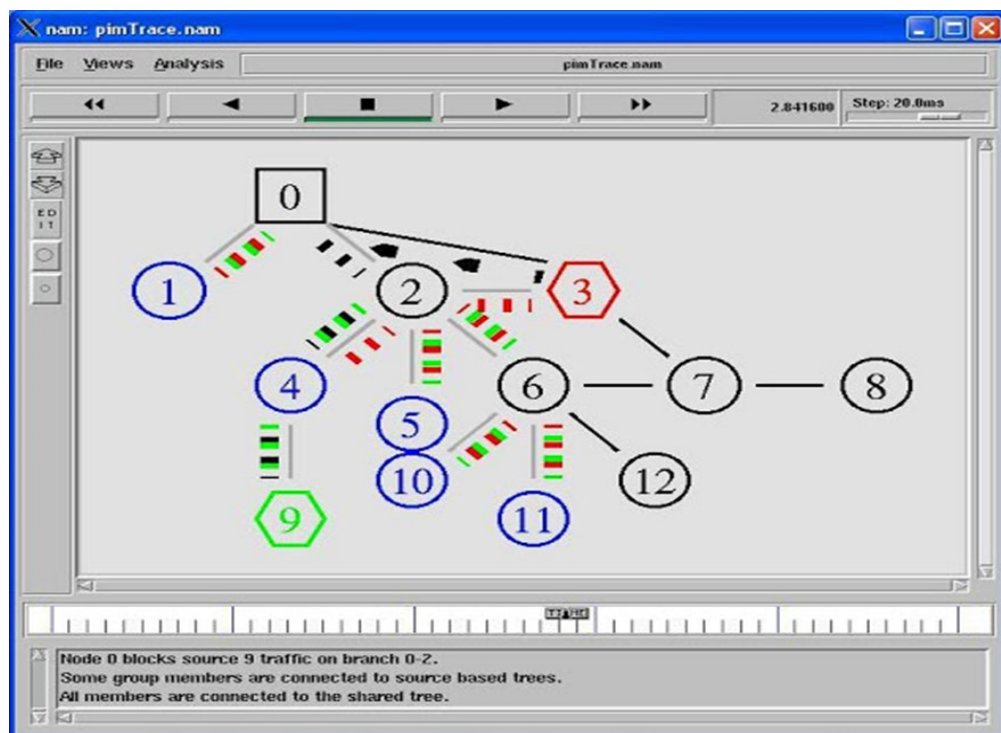


Figure 4.2: NS-2 simulation interface

3. Global mobile information systems simulation library (GloMoSim) is a simulation tool designed to implement large-scale wireless and wired networks (Walia et al., 2022). GloMoSim design follows a layered approach similar to the open systems interconnection (OSI) seven-layer model; and provides support to varying protocols at each layer. GloMoSim supports asymmetric and multicast path configurations (Walia et al., 2022; Gomez et al., 2023). The QualNet simulator is the most widely used commercial version of GloMoSim (Kumar & Singla, 2022). QualNet can simulate a large-scale network with

multiple nodes of heterogeneous communication paths, including satellite and multi-hop wireless communication, as illustrated in Figure 4.3. However, the setting up of QualNet on the Linux operating system is very difficult.

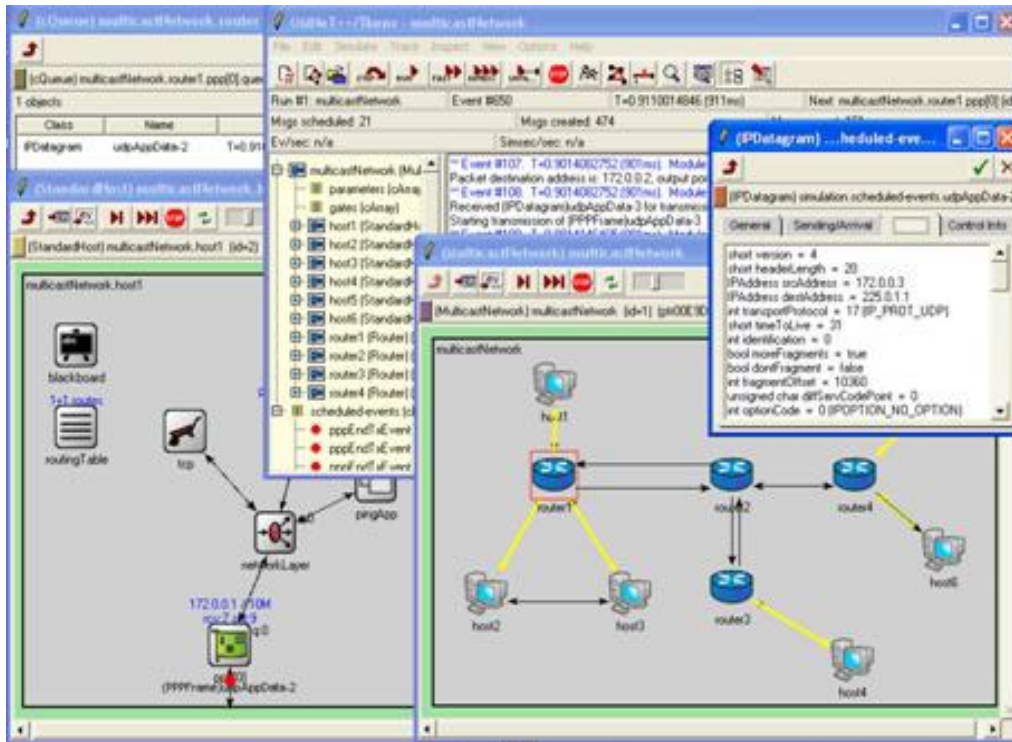


Figure 4.3: QualNet simulation interface

4. Optical micro-networks (OMNeT++) – is an expendable, modular, component-based C++ simulation library and framework, mostly used for building a larger-scale network with multiple nodes, as exhibited in Figure 4.4. OMNeT uses C++ to programme the components (modules) before assembling them into larger components and models using a high-level language (Raghunandan et al., 2022; Peşkırcioğlu Gökçe et al., 2024). However, OMNeT does not support a number of protocols; and as a result, it requires more effort to create the simulations in OMNeT++ than in other simulators.

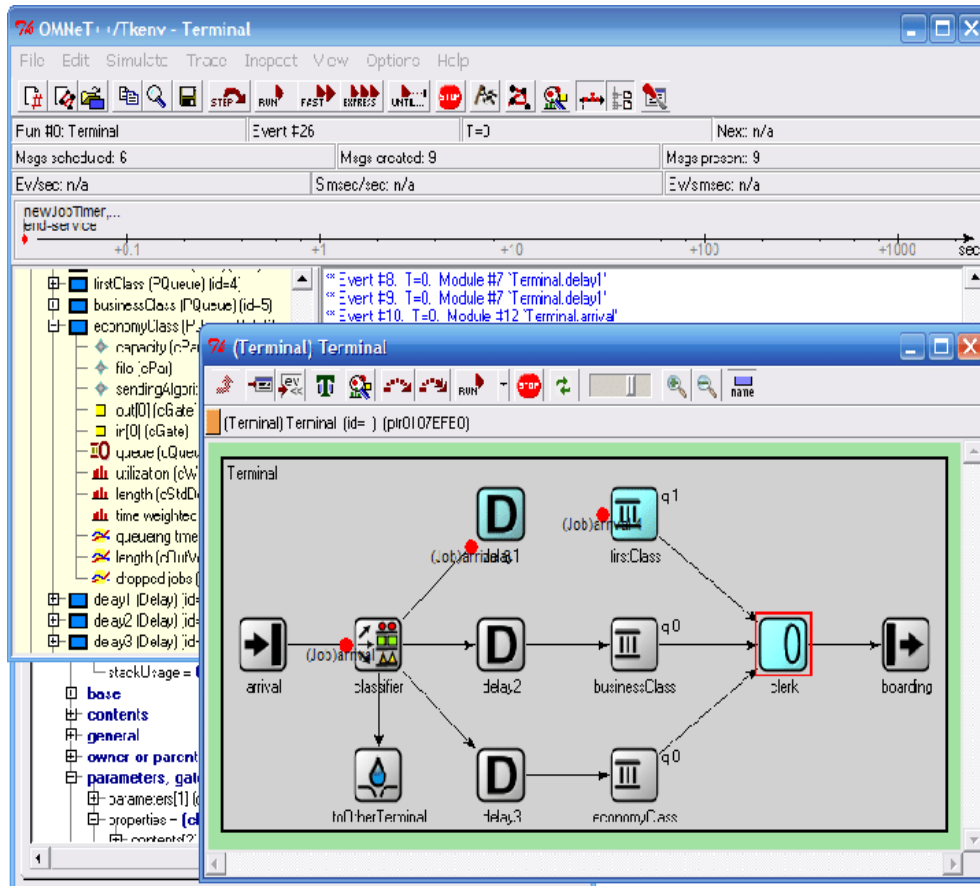


Figure 4.4: OMNeT++ simulation interface

5. MATrix LABoratory (MATLAB) is a multi-paradigm numerical computing environment and fourth-generation programming language developed in the 1970s by Cleve Moler, the chairperson of the computer science department at the University of New Mexico (Veerakumari & Resmi, 2017). The authors demonstrated that MathWorks developed MATLAB to allow matrix manipulations, the creation of user interfaces, implementation of new and existing algorithms, enhancement of existing algorithms, and plotting of data results, as reflected in Figure 4.5.

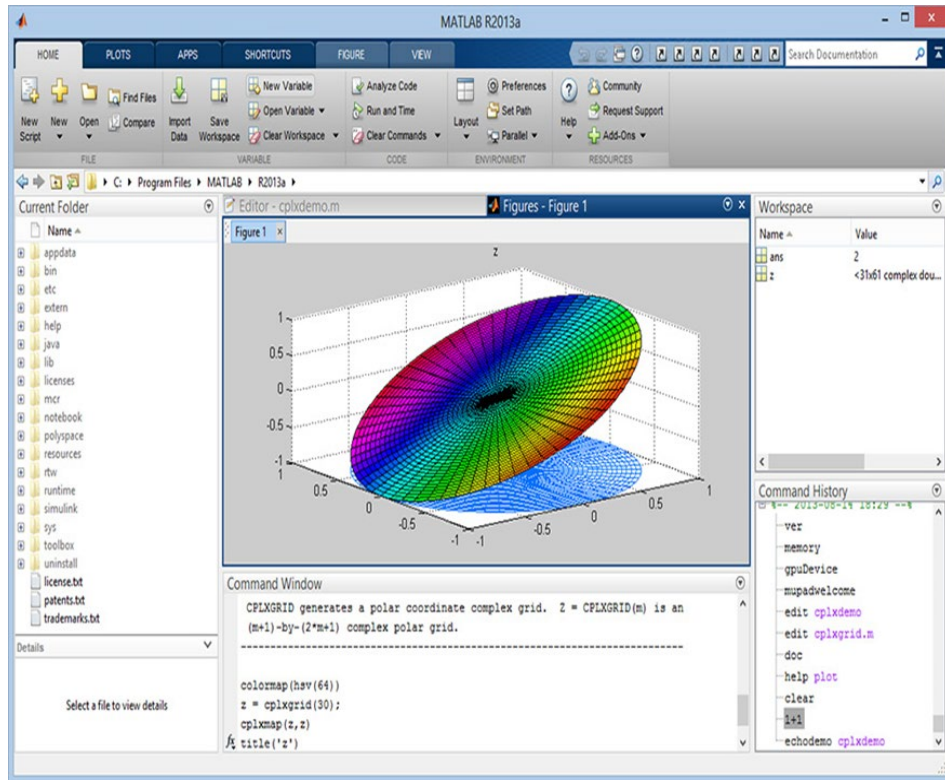


Figure 4.5: MATLAB simulator interface

MATLAB uses various programming languages that include C, C++, Java, FORTRAN, and Python (Srpak et al., 2023). MATLAB uses the MuPAD symbolic engine to allow access to computing capabilities. Moreover, the simulink is an additional package that adds graphical multi-domain simulation (Gowthami et al., 2022; Srpak et al., 2023). Industrial enterprises, academic and research institutions commonly use MATLAB to test their proposed solutions. MATLAB runs on cross-platforms that include Microsoft Windows, Linux, and Mac OS X. Using this software requires deep MATLAB knowledge in order to deal with all programming errors.

Table 4.1: Advantages and Disadvantages of Network Simulators

Network Simulators	Advantages	Disadvantages
NS-3	• Can simulate larger-scale wireless and wired network. • Online support group. • NS-3 provides features not available in NS-2, such as an implementation code execution within the environment.	• Relatively complex and challenging.
NS-2	• Complex scenarios can easily be tested. • NS-2 supports protocols such as TCP, UDP, FTP, HTTP, etc. • NS-2 allows for add-ons and modification of protocols.	• Real system too complex to model. • Bugs are unreliable.
GloMoSim	• Can simulate a larger-scale wireless and wired network. • Supports asymmetric and multicast paths configuration.	• Setting up of QualNet on the Linux operating system is very difficult.
OMNeT ++	• Expendable, modular, component-based C++ simulation.	• It does not support a several of protocols.

	• Can simulate large-scale wireless and wired networks.	
MATLAB	• MATLAB runs on cross-platforms that include Microsoft Windows, Linux, and Mac OS X. • Uses various programming languages that include C#, C++, Java, etc.	• Requires deep MATLAB knowledge to deal with all programming errors.

This research study presented the most-used open-source network simulators in this section. The comparison of these network simulators is discussed in the next section.

4.2.2. Comparison of network simulators

This research study provided the characteristics of the discussed network simulators using Table IV. As exemplified in Table IV, NS-3 is suitable for simulating small and large communication networks and is now used by many researchers all over the world. The NS-3 is more easily used to simulate the network environment than other network simulators, namely, MATLAB, OMNet++, and QualNet.

Table 4.2: Comparison of Network Simulators

Features	NS-3	NS-2	QualNet	OMNET++	MATLAB
Easy to set up	Easy	Easy	Difficult	Moderate	Moderate
Ease of use	Moderate	Easy	Difficult	Difficult	Difficult
Scalability	Poor	Poor	High	High	High
Programming Language	Python	C++	C#	C++	C#, C++, Java, Python, & FORTRAN
Larger Networks	No	No	Yes	Yes	Yes
Continuous Development	Yes	Yes	Yes	Yes	Yes
Graphical User Interface (GUI)	Yes	Yes	Yes	Yes	Yes
Obstacles	No	No	No	No	No
Accessibility	Yes	Yes	Yes	Yes	Yes

In addition, NS-3 has an online user-support group, which permits users to help one another with their NS-3 simulation project challenges. NS-3 is a graphical user interface (GUI)-based simulator that supports the simulation of all IEEE 802.11 models. NS-3 also supports the simulation of various routing protocols, queuing

protocols, as well as multicast protocols for both wired and wireless networks. In addition, NS-3 is open source; and its source code is freely available online.

NS-3 has a better wireless channel model than NS-2: it has an improved interference model; while QualNet, OMNET++, and MATLAB have the same interference model as NS-2. Furthermore, the advantage of passive scanning is low power and bandwidth requirements. However, this may increase the latency of the discovery phase for low-activity channels, as it does not proactively generate traffic in NS-2, NS-3, QualNet, OMNET++, and MATLAB. Since the risk occurs of switching to the channel immediately after the beacon, the listening time in NS-2, NS-3, QualNet, OMNET++, and MATLAB must be at least the same as the beaconing interval of the protocol. This prevents the channel from being considered empty while nodes are working in it. In addition, NS-3 has a built-in IEEE 802.21 standard.

These standard supports for algorithms are used between wired and wireless networks of the same type, as well as for the transferring of packets between various wired and wireless networks. In addition, the IEEE 802.21 standard specifies link-layer intelligence and other related network information to upper layers in order to optimize the traffic congestion in WSNs within the smart city. This makes it easier to implement the proposed PBCC algorithm in NS-3. In the next section, this research study provides an insight into the NS-3 environment.

4.3. The Network Simulator 3 Environment

In this section, this research study provides an insight into the NS-3 environment, which includes the NS-3 basic architecture, directory structure, class hierarchy, and format of trace files which recorded the results of traffic congestion in a smart city.

4.3.1 NS-3 Basic Architecture

NS-3 is made up of different languages, namely, Python and OTcl: Python is for the core functionality of the algorithm, and OTcl is for scripting a network topology. NS-3 also includes the NAM that provides a GUI and visualization of the designed and simulated network, as illustrated in Figure 4.6. In addition, since NS-3 uses Python to implement network algorithms, it was easier to implement PBCC, FITCC, and TCD.

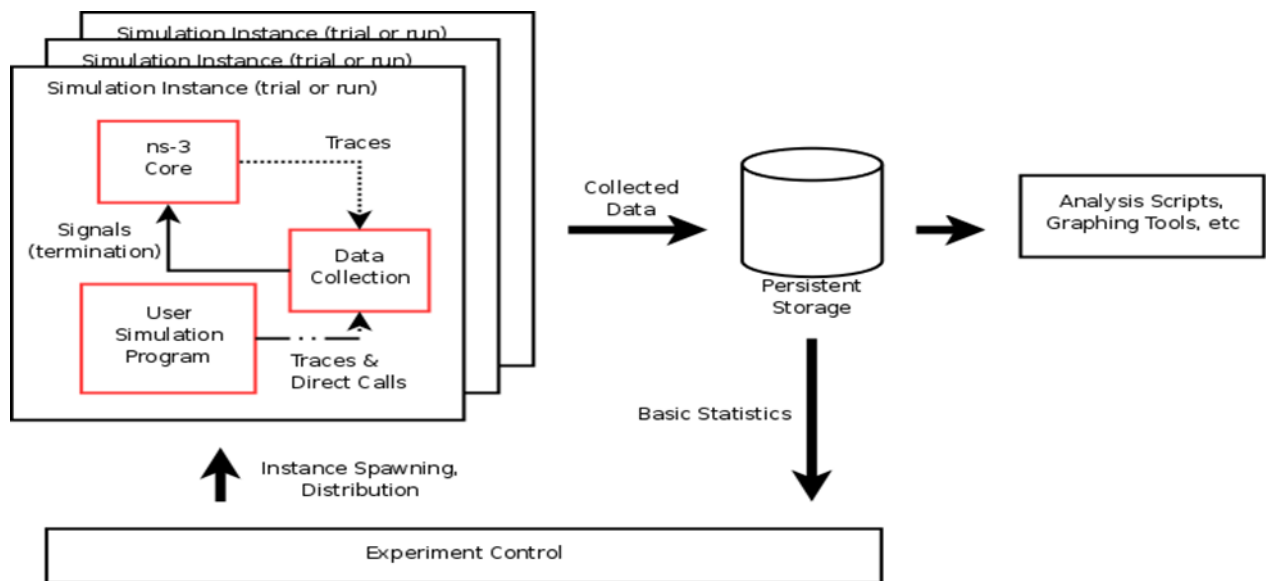


Figure 4.6: Basic NS-3 architecture

4.3.2 NS-3 directory structure

NS-3 software comes in the form of one integrated package, the NS-allinone, in order to simplify the installation; therefore, all the dependent packages are installed from one package. This research study used NS-allinone-3.35. The study illustrated the NS-3 directory structure, as shown in Figure 4.7. Both C++ and Python source code directories are located within the NS-3.35 directory. In order to compile the NS-3 Python source code, this research study executed the following: configure, make, and make instal commands in the terminals.

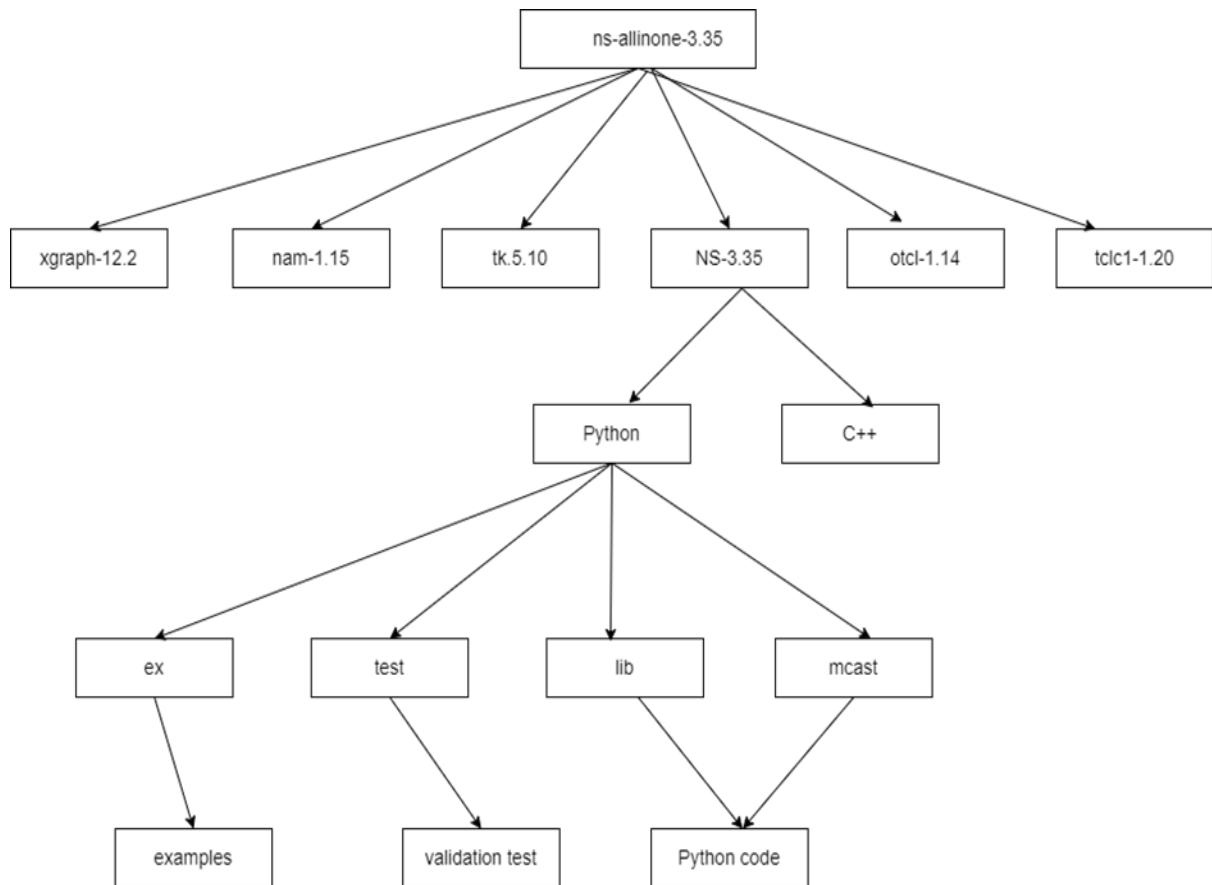


Figure 4.7: NS-3 Directory structure.

4.3.3 NS-3 Class Hierarchy

This research study illustrates a typical NS-3 C++ class hierarchy, as shown in Figure 4.8, which represents the class relationship between all the C++ classes. From the class hierarchy, the root of every object that belongs to the OTcl library is the TclObject. Such comprises a scheduler, timers, network components, and NAM-related objects. The NsObject is the base class for most basic network components that forward packets using two groups, namely, the connector, and the classifier. The connector defines the class of network objects that consist of a single output data route (i.e., downstream route).

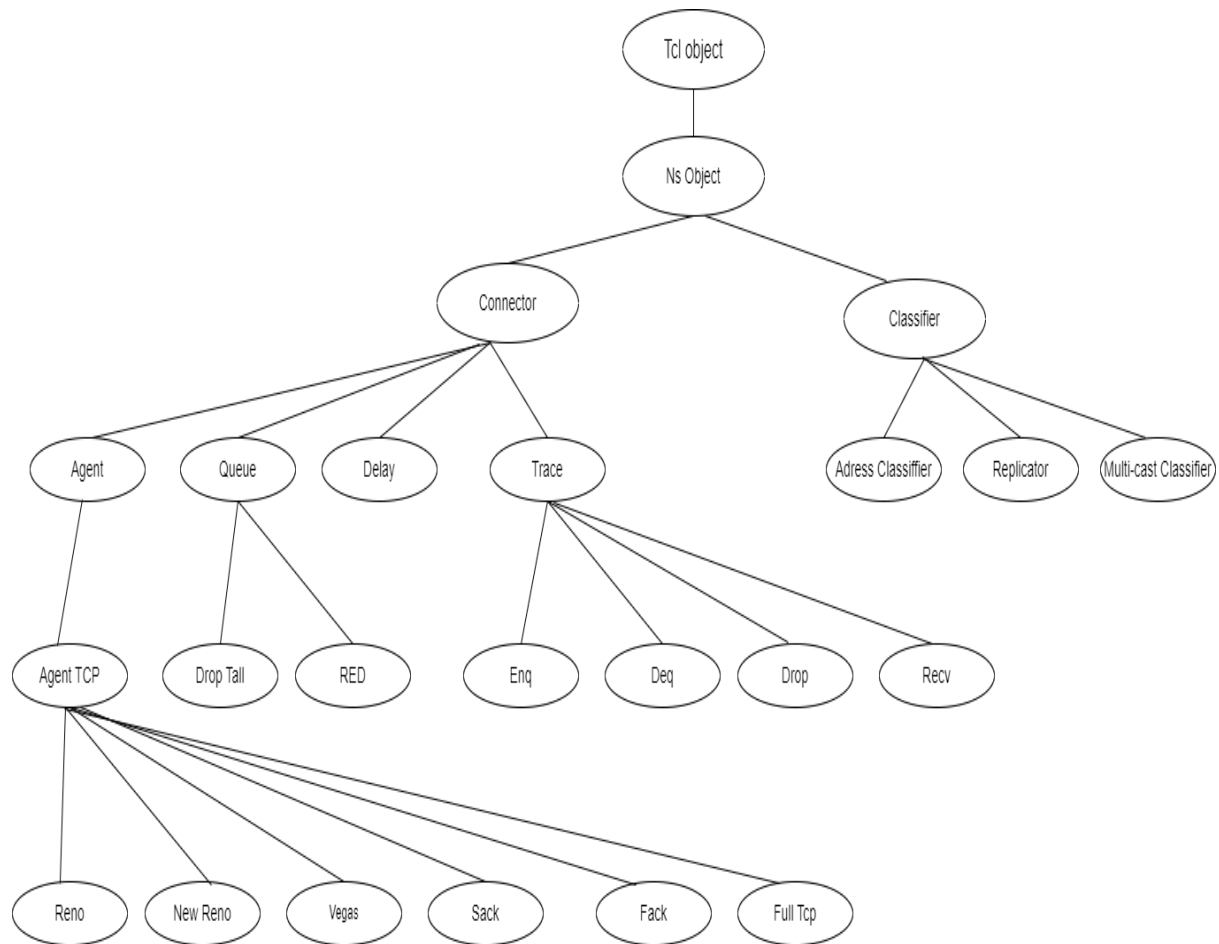


Figure 4.8: NS-3 C++ hierarchy

4.3.4 Trace file output format

A trace file contains information about the data transferred on the network between nodes in NS-3. The `cmu-trace.cc` file in the NS-3.35 directory defines the format for wireless networks. In addition, trace files are auto-generated by the simulator during simulations. In most cases, trace files contain 12 fields, discussed below:

1. Type Identifier **+** indicates enqueue operations, **-** indicates dequeue operations, **r** indicates receive events, **d** indicates drop event, **c** indicates packet collisions at the MAC level.
2. Time indicates the simulated time at which each event occurred, and is measured in seconds.

3. Source node indicates the node that created the packet.
4. Destination node indicates the node that received the packet.
5. Name indicates the descriptive name for the type of packet shared.
6. Size indicates the size of the packet shared.
7. Flags indicate the flag arrays in trace.cc.
8. ID indicates the IP flow identifier.
9. Source and destination addresses indicate the packet's source and destination node addresses, respectively.
10. Seq indicates the sequence number.
11. Uniq ID indicates a unique packet identifier assigned to each packet created.

This research study coded AWK scripts in order to analyse/interpret the trace files data. This helped to gain useful information such as the number of packets lost, and the throughput of the network. AWK is a command/tool available in all the Linux/Unix flavours for text filtering, manipulation, etc. AWK can process text, reporting with greater ease than with other text-parsing tools available in Linux/Unix. The AWK is a programming language designed to process the content of text files, to validate data, to manage small database files, to generate reports, to parse command outputs, and to log files. The AWK can parse more than one file at a time.

4.4. Simulation of the Priority-based Congestion Control (PBCC) Algorithm

In this section, this research study provides insight into the implementation of the proposed PBCC algorithm using the NS-3. The simulation of the PBCC algorithm was conducted to evaluate its performance based on traffic congestion within smart cities

which is caused by a large volume of vehicles, leading to air and noise pollution, traffic density, and travel time.

4.4.1 Network topology

This research study illustrates the NAM simulation scenario based on the topology created for the simulations, as shown in Figure 4.9. This research study implemented 8 BSs within the NS-3 simulator in order to gather information within the smart city before sending it to the cloud. The vehicles are equipped with the OBUs which allows them to act as relay nodes, and thus be able to communicate and send the messages (packets) to the RSUs.

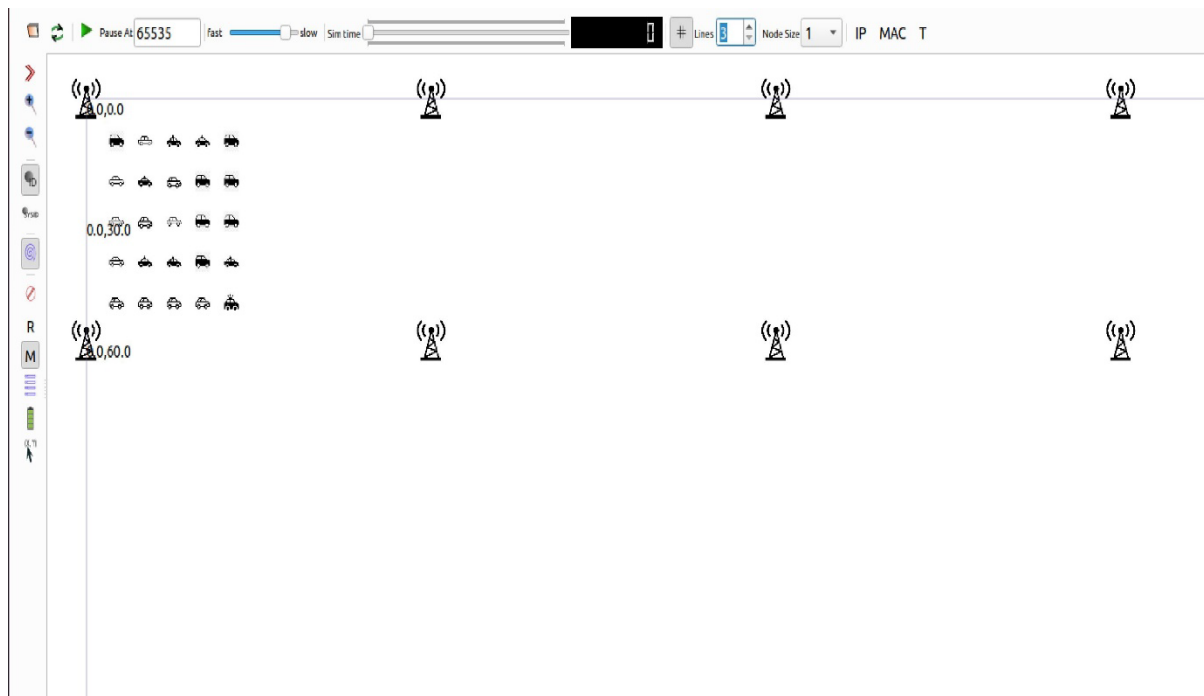


Figure 4.9: Smart city NAM Simulation Scenario 1

The topology illustrated in Figure 4.9 represents the smart city WSN architecture, used to carry out the simulations. In these simulations, the network topology covered an area of 900m x 900m, with 8 fixed RSUs in the smart city and 25 placed vehicles equipped with the OBUs for communication. The vehicles communicate with one

another using the OBUs and with the RSUs through the same technology. This setup is used to compare the three algorithms, as exemplified in Figures 4.9 and 4.10.

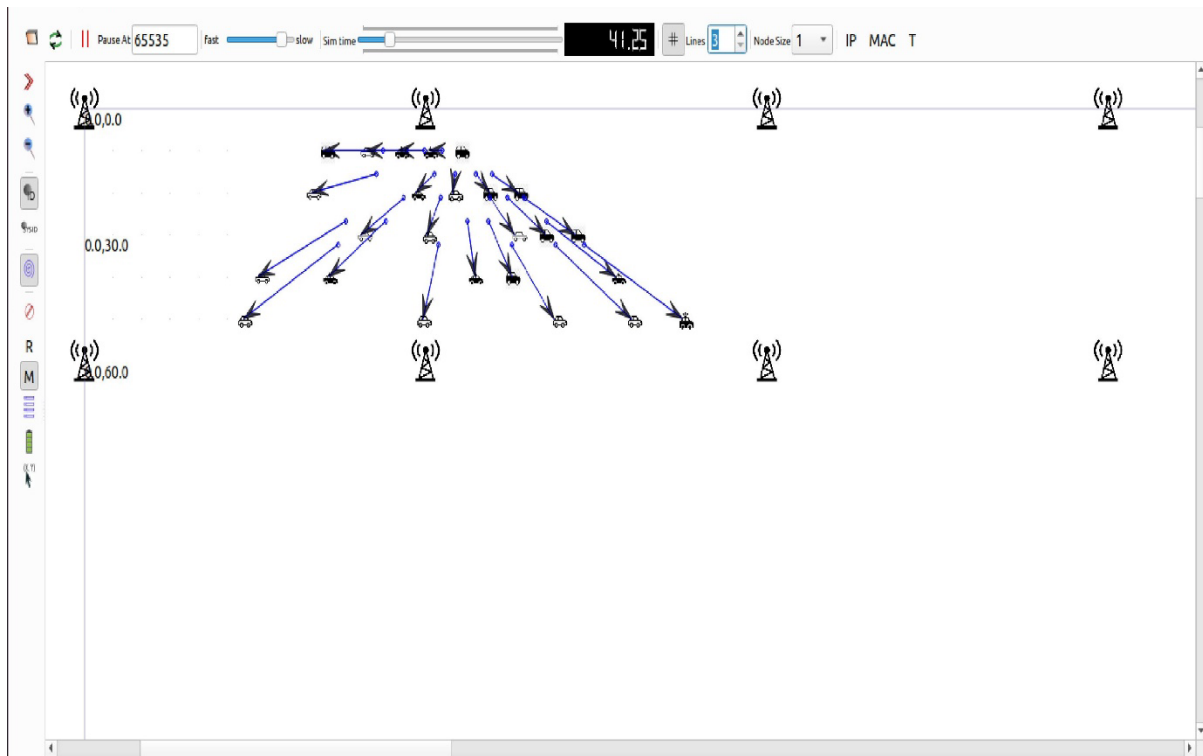


Figure 4.10: Smart city NAM Simulation Scenario 2

The above Figure 4.10 illustrates the communication between the sensor nodes and the RSUs within the smart city. The OBUs send information to the nearest RSUs, which then forward the information to the cloud. As vehicles continue to move, the OBUs communicate with the nearest RSUs, and the RSUs will then share the information about the on-road traffic congestion with the driver, thereafter, being alerted to any related traffic across different lanes within the smart city. This process occurs because, during the simulation configuration, the RSUs were set up as nodes to broadcast the information from the OBUs to the vehicles, whether vehicles are stationary or in motion. As vehicles continue to move along the lanes, the next RSUs gather information about any related traffic from the OBUs, as shown in Figure 4.11.

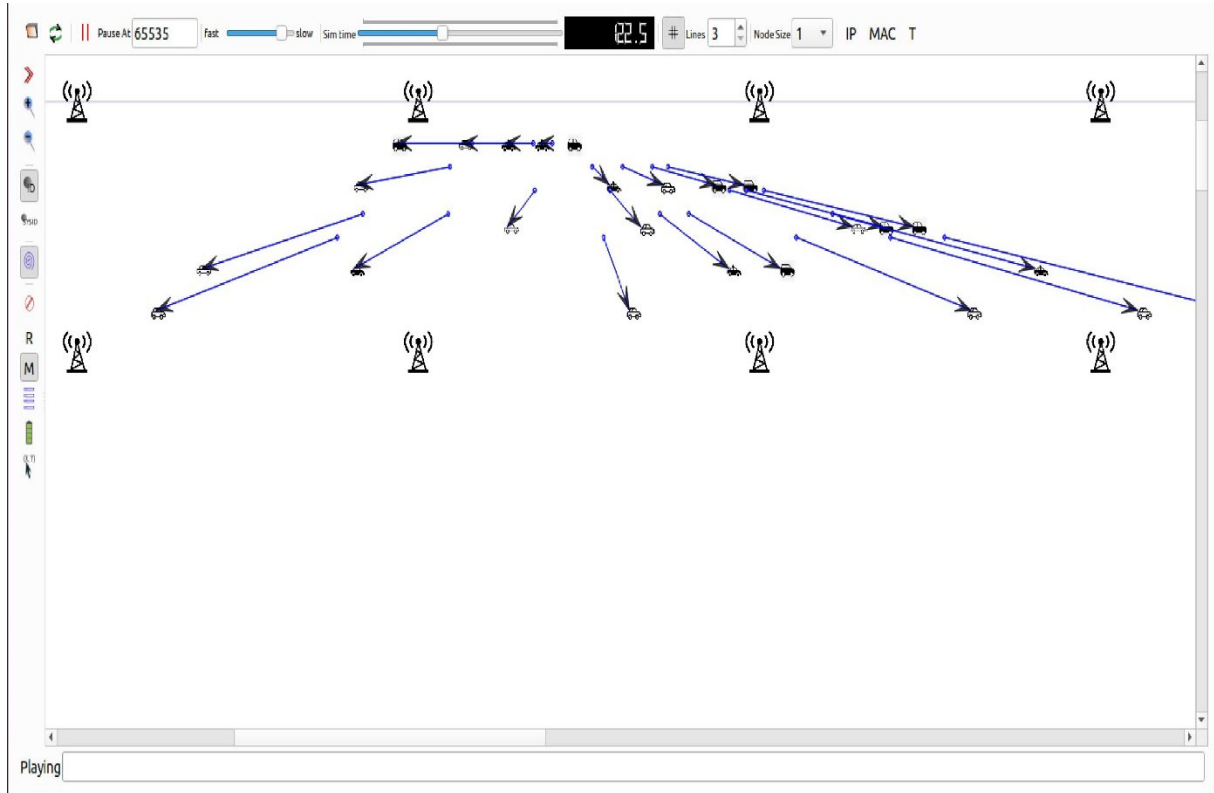


Figure 4.11: Smart city NAM Simulation Scenario 3

The above Figure 4.11 shows other RSUs receiving the information from the OBUs immediately after the vehicles begin to move within the smart city. This means that during the configurations on NS-3, these RSUs are configured to receive and broadcast the information regarding the on-road traffic congestion within the smart city. After receiving the information from the OBUs, the RSUs will broadcast the packets so that the drivers can be notified about the traffic conditions within the smart city. This research study assumes that each RSU simulated within the smart city covers a radius of 900 meters. This research study configured the simulation to start transmitting collected information at 0.5 seconds, and to stop transmitting at 100 seconds across different lanes, as shown in Figures 4.9, 4.10, and 4.11 within the smart city. This research study performed the simulations ten times in order to obtain reliable results.

This research study used NS-3, out.tr script to record performance results gained, in order to compare the three algorithms, namely, the PBCC algorithm, the FITCC algorithm, and the TCD algorithm. The detection rate (DR), false alarm rate (FAR), and mean time to detection (MTTD) are the performance metrics analysed in our simulations. This is because these performance metrics were those that this research study needed most urgently to improve the on-road traffic within the smart city. This research study summarizes the simulation parameters and values used in the simulations in Table V.

Table 4.3: Simulation Parameters

Parameters	Values
Channel Type	Wireless Channel
Network Interface Type	Wireless Phy
Number of Nodes	33
Propagation Model	TwoRayGround
Antenna Model	OmniAntenna
Link Layer Type	LL
Simulation Area	900mX900m
Simulation Time	100 seconds
Bandwidth	20 GB/s for each AP
Communication Standard	TCP
MAC Protocol	IEEE 802.11ac
Queue Type	Queue/DropTail/PriQueue

4.4.2 Dataset structure

The dataset provides a detailed view of traffic data within the smart city (Liao et al., 2023; Azab et al., 2024). Table 4.4 demonstrated the features of a dataset containing over 1.2 million records. The dataset can be found on the link provided within the Footnote¹. This research study used the futuristic_city_traffic dataset. Each record represents a unique snapshot of various features used on traffic conditions within the smart city. The proposed PBCC algorithm used 80% of the dataset for training and 20% for testing.

Table 4.4: Dataset Structure Features.

Smart city	Vehicle Type	Weather	Day of Week	# Hour of Day	# Speed	# Is Peak Hour	Energy Consumption of Vehicle	Traffic Density
	Truck	Rainy	Sunday	20	29.4268	0	14.7134	0.5241
	Bus	Clear	Wednesday	2	118.8	0	143.5682	0.3208
	Autonomous Vehicle	Clear	Wednesday	16	100.3904	0	91.264	0.0415
	Truck	Clear	Thursday	8	76.8	1	46.0753	0.1811
	Autonomous Vehicle	Clear	Saturday	16	45.2176	0	40.1934	0.4544
	Autonomous Vehicle	Rainy	Thursday	20	30.5179	0	37.5562	0.0843
	Autonomous Vehicle	Rainy	Monday	21	43.9222	0	39.042	0.0293
	Autonomous Vehicle	Rainy	Friday	20	20.4236	0	18.1543	0.1393

¹ https://drive.google.com/file/d/1G8VJ_P-9p7yWZC9mg7OzhPt6J98MS2Ql/view?usp=sharing

	Truck	Rainy	Saturday	7	69.9735	0	41.8502	0.1505
	Truck	Rainy	Saturday	15	37.9431	0	18.9715	0.2485
	Truck	Rainy	Saturday	13	29.5809	1	14.1535	0.0626
	Truck	Clear	Tuesday	7	40.4778	1	18.399	0.5179
	Autonomous Vehicle	Clear	Sunday	7	87.8801	0	79.891	0.1248
	Autonomous Vehicle	Rainy	Sunday	10	60.0447	0	57.459	0.0342
	Truck	Clear	Monday	8	70.2698	1	35.1349	0.2389
	Car	Rainy	Friday	9	66.8856	0	66.8856	0.2521
	Autonomous Vehicle	Rainy	Monday	21	126.7644	0	107.1712	0.0784
	Bus	Clear	Friday	21	87.0448	1	118.6975	0.1352
	Truck	Clear	Friday	3	62.3252	0	31.1626	0.2185

The dataset structure presented in Table 4.4 demonstrates the various features used during training and testing. The vehicle type feature represents the various types of vehicles recorded during testing, while the weather feature indicates the conditions, whether it was a rainy day or a sunny (clear) day. Additionally, the dataset includes features such as day of the week, hour of the day, speed, peak hour, energy consumption of vehicles, and traffic density. These features were collected to capture all factors affecting traffic conditions, enabling accurate measurement.

4.5. Summary

This research study in this chapter provided a detailed discussion regarding the comparison of the network simulators, system implementation, and simulation of the proposed PBCC algorithm using NS-3. This research study discussed how the results were recorded and which plotting tool was used to graphically display the results obtained during the simulations. Furthermore, this research study outlined the dataset that it used during the training and testing. In the next chapter, this research study presents the simulation results recorded during the several simulations conducted to evaluate the performance of the proposed PBCC algorithm.

Chapter 5: Results and Discussion

5.1 Introduction

Network simulators allow researchers to evaluate protocols and applications under varying network configurations. Simulators allow researchers to model the network in order to gain approximate behaviour of the real communication networks. It is expensive and time-consuming to set up a real network, thus, the researchers in this study used a network simulator to evaluate the performance of the proposed solution against existing solutions. In this chapter, this research study discusses the performance of the proposed PBCC algorithm, based on the results analysed when conducting the simulations with respect to the three performance metrics that this research study considered, namely, DR, FAR, and MTTD. This is because on-road traffic congestion depends on these three performance metrics when vehicles move around a smart city in order to predict and determine the traffic congestion.

This research study used trace files to record the results obtained during simulations, thereafter to analyse the average of DR, FAR, and MTTD within the smart city. This research study used the Python programming language to display the analysed results graphically obtained from the several simulations conducted. The sample Python programming source code is provided. This research study further provides the code AWK written to calculate the total average of the performance metrics considered in this study.

This research study organized the remainder of this chapter as follows: In Section 5.2, this research study presents the computer simulation evaluation. In Section 5.3, the simulation results and discussions pertinent to this research study are reflected. In Section 5.4, this research study presents the computational complexity validation and limitations. Finally, Section 5.5 provides a chapter summary.

5.2 Computer Experimental Evaluation

This research study created a simulation topology using a TCL script to evaluate the proposed PBCC algorithm. In addition, this research study used NAM interface/simulation topology, as explained in Section 4.4, and illustrated in Figures 4.9, 4.10, and 4.11, to observe node movement and transmission of traffic during the simulations. Python programming language was implemented to display the obtained and analysed simulation results in a graphical format. Moreover, this research study coded AWK programmes to calculate the total average of the performance metrics considered in this study, such as the DR, FAR, and MTDD.

5.3 Findings and Discussion

In this section, this research study provides insight into the analysis of the results obtained from the simulations performed to evaluate the proposed PBCC algorithm vis-à-vis the FITCC and the TCD. This research study ran the simulation multiple times during the training and testing in order to present reliable results. The performance metrics analysed in our simulations include:

- DR: occurs when the ratio of true positive and the total nonself samples are identified by the detector set, where true positive and false negative are the tallies of true positive and false negative.
- FAR: is the number of false positives relative to the sum of the number of true positives and the false positives. FAR further measures how often a detected target is correctly identified, offering a way of evaluating the accuracy of detection systems in areas such as sensing and traffic monitoring.
- MTDD: is used to calculate the average amount of time it takes to detect a traffic or anomaly within the system after it occurs. To measure the time

taken, the total amount of time it takes to detect the traffic during a given period is added and divided by the number of vehicles detected.

This research study compared the PBCC algorithm with the FITCC and the TCD mechanisms. This research study chose the FITCC, mainly because the algorithm managed to collect traffic data and route the traffic on available routes in smart cities to alleviate traffic congestion which causes air and noise pollution. Furthermore, the proposed algorithm makes use of ML techniques to allow drivers to view remotely the traffic flow and the volume of vehicles available on the road, intending to avoid traffic jams, as discussed in Section 3.3.1. The TCD algorithm was chosen because it can detect the possible traffic congestion before it occurs, based on traffic features such as speed, occupancy, and flow, as discussed in Section 3.3.2.

Both the FITCC and the TCD use multiple parameters to on-road traffic congestion within the smart city. The review of the literature claims that the FITCC and the TCD both work best when it comes to mitigating the on-road traffic congestion within the smart city. Therefore, this research study attempted to ascertain whether the proposed PBCC algorithm could outperform those two algorithms.

This research study evaluated the proposed PBCC algorithm, FITCC, and TCD algorithms using two scenarios based on weather conditions, namely, rainy days and sunny days within the smart city environment. Scenario 1 deals with the data collected during a sunny day, while Scenario 2 deals with the data collected during a rainy day. This study separated the results for both the sunny day and the rainy day in order to demonstrate the differences.

5.3.1 Scenario 1: Performance evaluation for sunny day

5.3.1.1 Average detection rate

The DR occurs when the ratio of true positive and the total nonself samples are identified by the detector set, where true positive and false negative are the tallies of true positive and false negative. Therefore, it is important that this research study improves the detection rate in order to reduce traffic congestion within the smart city. The proposed PBCC algorithm was compared with both the FITCC and the TCD algorithms to test the effectiveness of the proposed algorithm, as illustrated in Figure 5.1. Furthermore, Figure 5.1 compares the average detection rate of the three algorithms based on a sunny day.

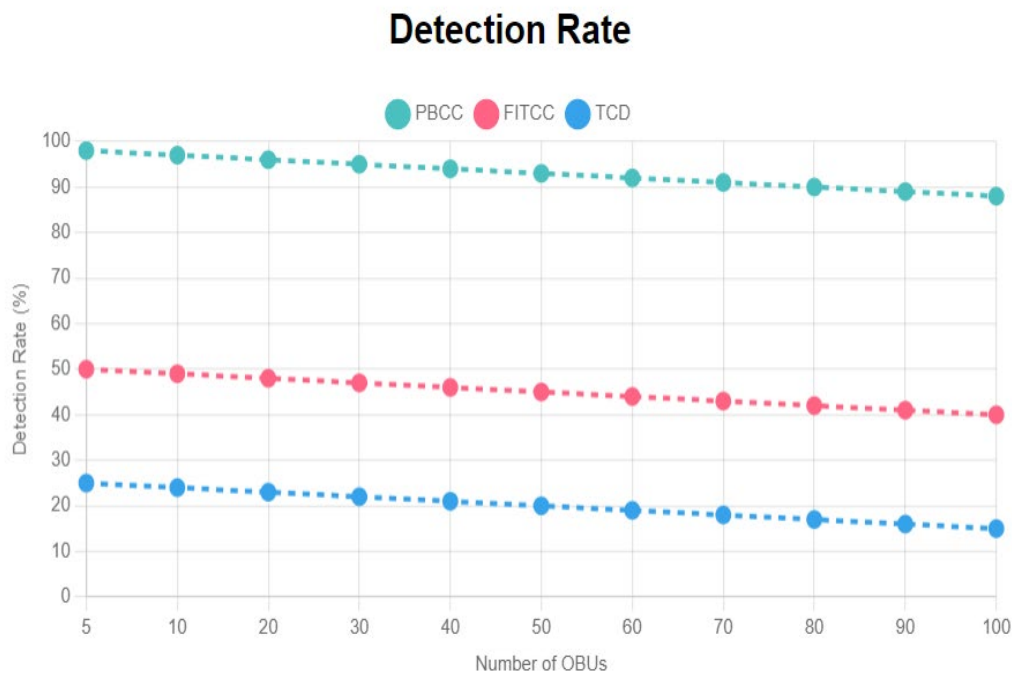


Figure 5.1: Average detection rate.

The simulation results showed that the proposed PBCC algorithm, FITCC, and TCD, each experienced different detection rates of 84.03%, 46.02%, and 18.02%, respectively. The proposed PBCC managed to increase the detection rate by 56%.

The overall experimental evaluations show that, in general, the proposed PBCC algorithm outperforms both the FITCC and the TCD algorithms in terms of detection rate. The process of estimating the total number of vehicles and the traffic behaviour in terms of increasing or decreasing within the smart city in Section 3.3 aided this improved performance.

5.3.1.2 Average false alarm rate

Normally, the FAR is based on the number of false positives relative to the sum of the number of true positives and the false positives. The FAR further measures how often a detected target is correctly identified. Therefore, it is important for this research study to reduce the FAR within the smart city. The experimental evaluations for FAR are illustrated in Figure 5.2. This is to determine the effectiveness of the proposed algorithm compared with the FITCC and the TCD algorithms.

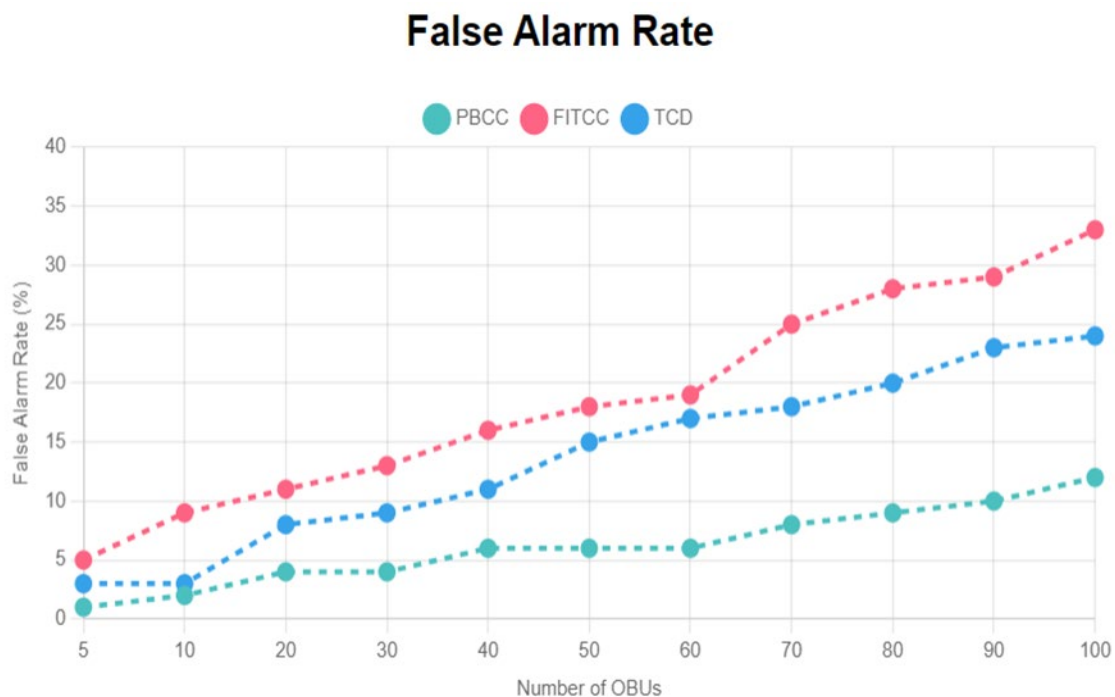


Figure 5.2: Average false alarm rate.

The proposed PBCC algorithm, FITCC, and TCD, each obtained an average FAR of 0.83%, 2.08%, and 5.07%, respectively (See Figure 5.2). The proposed PBCC

algorithm produces less FAR, as it is reduced by 0.6% compared with the others. As can be seen from the experiment, the proposed PBCC outperforms the FITCC and the TCD from the start of the simulation to the end. The reason behind this is that this research study considered the change in traffic congestion using vehicle speed, as defined in Section 3.3; as well as the importance of the data being transferred from OBUs to RSUs within the smart city.

5.3.1.3 Average mean time to detection

In general, the MTTD is used to calculate the average length of time it takes to detect a traffic or anomaly within the system after it occurs. This section discusses the evaluation and analysis of the effectiveness of the MTTD on a sunny day for the proposed PBCC, the FITCC and the TCD algorithms. The proposed PBCC algorithm produces better results for MTTD, as illustrated in Figure 5.3.



Figure 5.3: Average mean time to detection.

In comparing the proposed PBCC algorithm with the FITCC and TCD, the three algorithms obtained the MTTD average by 76.03%, 42.06 %, and 37.04%, respectively

(See Figure 5.3). The PBCC algorithm produced better MTTD average results, demonstrating that it does detect traffic in the smart city on sunny days. The results obtained through the integration of the algorithms was discussed in Section 3.3.

5.3.2 Scenario 2: performance evaluation for rainy day

5.3.2.1 Average detection rate

The DR occurs when the ratio of true positive and the total nonself samples are identified by the detector set, in which true positive and false negative are the tallies of true positive and false negative. Therefore, it is essential that this research study improves the DR in order to manage the traffic congestion within the smart city. According to Figure 5.4, the proposed PBCC algorithm was compared with the FITCC and TCD to test the effectiveness of the proposed algorithm. Furthermore, Figure 5.4 shows the comparison of the average DR for the three algorithms based on a rainy day.

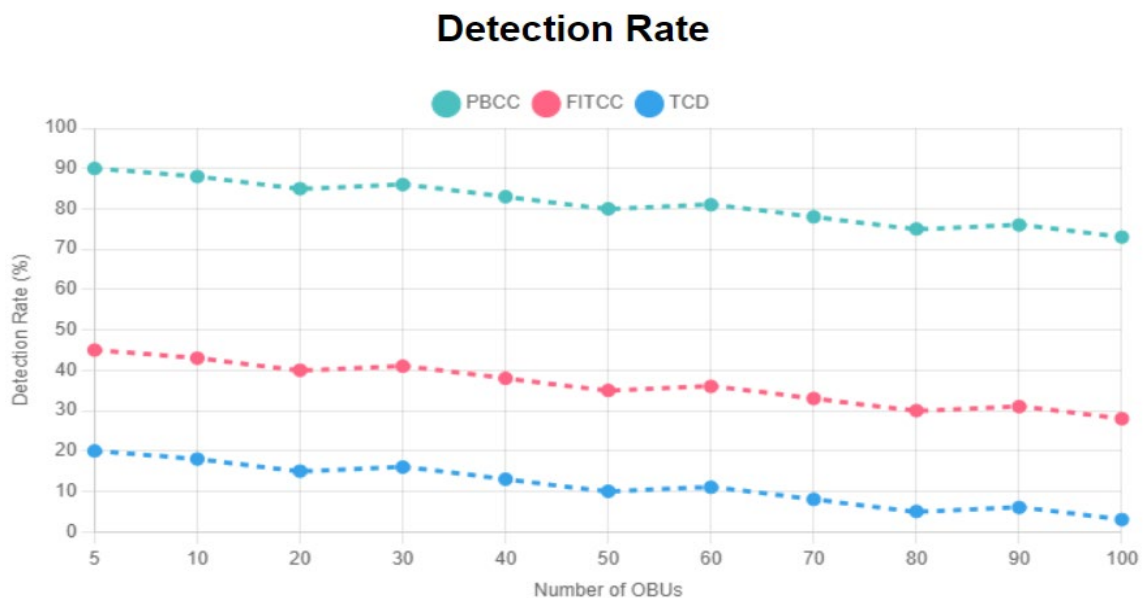


Figure 5.4: Average detection rate

The simulation results showed that the DR of the proposed PBCC algorithm against the FITCC and TCD reflected an average of 78.04 %, 29.02 %, and 7.04 %, respectively.

respectively. In general, experimental evaluations demonstrate that the proposed PBCC algorithm outperforms both the FITCC and TCD algorithms in terms of DR. As a result, the process of estimating the total number of vehicles and the traffic behaviour in terms of increasing or decreasing within a smart city in Section 3.3 aided in the performance improvement.

5.3.2.2 Average false alarm rate

Typically, the FAR is based on the number of false positives relative to the sum of the number of true positives and the false positives. The FAR further measures how often a detected target is correctly identified. Consequently, this research study aims to decrease the FAR during the transmission of data when predicting the traffic congestion within a smart city. The experimental evaluation for FAR is presented in Figure 5.5. The purpose here is to assess the efficiency of the proposed PBCC algorithm compared with that of the FITCC and TCD.

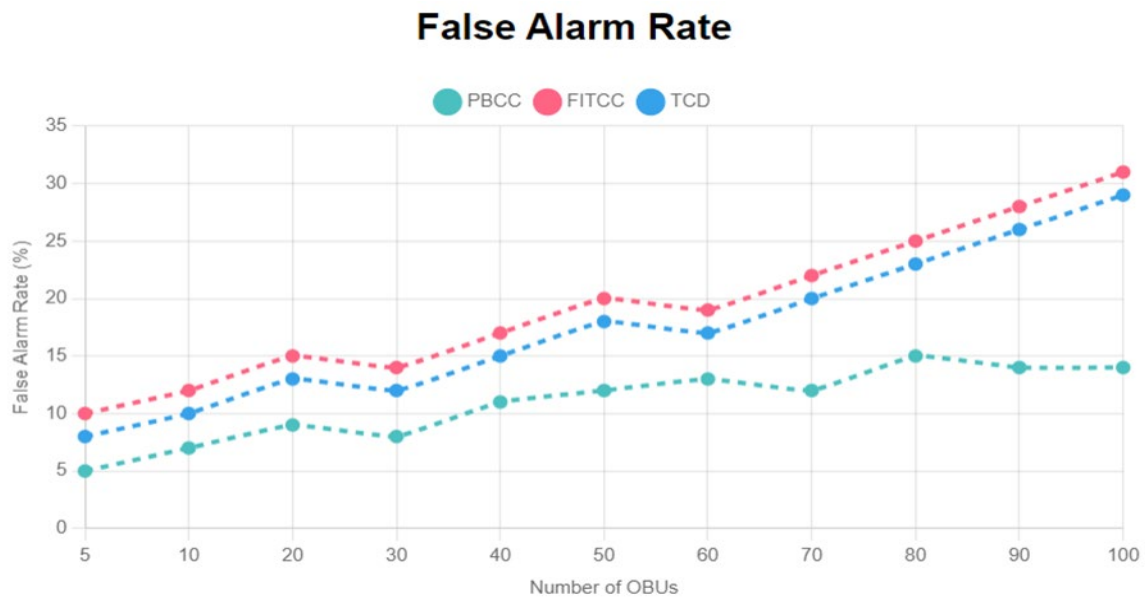


Figure 5.5: Average false alarm rate.

The simulation results demonstrated that the proposed PBCC algorithm decreased the average FAR. The PBCC, the FITCC and the TCD obtained an average FAR of

14.09%, 33.14%, and 28.19%, respectively. The proposed PBCC algorithm decreases the FAR by 39.2%. According to Figure 5.5, these algorithms are compared in terms of rainy-day conditions. The proposed PBCC consistently outperforms the FITCC and the TCD from the beginning to the end of the simulation, as shown in the experiment. The reason behind the success of the proposed PBCC algorithm is that this research study considered the change in traffic congestion using vehicle speed and time, as defined in Section 3.3; as well as the importance of the data being transferred from OBUs to RSUs within the smart city.

5.3.2.3 Average mean time to detection

The concept of MTTD is usually used to calculate the average length of time it takes to detect a traffic or anomaly within the system after it occurs. Furthermore, to measure the time taken, the total amount of time it takes to detect the traffic during a given period is added and divided by the number of vehicles detected. This research study analyses and evaluates the MTTD performance in rainy weather conditions based on a comparison of the proposed PBCC algorithm with both the FITCC and TCD algorithms. Figure 5.6 demonstrates that the performance of the proposed PBCC algorithm remained consistent from the beginning to the end of the simulation.

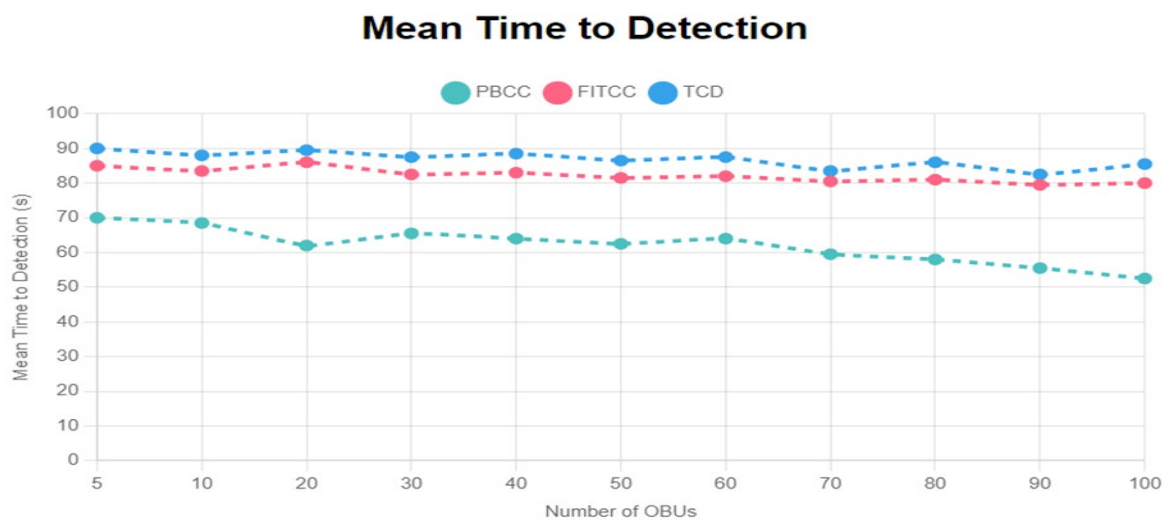


Figure 5.6: Average mean time to detection.

The MTTD average for the proposed PBCC algorithm and that of the FITCC and TCD is seen at 57.03%, 31.02 %, and 26.07%, respectively. The PBCC algorithm produced better MTTD average results, demonstrating that it does detect traffic within the smart city during a rainy day. The results obtained through the integrated of the algorithms, are discussed in Section 3.3.

In this study, the simulation results demonstrated that the proposed PBCC algorithm experienced high DR, improved MTTD, and low FAR for both the sunny days and rainy days within the smart city, unlike the FITCC and TCD, as illustrated by the experimental evaluations. This research study used ML techniques as discussed in Section 3.3 of Chapter 3 to predict the possible traffic congestion within a smart city. The ML techniques allowed fast transmission of data between the OBUs and RSUs. Moreover, the proposed PBCC algorithm obtains better results by integrating the FITCC with the TCD; and uses the ML techniques to ensure the most efficient data distribution within the smart city when predicting traffic congestion. Both the FITCC and the TCD seem to experience more difficulties when it comes to DR, FAR, and MTTD than the proposed PBCC algorithm.

5.4 Chapter Summary

This chapter discussed the computer simulation evaluation, and provided result discussions based on the simulation scenarios provided in Section 5.3. This research study discussed the results of the three-performance metrics considered in this chapter, namely, DR, FAR, and MTTD. The on-road traffic congestion relies on these performance metrics during the transfer of data between the OBU and RSUs within smart cities. As illustrated in Figures 5.1 – 5.6, this research study demonstrated a behavioural comparison of the proposed PBCC algorithm, FITCC, and TCD algorithms. In the next chapter, this research study presents a conclusion and

suggests future work. Furthermore, this research study discusses issues that are not addressed in this research study.

Chapter 6: Conclusion and Future Work

6.1 Introduction

This research study designed and implemented the proposed PBCC algorithm to reduce on-road traffic congestion caused by the high volume of vehicles travelling around smart cities, which leads to time delays, congestion rate, and accidents. The proposed PBCC algorithm is designed by integrating the FITCC with the TCD algorithms to overcome the traffic congestion caused by a high volume of vehicles travelling within smart cities. This research study introduced the FITCC into the proposed PBCC algorithm in order to collect traffic data and to route the traffic on available routes to alleviate traffic congestion in smart cities, via ML techniques. This means that the routing of vehicle technique becomes more intelligent, considering the process of avoiding traffic congestion within the smart city.

This research study organized the remainder of this chapter as follows: In Section 6.2, this research study discusses in detail how the set aim and objectives were achieved. In Section 6.3 a summary or conclusion of this study is presented. The challenges encountered during the study are discussed in Section 6.4. This research study suggests future work in Section 6.5. Finally, Section 6.6 provides a chapter summary.

6.2 Aim and Objectives Achievement

In this section, this research study discusses how the set aim and objectives have been achieved, as outlined in Section 1.5 of Chapter 1.

1. To identify and analyse the current algorithms used for traffic congestion within a smart city.

In Section 2.7, this research study identified and analysed the traffic congestion algorithms previously used. These algorithms helped to avoid the on-road traffic

congestion resulting in time delays, noise pollution, and wasted fuel which increase air pollution within smart cities. This research study identified that the previously proposed traffic congestion algorithms had failed to minimize the on-road traffic congestion during peak hours when there are more vehicles moving around the smart city. As a result, these previously proposed traffic congestion algorithms have caused high-time delays and noise pollution during peak hours within smart cities. This is mainly because these traffic congestion algorithms do not consider the traffic congestion index when estimating the congestion level in a smart city.

This research study proposed an PBCC algorithm to ensure that there is an estimation or prediction of the congestion level using the congestion index during data collection and routing of vehicles within smart cities. This research study designed an PBCC algorithm by integrating the FITCC and the TCD algorithms to reduce the on-road traffic congestion within smart cities.

The proposed PBCC algorithm achieved this reduction by utilizing the ML techniques such as predictive analytics and anomaly detection within the smart city. For predicting traffic congestion and routing of vehicles, the proposed PBCC algorithm used multiple input parameters, as mentioned in Section 1.1 of Chapter 1.

2. To design a PBCC algorithm that reduces traffic congestion within a smart city.

This research study designed an PBCC algorithm by effectively integrating the FITCC and the TCD algorithms in order to lessen the on-road traffic congestion which leads to time delay, traffic density, and air and noise pollution within smart cities, as presented in Section 3.2 of Chapter 3. This research study presented the proposed network system architecture using Figure 3.1. During the design of the proposed PBCC algorithm, this research study used the FITCC to analyse, identify, and collect

the pattern, based on the traffic congestion that occurs on the road within the smart city. The study used the TCD to detect traffic congestion in real-time without any human intervention or prior knowledge. This process ensures reduction of incidents that may occur within a smart city, decreasing the time delay, traffic density, and air and noise pollution.

3. To implement the PBCC algorithm to reduce and control traffic congestion within a smart city thereby improving traffic flow in the city.

This research study used the NS-3 to implement the proposed PBCC algorithm as illustrated in Section 4.4 of Chapter 4. The NS-3 makes use of Python to carry out core communication functions; and deploys an OTcl to create a simulation interface. This research study presented the proposed network topology in Figure 4.9 used in carrying out the simulations.

This research study simulated the PBCC algorithm over ten times to gain convincing results. Moreover, this research study used NS-3.35 out.tr script to record the performance gained when comparing the proposed PBCC algorithm with the FITCC and the TCD algorithms. In addition, this research coded AWK scripts to interpret the trace files data, thus gaining useful information such as the DR, FAR, and MTTD. Moreover, the R programming language was used to display the obtained and analysed simulation results in the form of graphs.

In Chapter 4, Table 4.3 was used to summarize the simulation parameters and values of the simulation. The proposed PBCC algorithm was displayed as the most effective algorithm to mitigate the on-road traffic congestion compared with the FITCC and the TCD. The proposed PBCC algorithm managed to lessen the on-road traffic congestion using parameters such as traffic volume, traffic density, occupancy, traffic congestion index, and travel time within smart cities.

4. To evaluate the effectiveness of the PBCC algorithm compared with the current algorithms.

This research study has evaluated the performance of the proposed PBCC algorithm compared with the FITCC and the TCD within the smart city. In this research study, the metrics performance of simulations analysed include DR, FAR, and MTTD. This is because the mitigation of on-road traffic congestion relies on these three performance metrics within the smart city. Simulation results were presented using Figure 5.1 – 5.6, as illustrated in Section 5.3 of Chapter 5.

The proposed PBCC algorithm outperformed the FITCC and the TCD during performance evaluation. This is because the proposed PBCC algorithm utilizes the ML techniques implemented by FITCC in order to reduce the on-road traffic congestion within smart cities. Furthermore, the proposed PBCC algorithm used the TCD to detect traffic congestion and redirect the road users to the lane with less traffic congestion. This decreases the time delay, congestion rate, accidents, and air and noise pollution while increasing the flexibility and stability of vehicle movement in comparison with the FITCC and the TCD within the smart city.

6.3 Research Summary/Conclusion

This research study has tracked the expansion of WSNs implemented within the smart city over the past decade. WSNs make use of wireless devices to communicate with one another within the network infrastructure such as in smart cities, homes, and military operations. WSNs make it easier for network users to collect data and monitor their environments. However, the implementation of WSNs within the smart city comes with challenges of traffic congestion which leads to time delays, congestion rate, accidents, and air and noise pollution, among many other features.

This research study proposed and designed the PBCC algorithm to overcome the challenge of on-road traffic congestion within the smart city, which results in time delay, congestion rate, accidents, and air and noise pollution. In Chapter 3, this research study discussed the design of a proposed PBCC algorithm by integrating the FITCC and the TCD to reduce the on-road traffic congestion within smart cities.

This research study used the NS-3 to simulate the proposed PBCC algorithm. The NS-3 is less complex than other simulators such as OPNet, OMNeT, QualNet and MATLAB. In Chapter 4, this research study discussed the implementation of the proposed PBCC algorithm using NS-3. In Chapter 5, this research study provided simulation results and analysis based on three performance metrics, namely, DR, FAR, and MTTD, using Figure 5.1 – 5.6, as illustrated in Section 5.3 of Chapter 5. Simulation results presented using NS-3 demonstrated that the proposed PBCC algorithm produces better results than the FITCC and the TCD based on the on-road traffic congestion within smart cities. The proposed PBCC algorithm demonstrated an average of 0.83% FAR, 76.0% MTTD, and 84.3% DR better than the FITCC and the TCD on a sunny day; while during a rainy day, obtaining the average of 14.09% FAR, 57.03% MMTD, and a 78.04% DR, better than the FITCC and the TCD.

The demonstrated simulation results show that the proposed PBCC algorithm outperforms the FITCC and the TCD in terms of DR, FAR, and MTTD. Thus, the researchers in this study believe that they have successfully achieved the objectives of this study; and that the research will find applications in industries, academia, inter alia.

6.4 Challenges Encountered

This research study required much time to implement the required resources. The researchers had to work hard every day, even during recess and holidays. Evaluating the proposed solution using other simulation tools such as QualNet, OMNeT++, and MATLAB would have taken undue extra time. Hence, this research study used an open-source software named NS-3 to implement the proposed PBCC algorithm. In addition, this research study collaborated with other researchers to help with the required Python programming skills, which this research study needed in implementing the proposed PBCC algorithm in NS-3.

6.5 Future Work

In this section, this research study suggests future work that can be considered to improve the performance of the proposed PBCC algorithm.

1. Future work can focus on evaluating the proposed PBCC algorithm to accommodate various other environments which include complex environments such as township roads, amongst others.
2. Furthermore, the proposed PBCC algorithm performance may be improved by testing it on a real smart city environment.
3. During the collection of data and routing of the on-road traffic, this research study did not consider any security issues that may arise within the smart city. Future work can thus also focus on possibilities of security issues during the collection of data and routing of on-road traffic.

6.6 Summary

In this chapter, this research study discussed how the set aim and objectives were achieved. This research study provided a summary based on the presented simulation results, using Figures 5.1 – 5.6, as indicated in Section 5.3 of Chapter 5. This research study believes that the objectives of this study were successfully attained. This research study further discussed the challenges encountered when conducting the research, and how each challenge was tackled. In addition, this research study suggested future research work, focusing on testing the proposed PBCC algorithm performance on smart cities.

Reference

- ABDULWAHID, H. M. & MISHRA, A. 2022. Deployment Optimization Algorithms in Wireless Sensor Networks for Smart Cities: A Systematic Mapping Study. *Sensors*, 22, 5094.
- ABUHDIMA, E., LIU, J., ZHAO, C., ELQAOUAQ, A., COMERT, G., HUANG, C.-T., PISU, P. & NAZERI, A. H. 2022. Impact of dust and sand on 5G communications for connected vehicles applications. *IEEE Journal of Radio Frequency Identification*, 6, 229-239.
- AGEYEV, D., RADIVILOVA, T., MULESA, O., BONDARENKO, O. & MOHAMMED, O. 2022. Traffic monitoring and abnormality detection methods for decentralized distributed networks. *Information Security Technologies in the Decentralized Distributed Networks*. Springer.
- AGRAWAL, R., FAUJDAR, N., ROMERO, C. A. T., SHARMA, O., ABDULSAHIB, G. M., KHALAF, O. I., MANSOOR, R. F. & GHONEIM, O. A. 2022. Classification and comparison of ad hoc networks: A review. *Egyptian Informatics Journal*.
- AKHTAR, M. S. & FENG, T. 2022. Techniques For Reducing Energy And Delay For Data Aggregation In Wireless Sensor Networks. *EAI Endorsed Transactions on Creative Technologies*, 9, e4-e4.
- AL-HRAISHAWI, H., CHOUGRANI, H., KISSELEFF, S., LAGUNAS, E. & CHATZINOTAS, S. 2022. A survey on non-geostationary satellite systems: The communication perspective. *IEEE Communications Surveys & Tutorials*.
- AL HOMSSI, B., AL-HOURANI, A., WANG, K., CONDER, P., KANDEEPAN, S., CHOI, J., ALLEN, B. & MOORES, B. 2022. Next generation mega satellite networks for access equality: Opportunities, challenges, and performance. *IEEE Communications Magazine*, 60, 18-24.
- ALBATTAH, W., HABIB, S., ALSHAREKH, M. F., ISLAM, M., ALBAHLI, S. & DEWI, D. A. 2022. An Overview of the Current Challenges, Trends, and Protocols in the Field of Vehicular Communication. *Electronics*, 11, 3581.
- ALEKO, D. R. & DJAHEL, S. 2020. An efficient adaptive traffic light control system for urban road traffic congestion reduction in smart cities. *Information*, 11, 119.
- ALGHAZZAWI, D., BAMASAQ, O., BHATIA, S., KUMAR, A., DADHEECH, P. & ALBESHRI, A. 2021. Congestion control in cognitive IoT-based WSN network for smart agriculture. *IEEE Access*, 9, 151401-151420.

- ALIMI, I. A., PATEL, R. K., ABODERIN, O., ABDALLA, A. M., GBADAMOSI, R. A., MUGA, N. J., PINTO, A. N. & TEIXEIRA, A. L. 2021. Network-on-chip topologies: Potentials, technical challenges, recent advances and research direction. *Network-on-Chip-Architecture, Optimization, and Design Explorations*.
- AMODU, O. A., OTHMAN, M., YUNUS, N. A. M. & HANAPI, Z. M. 2021. A primer on design aspects and recent advances in shuffle exchange multistage interconnection networks. *Symmetry*, 13, 378.
- AMOORDON, A., DENIAU, V., FLEURY, A. & GRANSART, C. 2022. A single supervised learning model to detect fake access points, frequency sweeping jamming and deauthentication attacks in IEEE 802.11 networks. *Machine Learning with Applications*, 10, 100389.
- ANGURALA, M., BALA, M. & KHULLAR, V. 2022. A survey on various congestion control techniques in wireless sensor networks. *International Journal on Recent and Innovation Trends in Computing and Communication*, 10, 47-54.
- ANITHA, P., VIMALA, H. & SHREYAS, J. 2023. Comprehensive review on congestion detection, alleviation, and control for IoT networks. *Journal of Network and Computer Applications*, 103749.
- ARAÚJO, G., JUNIOR, J. V. R., RABELO, R. A., DA SILVA, T. A., AMARAL, R. & OOVERNEY, K. G. A Comparative Study Between LTE and WiMAX Technologies Applied to Transmission Power System. 2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2020. IEEE, 2311-2316.
- ARIETH, M., ANURADHA, K., HARIKA, B., ALI, D. N. & CHOWDHURY, S. 2022. Congestion Management of GSTEB Routing Protocol Using K-Means Algorithm in Wireless Sensor Network. *ECS Transactions*, 107, 13147.
- ARORA, V. K., SHARMA, V. & SACHDEVA, M. 2022. On QoS evaluation for ZigBee incorporated Wireless Sensor Network (IEEE 802.15. 4) using mobile sensor nodes. *Journal of King Saud University-Computer and Information Sciences*, 34, 27-35.
- ASHRAF, S., GAO, M., CHEN, Z., NAEEM, H. & AHMED, T. 2022. CED-OR based opportunistic routing mechanism for underwater wireless sensor networks. *Wireless Personal Communications*, 125, 487-511.
- ATTAR, M., REPO, S. & MANN, P. 2022. Congestion management market design- Approach for the Nordics and Central Europe. *Applied Energy*, 313, 118905.

- ATTRAH, N. H., ABDUL-MAJEED, G. H. & ABDULLAH, M. Z. 2021. Implementation of chordal ring network topology to enhance the performance of wireless broadband network. *EUREKA: Physics and Engineering*,(2), 11-18.
- AZAB, A., KHASAWNEH, M., ALRABAE, S., CHOO, K.-K. R. & SARSOUR, M. 2024. Network traffic classification: Techniques, datasets, and challenges. *Digital Communications and Networks*, 10, 676-692.
- BAGAI, N., FADEL, E. & AL-MANSOUR, A. 2022. An Efficient Path Planning Strategy in Mobile Sink Wireless Sensor Networks. *Computers, Materials & Continua*, 73.
- BAI, J., ZHANG, T., WANG, C. & XIE, G. MA-CC: Cross-Layer Congestion Control via Multi-agent Reinforcement Learning. *Science and Information Conference*, 2023. Springer, 659-671.
- BAKIKO, V., POPOVYCH, P. & SHVAICHENKO, V. 2022. Investigation of the Influence of the Interference Distribution Law by Different Types of Modulation for Modern Wireless Technologies on the Electromagnetic Environment. *Вісник НТУУ «КПІ». Радіотехніка, радіоапаратобудування: збірник наукових праць, Вип. 88*.
- BALADOR, A., CINQUE, E., PRATESI, M., VALENTINI, F., BAI, C., GÓMEZ, A. A. & MOHAMMADI, M. 2022. Survey on decentralized congestion control methods for vehicular communication. *Vehicular Communications*, 33, 100394.
- BANERJEE, A., DE, S. K., MAJUMDER, K., DAS, V., CHATTOPADHYAY, S., SHAW, R. N. & GHOSH, A. 2022. Building of efficient communication system in smart city using wireless sensor network through hybrid optimization technique. *AI and IoT for Smart city Applications*, 15-30.
- BANIATA, M. & HONG, J. 2017. Energy-efficient unequal chain length clustering for wireless sensor networks in smart cities. *Wireless Communications and Mobile Computing*, 2017.
- BARTHWAL, N. & VERMA, S. K. 2021. An optimized routing algorithm for enhancing scalability of wireless sensor network. *Wireless Personal Communications*, 117, 2359-2382.
- BASHEER, H. & ITANI, M. 2023. Zero touch in fog, IoT, and manet for enhanced smart city applications: A survey. *Future Cities and Environment*, 9, 5.

- BAWANEH, M. & SIMON, V. 2023. Novel traffic congestion detection algorithms for smart city applications. *Concurrency and Computation: Practice and Experience*, 35, e7563.
- BEKAL, P., KUMAR, P., MANE, P. R. & PRABHU, G. 2023. A comprehensive review of energy efficient routing protocols for query driven wireless sensor networks. *F1000Research*, 12, 644.
- BHARTI, R. K., SUGANTHI, D., ABIRAMI, S., KUMAR, R. A., GAYATHRI, B. & KAYATHRI, S. Optimal Extreme Learning Machine based Traffic Congestion Control System in Vehicular Network. 2022 6th International Conference on Electronics, Communication and Aerospace Technology, 2022. IEEE, 597-603.
- BHOWMICK, S., VOJTECH, J. & VELC, R. Scope and application of Bi-directional EDFA for long distance optical transmissions. Micro-Structured and Specialty Optical Fibres VI, 2020. SPIE, 104-113.
- CAO, S., CHEN, X. & YUAN, B. Overview of short-range wireless communication protocol. 2022 7th International Conference on Computer and Communication Systems (ICCCS), 2022. IEEE, 519-523.
- CENA, G., SCANZIO, S., VAKILI, M. G., DEMARTINI, C. G. & VALENZANO, A. 2023. Assessing the Effectiveness of Channel Hopping in IEEE 802.15. 4 TSCH Networks. *IEEE Open Journal of the Industrial Electronics Society*.
- CHAUHAN, S., SHARMA, A., PANDEY, S., RAO, K. N. & KUMAR, P. IEEE 802.11 be: A review on Wi-Fi 7 use cases. 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO), 2021. IEEE, 1-7.
- CORONADO, E., VALERO, V., OROZCO-BARBOSA, L., CAMBRONERO, M.-E. & PELAYO, F. L. 2021. Modeling and simulation of the IEEE 802.11 e wireless protocol with hidden nodes using Colored Petri Nets. *Software and Systems Modeling*, 20, 505-538.
- DANG, S., AMIN, O., SHIHADA, B. & ALOUINI, M.-S. 2023. From a human-centric perspective: What might 6G be? *Authorea Preprints*.
- DE BEELDE, B., PLETS, D. & JOSEPH, W. 2021. Wireless Sensor Networks for Enabling Smart Production Lines in Industry 4.0. *Applied Sciences*, 11, 11248.
- DEBAUCHE, O., TRANI, J.-P., MAHMOUDI, S., MANNEBACK, P., BINDELLE, J., MAHMOUDI, S. A., GUTTADAURIA, A. & LEBEAU, F. 2021. Data management and internet of things: A methodological review in smart farming. *Internet of Things*, 14, 100378.

DE FARIAS MEDEIROS, D., VILLARIM, M. R., DE CARVALHO, F. B. S. & DE

SOUZA, C. P. Implementation and Analysis of Routing Protocols for LoRa Wireless Mesh Networks. 2020 11th IEEE Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), 2020. IEEE, 0020-0025.

DIXIT, S., DUTTA, A., AGRAWAL, S., OLIVAS, M. A., BHATIA, V., CARRILLO, C. D. A., JHA, P., JABHERA, M., KARNA, A. & KHANGANBA, S. P. Connecting the Unconnecte. 2022 IEEE Future Networks World Forum (FNWF), 2022. IEEE, 1-75.

DORAISAMY, R., MOHARIR, M. & ARUL, R. 2022. Congestion aware and game based odd even adaptive routing in network on chip many-core architecture. *Indonesian Journal of Electrical Engineering and Computer Science*, 28, 962-972.

DURGA, S. L., RAJESHWARI, C., ALLEHAIBI, K. H., GUPTA, N., ALBAQAMI, N. N., BHARTI, I. & BASORI, A. H. 2022. Deep Reinforcement Learning-Based Long Short-Term Memory for Satellite IoT Channel Allocation. *Intelligent Automation & Soft Computing*, 33.

EL HOUSSAINI, M.-A., NABOU, A., HADIR, A., EL HOUSSAINI, S. & EL KAFI, J. 2023. A new predictive approach for the MAC layer misbehavior in IEEE 802.11 networks. *International Journal of Pervasive Computing and Communications*.

ELHILALI, N., BADRI, M. & BOUAMI, M. F. 2023. Evaluation of QoS over IEEE 802.11 Wireless Network in the Implementation of Internet Protocols Mobility Supporting. *International Journal of Advanced Computer Science and Applications*, 14.

FAHMY, H. M. A. 2023. Simulators and emulators for wsns. Concepts, Applications, Experimentation and Analysis of Wireless Sensor Networks. Springer.

FAN, Y., WU, G., LI, K.-C. & CASTIGLIONE, A. 2022. Robust end hopping for secure satellite communication in moving target defense. *IEEE Internet of Things Journal*, 9, 16908-16916.

FATIMA, E., KUMAR, A. & DARBARI, M. 2022. A contemporary study on MAC protocols for wireless mesh network. *Emerging Trends in IoT and Computing Technologies*. Routledge.

- GAO, K., SUN, C., WANG, S., LI, D., ZHOU, Y., LIU, H. H., ZHU, L., ZHANG, M., DENG, X. & ZHOU, C. 2023. Buffer-Based High-Coverage and Low-Overhead Request Event Monitoring in the Cloud. *IEEE/ACM Transactions on Networking*.
- GHAZAL, T. M., HASAN, M. K., ALSHURIDEH, M. T., ALZOUBI, H. M., AHMAD, M., AKBAR, S. S., AL KURDI, B. & AKOUR, I. A. 2021. IoT for smart cities: Machine learning approaches in smart healthcare—A review. *Future Internet*, 13, 218.
- GÓMEZ-TORNERO, J. L. 2022. Smart Leaky-Wave Antennas for Iridescent IoT Wireless Networks. *Antenna and Array Technologies for Future Wireless Ecosystems*, 119-181.
- GOMEZ, J., KFOURY, E. F., CRICHIGNO, J. & SRIVASTAVA, G. 2023. A survey on network simulators, emulators, and testbeds used for research and education. *Computer Networks*, 237, 110054.
- GONG, W., HUANG, Y., ZHAO, J. & LIU, J. 2023. *Pervasive Ambient Communication for Internet of Things*, Springer Nature.
- GOWTHAMI, D., MONISHA, G., SRIMANIKANDAN, K., SUWETHA, A. & THIRUMURUGA, P. 2022. Smart Magical Mirror to Exhibit Programme for User through Image Processing.
- GROVER, A., KUMAR, R. M., ANGURALA, M., SINGH, M., SHEETAL, A. & MAHESWAR, R. 2022. Rate aware congestion control mechanism for wireless sensor networks. *Alexandria Engineering Journal*, 61, 4765-4777.
- GUPTA, N., JAIN, A., VAISLA, K. S., KUMAR, A. & KUMAR, R. 2021. Performance analysis of DSDV and OLSR wireless sensor network routing protocols using FPGA hardware and machine learning. *Multimedia Tools and Applications*, 80, 22301-22319.
- GURURAJ, H., NATARAJAN, R., ALMUJALLY, N. A., FLAMMINI, F., KRISHNA, S. & GUPTA, S. K. 2023. Collaborative Energy-Efficient Routing Protocol for Sustainable Communication in 5G/6G Wireless Sensor Networks. *IEEE Open Journal of the Communications Society*.
- HAJIEE, M., FARTASH, M. & OSATI ERAGHI, N. 2021. An energy-aware trust and opportunity based routing algorithm in wireless sensor networks using multipath routes technique. *Neural Processing Letters*, 53, 2829-2852.

- HASHEMINEJAD, E. & BARATI, H. 2021. A reliable tree-based data aggregation method in wireless sensor networks. *Peer-to-Peer Networking and Applications*, 14, 873-887.
- HEIDARI, A., JAFARI NAVIMIPOUR, N., UNAL, M. & ZHANG, G. 2023. Machine learning applications in internet-of-drones: systematic review, recent deployments, and open issues. *ACM Computing Surveys*, 55, 1-45.
- HU, B., IRSHAD, M., NOMAN, S. M., TANG, X., SONG, C. & HAIDER, S. A. Design and Simulation of Orthogonal Frequency-Division Multiplexing (OFDM) Signaling. 3D Imaging Technologies—Multidimensional Signal Processing and Deep Learning: Methods, Algorithms and Applications, Volume 2, 2021. Springer, 187-197.
- HUANG, D.-W., LUO, F., BI, J. & SUN, M. 2022. An efficient hybrid IDS deployment architecture for multi-hop clustered wireless sensor networks. *IEEE Transactions on Information Forensics and Security*, 17, 2688-2702.
- IBRAHIM, A. A., SALSABIL, S. I. & LAWAL, I. A. 2022. Effective Utilization of An Unused Bandwidth in IEEE 802.16 Network. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, 14, 15-22.
- INGA, E., INGA, J. & HINCAPIÉ, R. 2023. Maximizing Resource Efficiency in Wireless Networks through Virtualization and Opportunistic Channel Allocation. *Sensors*, 23, 3949.
- JAIN, A., DWIVEDI, R. K., ALSHAZLY, H., KUMAR, A., BOUROUIS, S. & KAUR, M. 2022. Design and simulation of ring network-on-chip for different configured nodes. *Computers, Materials & Continua*, 71, 4085-4100.
- JAMSHED, M. A., ALI, K., ABBASI, Q. H., IMRAN, M. A. & UR-REHMAN, M. 2022. Challenges, applications, and future of wireless sensors in Internet of Things: A review. *IEEE Sensors Journal*, 22, 5482-5494.
- JAN, T., AZAMI, P., IRANMANESH, S., AMERI SIANAKI, O. & HAJIEBRAHIMI, S. 2020. Determining the optimal restricted driving zone using genetic algorithm in a smart city. *Sensors*, 20, 2276.
- JAVID, M., QU, X., HUANG, F., LI, X., FARID, A., SHAH, A., DUAN, Y., ZHANG, Z., DONG, X. & PAN, L. 2021. In-situ synthesis of SiC/Fe nanowires coated with thin amorphous carbon layers for excellent electromagnetic wave absorption in GHz range. *Carbon*, 171, 785-797.

- JAYAKUMAR, J., NAGARAJ, B., CHACKO, S. & AJAY, P. 2021. Conceptual implementation of artificial intelligent based E-mobility controller in smart city environment. *Wireless Communications and Mobile Computing*, 2021, 1-8.
- JAYARAMAN, R., RAO, D., KUMAR, M. & MISHRA, A. 2023. Understanding the Salient Features Related To Resource Management in Broadband Wireless Networks. *Resource Management in Advanced Wireless Networks*, 81, 81.
- JING, B., HU, Y. & CHEN, Y. Route Optimization and its Algorithm Based On Computer Network. 2023 World Conference on Communication & Computing (WCONF), 2023. IEEE, 1-6.
- JUBAIR, A. M., HASSAN, R., AMAN, A. H. M., SALLEHUDIN, H., AL-MEKHLAFI, Z. G., MOHAMMED, B. A. & ALSAFFAR, M. S. 2021. Optimization of clustering in wireless sensor networks: techniques and protocols. *Applied Sciences*, 11, 11448.
- KÄHKÖNEN, H., ALA-LAURINAHO, J. & VIIKARI, V. 2022. A modular dual-polarized Ka-band Vivaldi antenna array. *IEEE Access*, 10, 36362-36372.
- KAMPOURAKIS, V., GKIOULOS, V. & KATSIKAS, S. 2023. A systematic literature review on wireless security testbeds in the cyber-physical realm. *Computers & Security*, 103383.
- KAPIL, A. K. & KUMAR, L. 2019. A survey on security threats and application of wireless sensor network. *International Journal of Research in Engineering and Innovation*, 3, 217-222.
- KARMAKAR, G., CHOWDHURY, A., KAMRUZZAMAN, J. & GONDAL, I. 2020. A smart priority-based traffic control system for emergency vehicles. *IEEE Sensors Journal*, 21, 15849-15858.
- KHALIFEH, A., DARABKH, K. A., KHASAWNEH, A. M., ALQAISIEH, I., SALAMEH, M., ALABDALA, A., ALRUBAYE, S., ALASSAF, A., AL-HAJALI, S. & AL-WARDAT, R. 2021. Wireless sensor networks for smart cities: Network design, implementation and performance evaluation. *Electronics*, 10, 218.
- KHALIL, M. & GHANIM, M. 2023. Performance Evaluation of Fiber Optic Hybrid Topology in Modern Communication System. *International Journal of Computing and Digital Systems*, 14, 1-xx.
- KHANNA, A., GOYAL, R., VERMA, M. & JOSHI, D. Intelligent traffic management system for smart cities. *Futuristic Trends in Network and Communication Technologies: First International Conference, FTNCT 2018, Solan, India, February 9–10, 2018, Revised Selected Papers 1*, 2019. Springer, 152-164.

- KUMAR, S., CENGIZ, K., VIMAL, S. & SURESH, A. 2021. Energy efficient resource migration based load balance mechanism for high traffic applications IoT. *Wireless personal communications*, 1-19.
- KUMAR, V. & SINGLA, S. 2022. Performance Analysis of Commonly Used Simulators and Emulators in MANET. *Journal of Positive School Psychology*, 6, 5390-5398.
- KUMARI, P. & KAUR, P. 2021. A survey of fault tolerance in cloud computing. *Journal of King Saud University-Computer and Information Sciences*, 33, 1159-1176.
- LAKSHNA, A., GOKILA, S., RAMESH, K. & SURENDIRAN, R. 2023. Smart Traffic: Traffic Congestion Reduction by Shortest Route* Search Algorithm. *International Journal of Engineering Trends and Technology*, 71, 423-433.
- LEÓN, J. P. A., DE LA CRUZ LLOPIS, L. J. & RICO-NOVELLA, F. J. 2023. A machine learning based Distributed Congestion Control Protocol for multi-hop wireless networks. *Computer Networks*, 231, 109813.
- LI, X., JIANG, C., DU, D., WANG, R., FEI, M., LI, X. & TIAN, Y.-C. 2021. Optimization and control of cyber–physical power systems under dual-network interactive cascading failure. *Control Engineering Practice*, 111, 104789.
- LIAO, X., WU, G., YANG, L. & BARTH, M. J. 2023. A Real-World Data-Driven approach for estimating environmental impacts of traffic accidents. *Transportation research part D: transport and environment*, 117, 103664.
- LIN, J., ZHANG, X., GAO, X., KANG, P., ZHOU, Y., OUYANG, Y. & FENG, T. 2023a. Packet Reordering in the Era of 6G: Techniques, Challenges, and Applications. *Electronics*, 12, 3023.
- LIN, X., WANG, L., XU, H., YANG, M. & CHENG, X. 2023b. Event-trigger rolling horizon optimization for congestion management considering peer-to-peer energy trading among microgrids. *International Journal of Electrical Power & Energy Systems*, 147, 108838.
- LIN, Y., ZHOU, Y., LIU, Z., LIU, K., WANG, Y., XU, M., BI, J., LIU, Y. & WU, J. 2020. Netview: Towards on-demand network-wide telemetry in the data center. *Computer Networks*, 180, 107386.
- LIU, K., TIAN, C., WANG, Q., CHEN, Y., TIAN, B., SUN, W., MENG, K., YAN, L., HAN, L. & FU, J. 2022. PayDebt: Reduce Buffer Occupancy Under Bursty Traffic on

- Large Clusters. *IEEE Transactions on Parallel and Distributed Systems*, 33, 4707-4722.
- MAHMOODI KHANIABADI, S., JAVADPOUR, A., GHEISARI, M., ZHANG, W., LIU, Y. & SANGAIAH, A. K. 2023. An intelligent sustainable efficient transmission internet protocol to switch between User Datagram Protocol and Transmission Control Protocol in IoT computing. *Expert Systems*, 40, e13129.
- MAJID, M., HABIB, S., JAVED, A. R., RIZWAN, M., SRIVASTAVA, G., GADEKALLU, T. R. & LIN, J. C.-W. 2022. Applications of wireless sensor networks and internet of things frameworks in the industry revolution 4.0: A systematic literature review. *Sensors*, 22, 2087.
- MATOLAK, D. W., JAMAL, H., HOSSEINI, N., AFROZE, Z., STOFIK, N., KING, E. & MOHSEN, M. Novel Filterbank Multicarrier Waveform for L-band Digital Aeronautical Communications: Initial Field Test Results. 2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC), 2021. IEEE, 1-10.
- MAZUMDAR, N., ROY, S., NAG, A. & NANDI, S. 2021. An adaptive hierarchical data dissemination mechanism for mobile data collector enabled dynamic wireless sensor network. *Journal of Network and Computer Applications*, 186, 103097.
- MEDEIROS, D. D. F., SOUZA, C. P. D., CARVALHO, F. B. S. D. & LOPES, W. T. A. 2022. Energy-Saving Routing Protocols for Smart Cities. *Energies*, 15, 7382.
- MEMON, S. K., NISAR, K., HIJAZI, M. H. A., CHOWDHRY, B., SODHRO, A. H., PIRBHULAL, S. & RODRIGUES, J. J. 2021. A survey on 802.11 MAC industrial standards, architecture, security & supporting emergency traffic: Future directions. *Journal of Industrial Information Integration*, 24, 100225.
- MIDDA, S. & KONER, R. Application of Low Cost Energy Efficient ZigBee Network Protocol To Develop A Cyber-Physical Intelligent System For Monitoring Mine Slope Health. 2022 OPJU International Technology Conference on Emerging Technologies for Sustainable Development (OTCON), 2023. IEEE, 1-6.
- MISHRA, S., SINGH, V., GUPTA, A., BHATTACHARYA, D. & MUDGAL, A. 2023. Adaptive traffic signal control for developing countries using fused parameters derived from crowd-source data. *Transportation letters*, 15, 296-307.
- MIZUNO, Y., OHISHI, Y. & ISHIKAWA, N. 2023. A simple metric that correlates with public Wi-Fi throughput. *Electronics Letters*, 59, e12795.
- MOHAMED, N., AL-JAROODI, J., JAWHAR, I., IDRIES, A. & MOHAMMED, F. 2020. Unmanned aerial vehicles applications in future smart cities. *Technological forecasting and social change*, 153, 119293.

- MOHAMMAD, H., KRISHNA, K., GANGADHAR, C. & MOHAMMED, R. 2023. Token Bucket Algorithm with Modernization Techniques to avoid Congestion in DEC Protocol of WSN. *International Journal of Electronics and Telecommunications*, 507-513-507-513.
- MONDAL, M. A. & REHENA, Z. 2022. Priority-Based Adaptive Traffic Signal Control System for Smart Cities. *SN Computer Science*, 3, 417.
- MORIDI, E., HAGHPARAST, M., HOSSEINZADEH, M. & JASSBI, S. J. 2020. Fault management frameworks in wireless sensor networks: A survey. *Computer communications*, 155, 205-226.
- MUBASHAR, R., SIDDIQUE, M. A. B., REHMAN, A. U., ASAD, A. & RASOOL, A. 2021. Comparative performance analysis of short-range wireless protocols for wireless personal area network. *Iran Journal of Computer Science*, 4, 201-210.
- MUGERWA, D., NAM, Y., CHOI, H., SHIN, Y. & LEE, E. 2023. Implicit Overhearing Node-Based Multi-Hop Communication Scheme in IoT LoRa Networks. *Sensors*, 23, 3874.
- MUNTHALI, E. 2024. Performance analysis of VOIP codec schemes and queuing techniques and their impact on FTP and video conferencing. The University of Zambia, 117-123
- NAGARAJU, R., GOYAL, S., VERMA, C., SAFIRESCU, C. O. & MIHALTAN, T. C. 2022. Secure routing-based energy optimization for IOT application with heterogeneous wireless sensor networks. *Energies*, 15, 4777.
- NAIM, M. H., ZAIN, J. M. & ABD JALIL, K. 2022. Fault Tolerance Mechanism for Software Application Through Fog Computing as Middleware. *International Journal of Computing and Digital Systems*, 11, 45-54.
- NAVARRO-ESPINOZA, A., LÓPEZ-BONILLA, O. R., GARCÍA-GUERRERO, E. E., TLELO-CUAUTLE, E., LÓPEZ-MANCILLA, D., HERNÁNDEZ-MEJÍA, C. & INZUNZA-GONZÁLEZ, E. 2022. Traffic flow prediction for smart traffic lights using machine learning algorithms. *Technologies*, 10, 5.
- NEELAKANDAN, S., BERLIN, M., TRIPATHI, S., DEVI, V. B., BHARDWAJ, I. & ARULKUMAR, N. 2021. IoT-based traffic prediction and traffic signal control system for smart city. *Soft Computing*, 25, 12241-12248.
- NI, Y., ZHENG, Z., LIN, X., GAO, F., ZENG, X., LIU, Y., XU, T., WANG, H., ZHANG, Z. & DU, S. CellFusion: Multipath Vehicle-to-Cloud Video Streaming with

- Network Coding in the Wild. Proceedings of the ACM SIGCOMM 2023 Conference, 2023. 668-683.
- NIU, Y., NAZERI, S., HASHIM, W., ALKAHTANI, A. A. A. & ABDULSHAHEED, H. R. 2021. A survey on short-range WBAN communication; technical overview of several standard wireless technologies. *Periodicals of Engineering and Natural Sciences*, 9, 877-885.
- OKADA, S., AKASHI, K., MIYAMOTO, D., SEKIYA, Y., TAKASE, H. & NAKAMURA, H. 2023. Memory-saving LDoS Attacker Detection Algorithms in Zigbee Network. *Journal of Information Processing*, 31, 537-549.
- PARASHAR, V., KASHYAP, R., RIZWAN, A., KARRAS, D. A., ALTAMIRANO, G. C., DIXIT, E. & AHMADI, F. 2022. Aggregation-based dynamic channel bonding to maximise the performance of wireless local area networks (WLAN). *Wireless Communications and Mobile Computing*, 2022.
- PEŞKIRCIOĞLU GÖKÇE, İ., ÜÇÜNCÜ, M. & GÜRAN SCHMIDT, E. 2024. OMNeT++ Simulation Framework for Avionics Full-Duplex Switched Ethernet. *Journal of Aerospace Information Systems*, 21, 443-454.
- POPESCU, M. & NAAJI, A. 2022. Estimation of the satellite bandwidth required for the transmission of information in supervisory control and data acquisition systems. *Applications of Computational Intelligence in Multi-Disciplinary Research*. Elsevier.
- PRABHA, R., SENTHIL, G., SANGEETHA, S., SUGANTHI, S. & ROOPA, D. 2023a. Broadband Wireless Network Era in Wireless Communication–Routing Theory and Practices. *Modeling and Optimization of Optical Communication Networks*, 267.
- PRABHA, R., SENTHIL, G., SANGEETHA, S., SUGANTHI, S. & ROOPA, D. 2023b. Network Routing and Its Real-Time Practice in Broadband Wireless Networks. *Resource Management in Advanced Wireless Networks*, 99, 99.
- RADY, A., EL-RABAIE, E. S. M., SHOKAIR, M. & ABDEL-SALAM, N. 2021. Comprehensive survey of routing protocols for Mobile Wireless Sensor Networks. *International Journal of Communication Systems*, 34, e4942.

- RAGHUNANDAN, S., BAUER, M., ROY, S., KOULAGI, K. & ROHDE, C. Throughput characterization of 5G-NR broadband satellite networks using omnet++ based system level simulator. *IET Conference Proceedings CP828*, 2022. IET, 115-121.
- RAGHUWANSHI, A. K. & CHAUDHARY, V. S. 2022. Energy Efficient Medium Access Control (MAC) Layer Protocol for Wireless Sensor Network. *SAMRIDDHI: A Journal of Physical Sciences, Engineering and Technology*, 14, 270-279.
- RAJAN, M. S., DILIP, G., KANNAN, N., NAMRATHA, M., MAJJI, S., MOHAPATRA, S. K., PATNALA, T. R. & KARANAM, S. R. 2023. Diagnosis of fault node in wireless sensor networks using adaptive neuro-fuzzy inference system. *Applied Nanoscience*, 13, 1007-1015.
- RAMASAMY-RAJESWARI, A., GANAPATHY, S., KULOTHUNGAN, K. & KANNAN, A. 2022. Fuzzy based congestion detection and control algorithm for energy efficient wireless sensor network (WSN). *Journal of the National Science Foundation of Sri Lanka*, 50, 563-576.
- RAGHAVI, T. S., REDDY, B. V. V. & SOWJANYA, P. D. Enabling Seamless Mobility in 5G NR Networks Using NS-3 and 4G Core Integration. 2024 2nd International Conference on Device Intelligence, Computing and Communication Technologies (DICCT), 2024. IEEE, 136-141
- RAVISHANKAR, C., GOPAL, R., BENAMMAR, N., ZAKARIA, G. & HUANG, X. 2021. Next-generation global satellite system with mega-constellations. *International journal of satellite communications and networking*, 39, 6-28.
- REDDY, G. V., THANDAPANI, K., SENDHILKUMAR, N., SENTHILKUMAR, C., HEMANTH, S., PERIANNASAMY, S. M. & HEMANAND, D. 2022. Optimizing QoS-based clustering using a multi-hop with single cluster communication for efficient packet routing. *International Journal of Electrical and Electronics Research*, 10, 69-73.
- RISTIĆ, V. B., TODOROVIĆ, B. M. & STOJANOVIĆ, N. M. 2022. Frequency hopping spread spectrum: History, principles and applications. *Vojnotehnički glasnik/Military Technical Courier*, 70, 856-876.
- SAHAR, G., BAKAR, K. A., RAHIM, S., KHANI, N. A. K. K. & BIBI, T. 2021. Recent advancement of data-driven models in wireless sensor networks: a survey. *Technologies*, 9, 76.

- SAHEB, S. I., KHAN, K. U. R. & BINDU, C. S. 2022. A Hybrid Modified Ant Colony Optimization-Particle Swarm Optimization Algorithm for Optimal Node Positioning and Routing in Wireless Sensor Networks. *International journal of electrical and computer engineering systems*, 13, 515-523.
- SALEEM, M., ABBAS, S., GHAZAL, T. M., KHAN, M. A., SAHAWNEH, N. & AHMAD, M. 2022. Smart cities: Fusion-based intelligent traffic congestion control system for vehicular networks using machine learning techniques. *Egyptian Informatics Journal*, 23, 417-426.
- SAMUYLOV, A., MOLTCHANOV, D., KOVALCHUKOV, R., GAYDAMAKA, A., PYATTAEV, A. & KOUCHERYAVY, Y. 2022. GAR: Gradient assisted routing for topology self-organization in dynamic mesh networks. *Computer Communications*, 190, 10-23.
- SANGEETHA, G., VIJAYALAKSHMI, M., GANAPATHY, S. & KANNAN, A. 2020. An improved congestion-aware routing mechanism in sensor networks using fuzzy rule sets. *Peer-to-Peer Networking and Applications*, 13, 890-904.
- SHAFI, I., ASHRAF, M. J., CHOI, G. S., DIN, S. & ASHRAF, I. 2022. Intelligent autonomous underwater vehicle mobility with energy efficient routing in sensor networks. *Environment, Development and Sustainability*, 1-13.
- SHAIKH, F. K., KARIM, S., ZEADALLY, S. & NEBHEN, J. 2022. Recent trends in internet of things enabled sensor technologies for smart agriculture. *IEEE Internet of Things Journal*.
- SHARAN, S., ZHENG, W., HSU, K.-F., XING, J., CHEN, A. & WANG, Z. 2022. Symbolic Distillation for Learned TCP Congestion Control. *Advances in Neural Information Processing Systems*, 35, 10684-10695.
- SHARMA, R., VASHISHT, V. & SINGH, U. 2020. Modelling and simulation frameworks for wireless sensor networks: a comparative study. *IET Wireless Sensor Systems*, 10, 181-197.
- SHARMA, S., KAUL, A., AHMED, S. & SHARMA, S. 2021. A detailed tutorial survey on VANETs: Emerging architectures, applications, security issues, and solutions. *International Journal of Communication Systems*, 34, e4905.
- SHARMA, S. & VERMA, V. K. 2022. An integrated exploration on internet of things and wireless sensor networks. *Wireless Personal Communications*, 124, 2735-2770.

- SIDAM, A., KOUTARAPU, P., METHUKU, M. & VUYVALA, S. A comparative analysis on benefits of unscheduled automatic power save delivery over legacy power save mode in IEEE 802.11 wireless networks. 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom), 2022. IEEE, 551-555.
- SINGAL, G., LAXMI, V., GAUR, M. S., RAO, D. V., KUSHWAHA, R., GARG, D. & KUMAR, N. 2021. QoS-aware mesh-based multicast routing protocols in edge ad hoc networks: Concepts and challenges. *ACM Transactions on Internet Technology (TOIT)*, 22, 1-27.
- SINGH, S., PISE, A., ALFARRAJ, O., TOLBA, A. & YOON, B. 2022. A cryptographic approach to prevent network incursion for enhancement of QoS in sustainable smart city using MANET. *Sustainable Cities and Society*, 79, 103483.
- SRPAK, I., HAVAŠ, L., HORVAT, T. & TOMIČIĆ, E. Aspects and roles of different programming languages and their use in stem education. Edulearn23 Proceedings, 2023. IATED, 2944-2953.
- SU, Y., JIANG, P., CHEN, H. & DENG, X. 2022. A QoS-Guaranteed and Congestion-Controlled SDN Routing Strategy for Smart Grid. *Applied Sciences*, 12, 7629.
- SUBRAMANI, N., MARDANI, A., MISHRA, A. R. & EZHUMALAI, P. 2023. A fuzzy logic and DEEC protocol-based clustering routing method for wireless sensor networks. *AIMS Math*, 8, 8310-8331.
- SUBRAMANIAM, M., RAMBABU, C., CHANDRASEKARAN, G. & KUMAR, N. S. 2022. A traffic density-based congestion control method for VANETs. *Wireless Communications and Mobile Computing*, 2022.
- SUMA, S. & HARSOOR, B. Detection of malicious activity for mobile nodes to avoid congestion in Wireless Sensor Network. 2022 IEEE fourth international conference on advances in electronics, computers and communications (ICAECC), 2022. IEEE, 1-6.
- SUN, Y., PENG, M., ZHANG, S., LIN, G. & ZHANG, P. 2022. Integrated satellite-terrestrial networks: architectures, key techniques, and experimental progress. *IEEE Network*, 36, 191-198.
- SUNASSEE, S., MUNGUR, A., ARMOOGUM, S. & PUDARUTH, S. A comprehensive review on congestion control techniques in networking. 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), 2021. IEEE, 305-312.

- SUTHAR, N. & BHAVSAR, A. Simulation and Performance Evaluation of WSN Based Wild Animal Protection Architecture Using NS-2. *International Conference on Computational Intelligence*, 2023. Springer, 699-707.
- SVERTOKA, E., SAAFI, S., RUSU-CASANDRA, A., BURGET, R., MARGHESCU, I., HOSEK, J. & OMETOV, A. 2021. Wearables for industrial work safety: A survey. *Sensors*, 21, 3844.
- SZOTT, S., KOSEK-SZOTT, K., GAWŁOWICZ, P., GÓMEZ, J. T., BELLALTA, B., ZUBOW, A. & DRESSLER, F. 2022. Wi-Fi meets ML: A survey on improving IEEE 802.11 performance with machine learning. *IEEE Communications Surveys & Tutorials*, 24, 1843-1893.
- TAAMI, T., AZIZI, S. & YARINEZHAD, R. 2023. An efficient route selection mechanism based on network topology in battery-powered internet of things networks. *Peer-to-Peer Networking and Applications*, 16, 450-465.
- TAMPUU, T., PRAKS, J., DE ZAN, F., KOHV, M. & KULL, A. 2023. Relationship between ground levelling measurements and radar satellite interferometric estimates of bog breathing in ombrotrophic northern bogs. *MIRES AND PEAT*, 29, 17.
- TANVEER, H., BALZ, T., CIGNA, F. & TAPETE, D. 2020. Monitoring 2011–2020 traffic patterns in Wuhan (China) with COSMO-SkyMed SAR, amidst the 7th CISM military world games and COVID-19 outbreak. *Remote Sensing*, 12, 1636.
- TENG, H., DONG, M., LIU, Y., TIAN, W. & LIU, X. 2021. A low-cost physical location discovery scheme for large-scale Internet of Things in smart city through joint use of vehicles and UAVs. *Future generation computer systems*, 118, 310-326.
- THEEYANCHERI, L., CHAKI, S., BHATTACHARJEE, T. & CHAKRABARTI, R. 2023. Active dynamics of linear chains and rings in porous media. *The Journal of Chemical Physics*, 159.
- TIAN, P., BOANO, C. A., MA, X. & WEI, J. 2023. LoRaHop: Multi-Hop Support for LoRaWAN Uplink and Downlink Messaging. *IEEE Internet of Things Journal*.
- TOLOUEIASHTIAN, M., GOLSORKHTABARAMIRI, M. & RAD, S. Y. B. 2022. An improved whale optimization algorithm solving the point coverage problem in wireless sensor networks. *Telecommunication Systems*, 79, 417-436.
- TRINH, C., HUYNH, B., BIDAHI, M., RAHMANI, A. M., HOSSEINZADEH, M. & MASDARI, M. 2022. Optimized fuzzy clustering using moth-flame optimization

- algorithm in wireless sensor networks. *Artificial Intelligence Review*, 55, 1915-1945.
- UMAR, A., KHALID, Z., ALI, M., ABAZEED, M., ALQAHTANI, A., ULLAH, R. & SAFDAR, H. 2023. A Review on Congestion Mitigation Techniques in Ultra-Dense Wireless Sensor Networks: State-of-the-Art Future Emerging Artificial Intelligence-Based Solutions. *Applied Sciences*, 13, 12384.
- VAL, I., SEIJO, Ó., TORREGO, R. & ASTARLOA, A. 2021. IEEE 802.1 AS clock synchronization performance evaluation of an integrated wired–wireless TSN architecture. *IEEE Transactions on Industrial Informatics*, 18, 2986-2999.
- VANITHA, G., AMUDHA, P. & SIVAKUMARI, S. 2022. Analysis of Algorithms to Control the Congestion by Improve Energy Efficiency in WSN. *ECS Transactions*, 107, 5191.
- VEERAKUMARI, K. P. & RESMI, R. 2017. Evaluation of continuous sampling plan (csp-5) parameters using gert technique and matlab. *Journal of Applied Mathematics, Statistics and Informatics*, 13, 63-75.
- VIJAYALAKSHMI, P., SARAVANAN, M., RANI, M. T., ASHOK, M., PALANIAPPAN, R. & NAGARAJ, V. A Novel Hybridclustering Model Forwireless Sensor Networks. 2023 International Conference on Artificial Intelligence and Knowledge Discovery in Concurrent Engineering (ICECONF), 2023. IEEE, 1-8.
- VINEELA, M., KRISHNA, V. R., RANI, K. S., SINGH, B. R. N., VARSHA, K. S. & VAISHNAVI, K. V. S. Implementation of Internet of Things in Wireless Sensor Networks for Environmental Monitoring. 2023 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI), 2023. IEEE, 1-6.
- WALIA, A. K., CHHABRA, A. & SHARMA, D. 2022. Comparative analysis of contemporary network simulators. *Innovative Data Communication Technologies and Application: Proceedings of ICIDCA 2021*. Springer.
- WAN, Q., XIE, Y., JI, W. & YAN, Z. 2021. High gain and low noise wideband folded-switching mixer employing transformer and complimentary CS/CG hybrid topology. *Microelectronics Journal*, 114, 105112.
- WAN, S., DING, S. & CHEN, C. 2022. Edge computing enabled video segmentation for real-time traffic monitoring in internet of vehicles. *Pattern Recognition*, 121, 108146.

- WANG, D., YI, Y., YAN, S., WAN, N. & ZHAO, J. 2023. A node trust evaluation method of vehicle-road-cloud collaborative system based on federated learning. *Ad Hoc Networks*, 138, 103013.
- WANG, M. & ZHOU, T. 2022. Understanding the dynamic relationship between smart city implementation and urban sustainability. *Technology in Society*, 70, 102018.
- WANG, T., KAN, H., SUN, Q., XIAO, S. & WANG, S. Congestion Detection and Link Control via Feedback in RDMA Transmission. 2022 International Conference on Service Science (ICSS), 2022. IEEE, 1-4.
- WRONA, A., DE SANTIS, E., PRISCOLI, F. D. & LAVACCA, F. G. An Intelligent Ground Station Selection Algorithm in Satellite Optical Communications via Deep Learning. 2023 31st Mediterranean Conference on Control and Automation (MED), 2023. IEEE, 493-499.
- WU, C., LU, H., CHEN, Y. & QIN, L. 2023. Cross-Layer Optimization for Statistical QoS Provision in C-RAN with Finite-Length Coding. *arXiv preprint arXiv:2311.09879*.
- WU, L., SHI, S., WANG, G., MOU, P., LIU, X., LIU, J., LI, L. & DU, C. 2022a. Carbon nanocoils/carbon foam as the dynamically frequency-tunable microwave absorbers with an ultrawide tuning range and absorption bandwidth. *Advanced Functional Materials*, 32, 2209898.
- WU, X., WU, D. & MODIANO, E. Queueing Delay Minimization in Overloaded Networks via Rate Control. 2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton), 2022b. IEEE, 1-8.
- XU, C., XIONG, Z., ZHAO, G. & YU, S. 2019. An energy-efficient region source routing protocol for lifetime maximization in WSN. *IEEE Access*, 7, 135277-135289.
- YADAV, S. L., UJJWAL, R., KUMAR, S., KAIWARTYA, O., KUMAR, M. & KASHYAP, P. K. 2021. Traffic and energy aware optimization for congestion control in next generation wireless sensor networks. *Journal of Sensors*, 2021, 1-16.
- YAHYAOU, H., ZHANI, M. F., BOUACHIR, O. & ALOQAILY, M. 2023. On minimizing flow monitoring costs in large-scale software-defined network networks. *International Journal of Network Management*, 33, e2220.
- YAKYMCHUK, N., SELEPYNA, Y., YEVSIIUK, M., PRYSTUPA, S. & MOROZ, S. 2022. Monitoring of link-level congestion in telecommunication systems using information criteria. *Informatyka, automatyka, pomiary w gospodarce i ochronie środowiska*, 12.

- YI, Z., LIU, X. C., MARKOVIC, N. & PHILLIPS, J. 2021. Inferencing hourly traffic volume using data-driven machine learning and graph theory. *Computers, Environment and Urban Systems*, 85, 101548.
- YING, Z., SI-NING, W., CHANG-YU, D. & YUAN-KUN, J. 2022. Research on Application of Wireless Bridge Technology in the Smart Grid. *Research Anthology on Smart Grid and Microgrid Development*. IGI Global.
- YOUSIF, Z., HUSSAIN, I., DJAHEL, S. & HADJADJ-AOUL, Y. 2021. A novel energy-efficient clustering algorithm for more sustainable wireless sensor networks enabled smart cities applications. *Journal of sensor and actuator networks*, 10, 50.
- ZHANG, R., ZHANG, J., LIU, L., YAN, Z. & DONG, M. Research on Active Detection Method of Network Congestion. GLOBECOM 2022-2022 IEEE Global Communications Conference, 2022. IEEE, 4685-4690.

APPENDIX A: TABLE OF NOTIONS

v_i	Set of vehicles in motion
P_i	Vehicles which are in stationary
T_P	Estimated total number of vehicles
C_n	Classified number of vehicles detected in the incoming lanes
D_P	Delay caused by the incoming vehicles
t_s	Time per second
X	Time series of the length
n	Observation sequence
T_t	Average time calculated
AA_c	Absolute amount
$\tilde{\mu}$	Short-term
μ	Long-term
$\tilde{w}$	Sliding window protocol
DR	Detection Rate percentage
FAR	False Alarm Rate percentage

N	Number of detected congestions
t_c	Alarm time
t_r	Time when the congestion appeared
ML_{ac}	Machine learning technique

APPENDIX B: TCL SCRIPT

```
# Traffic Congestion Detection Simulation in TCL
# Set simulation parameters
set SIMULATION_DURATION 1000 ;# in seconds
set NUM_CARS 20 ;# Number of cars (OBUs)
set NUM_RSUS 8 ;# Number of Road-Side Units (RSUs)
set CONGESTION_THRESHOLD 70 ;# percentage of utilization

# Create a procedure to generate random integer between min and max
proc random_int {min max} {
    expr {int(rand() * ($max - $min + 1) + $min)}
}

# Create data storage arrays
array set detection_rates {}
array set false_alarm_rates {}
array set mean_detection_times {}

# Procedure to simulate traffic for algorithms
proc simulate_traffic {algorithm_name rsus cars duration congestion_threshold} {
    set total_events 0
    set congestion_events 0
    set false_alarms 0
    set detection_times {}

    for {set t 0} {$t < $duration} {incr t} {
        for {set rsu 0} {$rsu < $rsus} {incr rsu} {
            set cars_per_rsu [expr {$cars / $rsus}]

            for {set car 0} {$car < $cars_per_rsu} {incr car} {
                set utilization [random_int 0 100]
                incr total_events

                switch $algorithm_name {
                    "PBCC" {
                        # More sensitive detection
                        if {$utilization > [expr {$congestion_threshold - 5}]} {
                            incr congestion_events
                            lappend detection_times $t
                        } elseif {[expr {rand()}] < 0.02} {
                            incr false_alarms
                        }
                    }
                    "FITCC" {
                        # Less sensitive detection
                        if {$utilization > [expr {$congestion_threshold + 10}]} {
                            incr congestion_events
                            lappend detection_times $t
                        } elseif {[expr {rand()}] < 0.05} {
                            incr false_alarms
                        }
                    }
                    "TCD" {
                        # Moderate sensitivity
                        if {$utilization > [expr {$congestion_threshold + 5}]} {
                            incr congestion_events
                        }
                    }
                }
            }
        }
    }
}
```

```

        lappend detection_times $t
    } elseif {[expr {rand()}] < 0.04} {
        incr false_alarms
    }
}
}
}
}
}

# Calculate metrics
set detection_rate [expr {double($congestion_events) / $total_events * 100}]
set false_alarm_rate [expr {double($false_alarms) / $total_events * 100}]

# Calculate mean detection time
if {[llength $detection_times] > 0} {
    set mean_detection_time [expr {double([::tcl::mathop::+ {*}]$detection_times) /
[llength $detection_times]}]
} else {
    set mean_detection_time 0
}

return [list $detection_rate $false_alarm_rate $mean_detection_time]
}

# Initialize random number generator
expr {srand([clock seconds])}

# List of algorithms to simulate
set algorithms [list "PBCC" "FITCC" "TCD"]

# Run simulations for each algorithm
foreach algorithm $algorithms {
    puts "\nSimulating $algorithm..."

    set metrics [simulate_traffic $algorithm $NUM_RSUS $NUM_CARS
$SIMULATION_DURATION $CONGESTION_THRESHOLD]
    lassign $metrics detection_rate false_alarm_rate mean_detection_time

    # Store results
    set detection_rates($algorithm) $detection_rate
    set false_alarm_rates($algorithm) $false_alarm_rate
    set mean_detection_times($algorithm) $mean_detection_time

    # Display results
    puts "Results of $algorithm Algorithm Simulation:"
    puts [format "Detection Rate (DR): %.2f%%" $detection_rate]
    puts [format "False Alarm Rate (FAR): %.2f%%" $false_alarm_rate]
    puts [format "Mean Time to Detection (MTTD): %.2f seconds"
$mean_detection_time]
}

# Optional: Export results to a file
set output_file [open "simulation_results.txt" w]
puts $output_file "Traffic Congestion Detection Simulation Results"
puts $output_file "=====

```

```
puts $output_file "\nSimulation Parameters:"
puts $output_file "Duration: $SIMULATION_DURATION seconds"
puts $output_file "Number of Cars: $NUM_CARS"
puts $output_file "Number of RSUs: $NUM_RSUS"
puts $output_file "Congestion Threshold: $CONGESTION_THRESHOLD%"
puts $output_file "\nResults:"

foreach algorithm $algorithms {
    puts $output_file "\n$algorithm Algorithm:"
    puts $output_file [format "Detection Rate: %.2f%%"
$detection_rates($algorithm)]
    puts $output_file [format "False Alarm Rate: %.2f%%"
>false_alarm_rates($algorithm)]
    puts $output_file [format "Mean Time to Detection: %.2f seconds"
$mean_detection_times($algorithm)]
}

close $output_file
```

APPENDIX C: PYTHON CODE

```
import numpy as np
import matplotlib.pyplot as plt
import random
# Parameters
SIMULATION_DURATION = 1000 # in seconds
NUM_CARS = 20 # Number of cars (OBUs)
NUM_RSUS = 8 # Number of Road-Side Units (RSUs)
CONGESTION_THRESHOLD = 70 # percentage of utilization
def simulate_traffic(algorithm_name, rsus, cars, duration, congestion_threshold):
    """
    Simulate network traffic for different congestion control algorithms.
    Args:
    algorithm_name (str): Name of the algorithm to be simulated
    rsus (int): Number of RSUs in the network
    cars (int): Number of cars (OBUs) in the network
    duration (int): Duration of the simulation in seconds
    congestion_threshold (int): Utilization threshold for congestion
    Returns:
    dict: Metrics - detection rate, false alarm rate, mean time to detection
    """
    total_events = 0
    congestion_events = 0
    false_alarms = 0
    detection_times = []
    for t in range(duration):
        for rsu in range(rsus):
            cars_per_rsu = cars // rsus
            for car in range(cars_per_rsu):
                utilization = random.randint(0, 100)
                total_events += 1

                if algorithm_name == 'PBCC':
                    # Adjusted behavior to be more effective
                    if utilization > (congestion_threshold - 5): # More sensitive
                        congestion_events += 1
                        detection_times.append(t)
                    elif random.random() < 0.02: # Lower false alarm rate
                        false_alarms += 1
                elif algorithm_name == 'FITCC':
                    if utilization > (congestion_threshold + 10): # Less sensitive
                        congestion_events += 1
                        detection_times.append(t)
                    elif random.random() < 0.05: # Higher false alarm rate
                        false_alarms += 1
                elif algorithm_name == 'TCD':
                    if utilization > (congestion_threshold + 5): # Moderate sensitivity
                        congestion_events += 1
                        detection_times.append(t)
                    elif random.random() < 0.04: # Moderate false alarm rate
                        false_alarms += 1
    detection_rate = congestion_events / total_events * 100
    false_alarm_rate = false_alarms / total_events * 100
    mean_detection_time = np.mean(detection_times) if detection_times else 0
    return {
```

```

        'algorithm': algorithm_name,
        'detection_rate': detection_rate,
        'false_alarm_rate': false_alarm_rate,
        'mean_detection_time': mean_detection_time
    }
# Run simulations for each algorithm
algorithms = ['PBCC', 'FITCC', 'TCD']
metrics_list = []
for algorithm in algorithms:
    metrics = simulate_traffic(algorithm, NUM_RSUS, NUM_CARS,
SIMULATION_DURATION, CONGESTION_THRESHOLD)
    metrics_list.append(metrics)

# Display results
for metrics in metrics_list:
    print(f"Results of {metrics['algorithm']} Algorithm Simulation:\n")
    print(f"Detection Rate (DR): {metrics['detection_rate']:.2f}%")
    print(f"False Alarm Rate (FAR): {metrics['false_alarm_rate']:.2f}%")
    print(f"Mean Time to Detection (MTTD): {metrics['mean_detection_time']:.2f}
seconds\n")
# Visualization of results
labels = ['PBCC', 'FITCC', 'TCD']
detection_rates = [metrics['detection_rate'] for metrics in metrics_list]
false_alarm_rates = [metrics['false_alarm_rate'] for metrics in metrics_list]
mean_detection_times = [metrics['mean_detection_time'] for metrics in metrics_list]
# Plot Detection Rate Comparison
plt.figure(figsize=(10, 6))
plt.bar(labels, detection_rates, color='green')
plt.ylabel('Detection Rate (%)')
plt.title('Comparison of Detection Rate between Algorithms')
plt.show()
# Plot False Alarm Rate Comparison
plt.figure(figsize=(10, 6))
plt.bar(labels, false_alarm_rates, color='red')
plt.ylabel('False Alarm Rate (%)')
plt.title('Comparison of False Alarm Rate between Algorithms')
plt.show()
# Plot Mean Time to Detection Comparison
plt.figure(figsize=(10, 6))
plt.bar(labels, mean_detection_times, color='blue')
plt.ylabel('Mean Time to Detection (s)')
plt.title('Comparison of Mean Time to Detection between Algorithms')
plt.show()

```

APPENDIX D: R CODE

This research study also used R Code to verify the results generated through Python.

```
#=====
# R code to graphically display the average output
#=====
autos_data <- read.table("D:\\Research\\2021\\Ndivhuho\\Results
Analysis\\TP.dat", header=T, sep="\t")
# Load required libraries
library(ggplot2)
library(dplyr)
library(tidyr)
library(gridExtra)
# Function to simulate traffic for different weather conditions
simulate_traffic <- function(algorithm, num_obus, weather_condition) {
  # Simulation parameters
  total_events <- 1000
  congestion_threshold <- 70
  # Adjust parameters based on weather condition
  weather_factor <- if(weather_condition == "rainy") 1.2 else 1.0
  # Initialize counters
  congestion_events <- 0
  false_alarms <- 0
  detection_times <- numeric()
  # Simulation logic
  for(t in 1:total_events) {
    utilization <- runif(1, 0, 100) * weather_factor
    # Algorithm-specific behavior
    if(algorithm == "PBCC") {
      if(weather_condition == "rainy") {
        if(utilization > (congestion_threshold - 5)) {
          congestion_events <- congestion_events + 1
          detection_times <- c(detection_times, t)
        } else if(runif(1) < 0.02) {
          false_alarms <- false_alarms + 1
        }
      } else { # sunny
        if(utilization > (congestion_threshold - 10)) {
          congestion_events <- congestion_events + 1
          detection_times <- c(detection_times, t)
        } else if(runif(1) < 0.01) {
          false_alarms <- false_alarms + 1
        }
      }
    } else if(algorithm == "FITCC") {
      if(weather_condition == "rainy") {
        if(utilization > (congestion_threshold + 10)) {
          congestion_events <- congestion_events + 1
          detection_times <- c(detection_times, t)
        } else if(runif(1) < 0.05) {
```

```

        false_alarms <- false_alarms + 1
    }
} else { # sunny
    if(utilization > (congestion_threshold + 5)) {
        congestion_events <- congestion_events + 1
        detection_times <- c(detection_times, t)
    } else if(runif(1) < 0.03) {
        false_alarms <- false_alarms + 1
    }
}
} else if(algorithm == "TCD") {
    if(weather_condition == "rainy") {
        if(utilization > (congestion_threshold + 5)) {
            congestion_events <- congestion_events + 1
            detection_times <- c(detection_times, t)
        } else if(runif(1) < 0.04) {
            false_alarms <- false_alarms + 1
        }
    } else { # sunny
        if(utilization > congestion_threshold) {
            congestion_events <- congestion_events + 1
            detection_times <- c(detection_times, t)
        } else if(runif(1) < 0.02) {
            false_alarms <- false_alarms + 1
        }
    }
}
}
}
# Calculate metrics
detection_rate <- (congestion_events / total_events) * 100
false_alarm_rate <- (false_alarms / total_events) * 100
mean_detection_time <- if(length(detection_times) > 0) mean(detection_times)
else 0
return(c(detection_rate = detection_rate,
        false_alarm_rate = false_alarm_rate,
        mean_detection_time = mean_detection_time))
}
# Generate data for different numbers of OBUs
num_obus_range <- c(5, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
algorithms <- c("PBCC", "FITCC", "TCD")
weather_conditions <- c("rainy", "sunny")

# Initialize results dataframe
results <- data.frame()
# Run simulations
for(weather in weather_conditions) {
    for(num_obus in num_obus_range) {
        for(algo in algorithms) {
            metrics <- simulate_traffic(algo, num_obus, weather)
            results <- rbind(results, data.frame(
                weather = weather,
                num_obus = num_obus,

```

```

    algorithm = algo,
    detection_rate = metrics["detection_rate"],
    false_alarm_rate = metrics["false_alarm_rate"],
    mean_detection_time = metrics["mean_detection_time"]
  ))
}
}
}
# Plotting function
create_plots <- function(data, weather_type) {
  # Detection Rate Plot
  p1 <- ggplot(data, aes(x = num_obus, y = detection_rate, color = algorithm)) +
    geom_line(linetype = "dashed") +
    geom_point() +
    scale_color_manual(values = c("PBCC" = "#20B2AA", "FITCC" = "#FF69B4",
    "TCD" = "#4169E1")) +
    labs(title = "Detection Rate",
         x = "Number of OBUs",
         y = "Detection Rate (%)") +
    theme_minimal() +
    ylim(0, 100)

  # False Alarm Rate Plot
  p2 <- ggplot(data, aes(x = num_obus, y = false_alarm_rate, color = algorithm))
  +
    geom_line(linetype = "dashed") +
    geom_point() +
    scale_color_manual(values = c("PBCC" = "#20B2AA", "FITCC" = "#FF69B4",
    "TCD" = "#4169E1")) +
    labs(title = "False Alarm Rate",
         x = "Number of OBUs",
         y = "False Alarm Rate (%)") +
    theme_minimal()

  # Mean Time to Detection Plot
  p3 <- ggplot(data, aes(x = num_obus, y = mean_detection_time, color =
algorithm)) +
    geom_line(linetype = "dashed") +
    geom_point() +
    scale_color_manual(values = c("PBCC" = "#20B2AA", "FITCC" = "#FF69B4",
    "TCD" = "#4169E1")) +
    labs(title = "Mean Time to Detection",
         x = "Number of OBUs",
         y = "Mean Time to Detection (s)") +
    theme_minimal()

  # Combine plots
  grid.arrange(p1, p2, p3, ncol = 1,
               top = paste("Simulation Results Visualization -",
                           ifelse(weather_type == "rainy",
                                "Rainy Day Conditions",
                                "Sunny Day Conditions"))))
}
# Create plots for each weather condition

```

```
rainy_data <- subset(results, weather == "rainy")
sunny_data <- subset(results, weather == "sunny")
# Save plots
pdf("rainy_day_results.pdf", height = 12, width = 8)
create_plots(rainy_data, "rainy")
dev.off()
pdf("sunny_day_results.pdf", height = 12, width = 8)
create_plots(sunny_data, "sunny")
dev.off()
# Print summary statistics
summary_stats <- results %>%
  group_by(weather, algorithm) %>%
  summarize(
    avg_detection_rate = mean(detection_rate),
    avg_false_alarm_rate = mean(false_alarm_rate),
    avg_detection_time = mean(mean_detection_time)
  )
print("Summary Statistics:")
print(summary_stats)
```